Collaborative Protection Profile for Network Devices

Version: 4.0
2025-11-25
National Information Assurance Partnership

Revision History

Version	Date	Comment
0.1	2014-09-05	Draft published for Public review
0.2	2014-10-13	Internal draft in response to public review comments, for iTC review
0.3	2014-10-17	Draft version released to accompany CCDB review of Supporting Document.
0.4	2015-01-26	Incorporated comments received from the CCDB review
1.0	2015-02-27	Released for use
1.1	2016-07-21	Updated draft published for public review
2.0	2017-05-05	Released for use
2.1	2018-09-24	Released for use
2.2	2019-12-20	Released for use
2.2e	2020-03-23	Released for use
3.0	2023-04-06	Incorporated comments received. Released for use
3.0e	2023-12-06	Released for use
4.0	2025-11-25	Released for use

Contents

1 Introduction

1.1 Overview

1.2 Terms

The following sections list Common Criteria and technology terms used in this document.

1.2.1 Common Criteria Terms

Assurance	Grounds for confidence that a TOE meets the SFRs [CC].
Base Protection Profile (Base-PP)	Protection Profile used as a basis to build a PP-Configuration.
Collaborative Protection Profile (cPP)	A Protection Profile developed by international technical communities and approved by multiple schemes.
Common Criteria (CC)	Common Criteria for Information Technology Security Evaluation (International Standard ISO/IEC 15408).
Common Criteria Testing Laboratory	Within the context of the Common Criteria Evaluation and Validation Scheme (CCEVS), an IT security evaluation facility accredited by the National Voluntary Laboratory Accreditation Program (NVLAP) and approved by the NIAP Validation Body to conduct Common Criteria-based evaluations.
Common Evaluation Methodology (CEM)	Common Evaluation Methodology for Information Technology Security Evaluation.
Direct Rationale	A type of Protection Profile, PP-Module, or Security Target in which the security problem definition (SPD) elements are mapped directly to the SFRs and possibly to the security objectives for the operational environment. There are no security objectives for the TOE.
Distributed TOE	A TOE composed of multiple components operating as a logical whole.
Extended Package (EP)	A deprecated document form for collecting SFRs that implement a particular protocol, technology, or functionality. See Functional Packages.
Functional Package (FP)	A document that collects SFRs for a particular protocol, technology, or functionality.
Operational Environment (OE)	Hardware and software that are outside the TOE boundary that support the TOE functionality and security policy.
Protection Profile (PP)	An implementation-independent set of security requirements for a category of products.

Protection Profile Configuration (PP-Configuration)	A comprehensive set of security requirements for a product type that consists of at least one Base-PP and at least one PP-Module.
Protection Profile Module (PP-Module)	An implementation-independent statement of security needs for a TOE type complementary to one or more Base-PPs.
Security Assurance Requirement (SAR)	A requirement to assure the security of the TOE.
Security Functional Requirement (SFR)	A requirement for security enforcement by the TOE.
Security Target (ST)	A set of implementation-dependent security requirements for a specific product.
Target of Evaluation (TOE)	The product under evaluation.
TOE Security Functionality (TSF)	The security functionality of the product under evaluation.
TOE Summary Specification (TSS)	A description of how a TOE satisfies the SFRs in an ST.

1.2.2 Technical Terms

1.3 Compliant Targets of Evaluation

1.4 TOE Features

1.4.1 Password-based remote administration

If the TOE provides remote administration using a password-based authentication mechanism, certain SFRs in this PP must be included.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FIA_AFL.1**

1.4.2 Password-based local authentication

If the TOE provides a password-based local authentication mechanism, certain SFRs in this PP must be included.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FIA_UAU.7**

1.4.3 Password-based authentication

If the TOE provides a password-based authentication mechanism, certain SFRs in this PP must be included.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FIA_PMG_EXT.1**
- **FPT_APW_EXT.1**

1.4.4 Pre-shared keys for IPsec

The TOE may support pre-shared keys for use in the IPsec protocol that conform to RFC 8784. If so, certain SFRs in this PP must be included.

If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FIA_PSK_EXT.1**

1.4.5 Admin management of crypto keys

True if cryptographic keys can be managed (e.g., modified, deleted or generated/imported) by the Security Administrator.

If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FMT_MTD.1/CryptoKeys**

1.4.6 Local security admin

True if the TOE provides the Security Administrator the ability to administer the TOE locally.

If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FTA_SSL_EXT.1**

1.4.7 Distributed TOE

True if the TOE is implemented as a Distributed TOE as described in the Distributed TOE section of this cPP.

If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

1.5 PP Overview

words

1.6 TOE Use Cases

Words

2 Conformance Claims

Conformance Statement

An ST must claim exact conformance to this PP.

The evaluation methods used for evaluating the TOE are a combination of the workunits defined in [CEM] as well as the Evaluation Activities for ensuring that individual SFRs and SARs have a sufficient level of supporting evidence in the Security Target and guidance documentation and have been sufficiently tested by the laboratory as part of completing ATE_IND.1. Any functional packages this PP claims similarly contain their own Evaluation Activities that are used in this same manner.

CC Conformance Claims

This PP is conformant to Part 2 (conformant) and Part 3 (conformant) of Common Criteria CC:2022, Revision 1.

PP Claim

This PP does not claim conformance to any Protection Profile.

There are no PPs or PP-Modules that are allowed in a PP-Configuration with this PP.

Package Claim

- This PP does not conform to any functional packages.
- This PP is Functional Package for Secure Shell Version 2.0 conformant.
- This PP is Functional Package for Transport Layer Security Version 2.1 conformant.
- This PP is Functional Package for X.509 Version 1.0 conformant.

The functional packages to which the PP conforms may include SFRs that are not mandatory to claim for the sake of conformance. An ST that claims one or more of these functional packages may include any non-mandatory SFRs that are appropriate to claim based on the capabilities of the TSF and on any triggers for their inclusion based inherently on the SFR selections made.

3 Security Problem Description

A Network Device has a network infrastructure role that it is designed to provide. In doing so, the Network Device communicates with other Network Devices and other network entities (i.e. entities not defined as Network Devices because they do not have an infrastructure role) over the network. At the same time, it must provide a minimal set of common security functionality expected by all Network Devices. The security problem to be addressed by a compliant Network Device is defined as this set of common security functionality that addresses the threats that are common to Network Devices, as opposed to those that might be targeting the specific functionality of a specific type of Network Device. The set of common security functionality addresses communication with the Network Device, both authorized and unauthorized, the ability to perform valid and secure updates, the ability to audit device activity, the ability to securely store and utilize device and Administrator credentials and data, and the ability to self-test critical device components for failures.

3.1 Threats

The threats for the Network Device are grouped according to functional areas of the device in the sections below. The description of each threat is then followed by a rationale describing how it is addressed by the SFRs in Section 6, Annex A, and Annex B.

T.SECURITY_FUNCTIONALITY_COMPROMISE

Threat agents may compromise credentials and device data enabling continued access to the Network Device and its critical data. The compromise of credentials includes replacing existing credentials with an attacker's credentials, modifying existing credentials, or obtaining the Administrator or device credentials for use by the attacker. Threat agents may also be able to take advantage of weak administrative passwords to gain privileged access to the device.

T.SECURITY_FUNCTIONALITY_FAILURE

An external, unauthorised entity could make use of failed or compromised security functionality and might therefore subsequently use or abuse security functions without prior authentication to access, change or modify device data, critical network traffic or security functionality of the device.

T.UNAUTHORISED_ADMINISTRATOR_ACCESS

Threat agents may attempt to gain Administrator access to the Network Device by nefarious means such as masquerading as an Administrator to the device, masquerading as the device to an Administrator, replaying an administrative session (in its entirety, or selected portions), or performing man-in-the-middle attacks, which would provide access to the administrative session, or sessions between Network Devices. Successfully gaining Administrator access allows malicious actions that compromise the security functionality of the device and the network on which it resides.

T.UNDETECTED_ACTIVITY

Threat agents may attempt to access, change, and/or modify the security functionality of the Network Device without Administrator awareness. This could result in the attacker finding an avenue (e.g., misconfiguration, flaw in the product) to compromise the device and the Administrator would have no knowledge that the device has been compromised.

T.UNTRUSTED_COMMUNICATIONS_CHANNELS

Threat agents may attempt to target Network Devices that do not use standardized secure tunnelling protocols to protect the critical network traffic. Attackers may take advantage of poorly designed protocols or

poor key management to successfully perform man-in-the-middle attacks, replay attacks, etc. Successful attacks will result in loss of confidentiality and integrity of the critical network traffic and potentially could lead to a compromise of the Network Device itself.

T.UPDATE_COMPROMISE

Threat agents may attempt to provide a compromised update of the software or firmware which undermines the security functionality of the device. Non-validated updates or updates validated using non-secure or weak cryptography leave the update firmware vulnerable to surreptitious alteration.

T.WEAK_AUTHENTICATION_ENDPOINTS

Threat agents may take advantage of secure protocols that use weak methods to authenticate the endpoints, e.g., a shared password that is guessable or transported as plaintext. The consequences are the same as a poorly designed protocol, the attacker could masquerade as the Administrator or another device, and the attacker could insert themselves into the network stream and perform a man-in-the-middle attack. The result is the critical network traffic is exposed and there could be a loss of confidentiality and integrity, and potentially the Network Device itself could be compromised.

T.WEAK_CRYPTOGRAPHY

Threat agents may exploit weak cryptographic algorithms or perform an exhaustive search against a weak cryptographic key to gain access to critical security parameters or manipulate TSF data. Flawed or deprecated cryptographic algorithms, insecure modes of operation, predictable pseudorandom numbers, and too-small key sizes might allow attackers to compromise secure communications, gain unauthorised access, or allow to intercept and decrypt sensitive data.

3.2 Assumptions

This section describes the assumptions made in identification of the threats and security requirements for Network Devices. The Network Device is not expected to provide assurance in any of these areas, and as a result, requirements are not included to mitigate the threats associated.

A.ADMIN_CREDENTIALS_SECURE

The Administrator’s credentials (private keys) used to access the Network Device are protected by the platform on which they reside.

A.COMPONENTS_RUNNING (applies to distributed TOEs only)

For distributed TOEs, it is assumed that the availability of all TOE components is checked as appropriate to reduce the risk of an undetected attack on (or failure of) one or more TOE components. It is also assumed that in addition to the availability of all components it is also checked as appropriate that the audit functionality is running properly on all TOE components.

A.LIMITED_FUNCTIONALITY

The device is assumed to provide networking functionality as its core function. TOE administrators are assumed to treat the TOE as not being a general-purpose computing platform and will not attempt to install or execute any non-TOE software or enable functionality unrelated to the TOE’s networking purpose, regardless of whether the platform provides an interface that could technically permit such actions.

Note: For a virtual TOE evaluated as a pND, following Case 2 vNDs as specified in Section 1.2, the VS is considered part of the TOE with only one vND instance for each physical hardware platform. The exception being where components of a distributed TOE run inside more than one virtual machine (VM) on a single VS. In Case 2 vND, no non-TOE guest VMs are allowed on the platform.

A.NO_THRU_TRAFFIC_PROTECTION

A standard/generic Network Device does not provide any assurance regarding the protection of traffic that traverses it. The intent is for the Network Device to protect data that originates on or is destined to the device itself, to include administrative data and audit data.

Note: Traffic that is traversing the Network Device, destined for another network entity, is not covered by the ND cPP. Additional protection will be covered by cPPs and PP-Modules for particular types of Network Devices (e.g., firewall).

A.PHYSICAL_PROTECTION

The Network Device is assumed to be physically protected in its operational environment and not subject to physical attacks that compromise the security or interfere with the device's physical interconnections and correct operation. This protection is assumed to be sufficient to protect the device and the data it contains. As a result, the cPP does not include any requirements on physical tamper protection or other physical attack mitigations. The cPP does not expect the product to defend against physical access to the device that allows unauthorised entities to extract data, bypass other controls, or otherwise manipulate the device. For vNDs, this assumption applies to the physical platform on which the VM runs.

A.REGULAR_UPDATES

The Network Device firmware and software is assumed to be updated by an Administrator on a regular basis in response to the release of product updates due to known vulnerabilities.

A.RESIDUAL_INFORMATION

The Administrator must ensure that there is no unauthorised access possible for sensitive residual information (e.g., cryptographic keys, keying material, PINs, passwords etc.) on networking equipment when the equipment is discarded or removed from its operational environment.

A.TRUSTED_ADMINISTRATOR

The Security Administrator(s) for the Network Device are assumed to be trusted and to act in the best interest of security for the organization. This includes appropriate training, following policy, and adhering to guidance documentation. Administrators are trusted to ensure passwords/credentials have sufficient strength and entropy and to lack malicious intent when administering the device. The Network Device is not expected to be capable of defending against a malicious Administrator that actively works to bypass or compromise the security of the device.

For TOEs supporting X.509v3 certificate-based authentication, the Security Administrator(s) are expected to fully validate (e.g., offline verification) any CA certificate (root CA certificate or intermediate CA certificate) loaded into the TOE's trust store (aka 'root store', ' trusted CA Key Store', or similar) as a trust anchor prior to use (e.g., offline verification).

A.VS_CORRECT_CONFIGURATION (applies to vNDs only)

For vNDs, it is assumed that the VS and VMs are correctly configured to support ND functionality implemented in VMs.

A.VS_ISOLATION (applies to vNDs only)

For vNDs, it is assumed that the VS implements and is configured to provide the necessary mechanisms to isolate resources of all VMs running on the same platform. Both virtual and physical resources require access control. It is assumed the VS enforces access control to all physical and virtual resources in support of isolation. In particular, it is assumed the VS implements mechanisms to isolate all resources associated with virtual networks and to limit a VM's access to only those virtual networks for which it has been configured. Furthermore, it is assumed that the VS adequately protects itself from software running inside VMs on the same platform.

A.VS_REGULAR_UPDATES (applies to vNDs only)

The VS software is assumed to be updated by the VS Administrator on a regular basis in response to the release of product updates due to known vulnerabilities.

A.VS_TRUSTED_ADMINISTRATOR (applies to vNDs only)

The Security Administrators for the VS are assumed to be trusted and to act in the best interest of security for the organization. This includes not interfering with the correct operation of the device. The Network Device

is not expected to be capable of defending against a malicious VS Administrator that actively works to bypass or compromise the security of the device.

3.3 Organizational Security Policies

P.ACCESS_BANNER

The TOE shall display an initial banner describing restrictions of use, legal agreements, or any other appropriate information to which Administrators consent by accessing the TOE.

4 Security Objectives

4.1 Security Objectives for the Operational Environment

The following security objectives for the operational environment assist the OS in correctly providing its security functionality. These track with the assumptions about the environment.

OE.ADMIN_CREDENTIALS_SECURE

The Administrator's credentials (private keys) used to access the TOE must be protected on any other platform on which they reside.

OE.COMPONENTS_RUNNING (applies to distributed TOEs only)

For distributed TOEs, the Security Administrator ensures that the availability of every TOE component is checked as appropriate to reduce the risk of an undetected attack on (or failure of) one or more TOE components. The Security Administrator also ensures that it is checked as appropriate for every TOE component that the audit functionality is running properly.

OE.NO_GENERAL_PURPOSE

There are no general-purpose computing capabilities (e.g., compilers or user applications) available on the TOE, other than those services necessary for the operation, administration and support of the TOE. Note: For vNDs the TOE includes only the contents of its own VM, and does not include other VMs or the VS.

OE.NO_THRU_TRAFFIC_PROTECTION

The TOE does not provide any protection of traffic that traverses it. It is assumed that protection of this traffic will be covered by other security and assurance measures in the operational environment.

OE.PHYSICAL

Physical security, commensurate with the value of the TOE and the data it contains, is provided by the environment.

OE.RESIDUAL_INFORMATION

The Security Administrator ensures that there is no unauthorised access possible for sensitive residual information (e.g., cryptographic keys, keying material, PINs, passwords etc.) on networking equipment when the equipment is discarded or removed from its operational environment. For vNDs, this applies when the physical platform on which the VM runs is removed from its operational environment.

OE.TRUSTED_ADMIN

Security Administrators are trusted to follow and apply all guidance documentation in a trusted manner. For vNDs, this includes the VS Administrator responsible for configuring the VMs that implement ND functionality.

For TOEs supporting X.509v3 certificate-based authentication, the Security Administrator is assumed to monitor the revocation status of all certificates in the TOE's trust store and to remove any certificate from the TOE's trust store in case such certificate can no longer be trusted.

OE.UPDATES

The TOE firmware and software are updated by an Administrator on a regular basis in response to the release of product updates due to known vulnerabilities.

OE.VM_CONFIGURATION (applies to vNDs only)

For vNDs, the Security Administrator ensures that the VS and VMs are configured to

- reduce the attack surface of VMs as much as possible while supporting ND functionality (e.g., remove unnecessary virtual hardware, turn off unused interVM communications mechanisms), and
- correctly implement ND functionality (e.g., ensure virtual networking is properly configured to support network traffic, management channels, and audit reporting).

The VS should be operated in a manner that reduces the likelihood that vND operations are adversely affected by virtualization features such as cloning, save/restore, suspend/resume, and live migration.

If possible, the VS should be configured to make use of features that leverage the VS's privileged position to provide additional security functionality. Such features could include malware detection through VM introspection, measured VM boot, or VM snapshot for forensic analysis.

4.2 Security Objectives Rationale

This section describes how the assumptions and organizational security policies map to operational environment security objectives.

Table 1: Security Objectives Rationale

Assumption or OSP	Security Objectives	Rationale
A.ADMIN_CREDENTIALS_SECURE	OE.ADMIN_CREDENTIALS_SECURE	
A.COMPONENTS_RUNNING (applies to distributed TOEs only)	OE.COMPONENTS_RUNNING (applies to distributed TOEs only)	
A.LIMITED_FUNCTIONALITY	OE.NO_GENERAL_PURPOSE	
A.NO_THRU_TRAFFIC_PROTECTION	OE.NO_THRU_TRAFFIC_PROTECTION	
A.PHYSICAL_PROTECTION	OE.PHYSICAL	
A.REGULAR_UPDATES	OE.UPDATES	
A.RESIDUAL_INFORMATION	OE.RESIDUAL_INFORMATION	
A.TRUSTED_ADMINISTRATOR	OE.TRUSTED_ADMIN	
A.VS_CORRECT_CONFIGURATION (applies to vNDs only)	OE.VM_CONFIGURATION (applies to vNDs only)	
A.VS_ISOLATION (applies to vNDs only)	OE.VM_CONFIGURATION (applies to vNDs only)	
A.VS_REGULAR_UPDATES (applies to vNDs only)	OE.UPDATES	
A.VS_TRUSTED_ADMINISTRATOR (applies to vNDs only)	OE.TRUSTED_ADMIN	

5 Security Requirements

This chapter describes the security requirements which have to be fulfilled by the product under evaluation. Those requirements comprise functional components from Part 2 and assurance components from Part 3 of [CC]. The following conventions are used for the completion of operations:

- **Refinement** operation (denoted by **bold text** or ~~strikethrough text~~): Is used to add details to a requirement or to remove part of the requirement that is made irrelevant through the completion of another operation, and thus further restricts a requirement.
- **Selection** (denoted by *italicized text*): Is used to select one or more options provided by the [CC] in stating a requirement.
- **Assignment** operation (denoted by *italicized text*): Is used to assign a specific value to an unspecified parameter, such as the length of a password. Showing the value in square brackets indicates assignment.
- **Iteration** operation: Is indicated by appending the SFR name with a slash and unique identifier suggesting the purpose of the operation, e.g. "/EXAMPLE1."

5.1 Security Functional Requirements

5.1.1 Auditable Events for Mandatory SFRs

Table 2: Auditable Events for Mandatory Requirements

Requirement	Auditable Events	Additional Audit Record Contents
FAU_GEN.1	No events specified	N/A
FAU_GEN.2	No events specified	N/A
FAU_STG_EXT.1	Configuration of local audit settings.	Identity of account making changes to the audit configuration.
FCS_CKM.1/AKG	No events specified	N/A
FCS_CKM.6	No events specified	N/A
FCS_CKM_EXT.7	No events specified	N/A
FCS_COP.1/DataEncryption	No events specified	N/A
FCS_COP.1/Hash	No events specified	N/A
FCS_COP.1/KeyedHash	No events specified	N/A
FCS_COP.1/SigGen	No events specified	N/A
FCS_RBG.1	No events specified	N/A

FIA_UIA_EXT.1	All use of identification and authentication mechanisms.	Origin of the attempt (e.g., IP address).
FMT_MOF.1/ManualUpdate	Any attempt to initiate a manual update.	No additional information
FMT_MTD.1/CoreData	No events specified	N/A
FMT_SMF.1	All management activities of TSF data.	No additional information
FMT_SMR.2	No events specified	N/A
FPT_SKP_EXT.1	No events specified	N/A
FPT_STM_EXT.1	Discontinuous changes to time - either Administrator actuated or changed via an automated process. (Note: No continuous changes to time need to be logged. See also application note on FPT_STM_EXT.1)	For discontinuous changes to time: The old and new values for the time. Origin of the attempt to change time for success and failure (e.g., IP address).
FPT_TST_EXT.1	No events specified	N/A
FPT_TUD_EXT.1	Initiation of update; result of the update attempt (success or failure).	No additional information
FTA_SSL.3	The termination of a remote session by the session locking mechanism.	No additional information
FTA_SSL.4	The termination of an interactive session.	No additional information
FTA_TAB.1	No events specified	N/A
FTP_ITC.1	Initiation of the trusted channel.	No additional information
	Termination of the trusted channel.	No additional information
	Failure of the trusted channel functions.	Reason for failure
FTP_TRP.1/Admin	Initiation of the trusted path.	No additional information
	Termination of the trusted path.	No additional information
	Failure of the trusted path functions.	Reason for failure.

5.1.2 Security Audit (FAU)

FAU_GEN.1 Audit data generation

FAU_GEN.1.1

The TSF shall be able to generate audit data of the following auditable events:

1. Start-up and shut-down of the audit functions;
2. All auditable events for the not specified level of audit; and
3. *All administrative actions comprising:*

a. *Administrative login and logout (name of Administrator account shall be logged if individual accounts are required for Administrators).*

b. *Changes to TSF data related to configuration changes (in addition to the information that a change occurred it shall be logged what has been changed).*

c. *Generating/import of, changing, or deleting of cryptographic keys (in addition to the action itself a unique key name or key reference shall be logged).*

d. [**selection**: *Resetting passwords (name of related Administrator account shall be logged), no other actions, [**assignment**: list of other uses of privileges]*];

4. *Specifically defined auditable events listed in Table 2.*

Application Note: Application Note 1

If the list of ‘administrative actions’ appears to be incomplete, the assignment in the selection should be used to list additional administrative actions which are audited.

The requirement to audit the "Generating/import of, changing, or deleting of cryptographic keys" refers to all types of cryptographic keys which are intended to be used longer than for just one session (i.e., it does not refer to ephemeral keys/session keys). The requirement applies to all named changes independently from how they are invoked. A cryptographic key could be generated automatically during initial start-up without administrator intervention or through administrator intervention. This requirement also applies to the management of cryptographic keys by adding, replacing or removing trust anchors in the TOE’s trust store. In all related cases the changes to cryptographic keys need to be audited together with a unique key name, key reference or unique identifier for the corresponding certificate.

The ST author replaces the cross-reference to the table of audit events with an appropriate cross-reference for the ST.

For distributed TOEs, each component must generate an audit record for each of the SFRs that it implements. If more than one TOE component is involved when an audit event is triggered, the event has to be audited on each component (e.g., rejection of a connection by one component while attempting to establish a secure communication channel between two components should result in an audit event being generated by both components). This is not limited to error cases but also includes events about successful actions like successful build up/tear down of a secure communication channel between TOE components.

Application Note 2

The ST author can include other auditable events directly in the table; they are not limited to the list presented.

The audit events that correspond to defined management functions are highly dependent on the FMT_SMF.1 selections. Therefore, there is only a generic requirement specified in Table 2 for FMT_SMF.1 ('All management activities of TSF data.') that is intended to cover all mandatory and selection-based management functions. If, for example, the ‘Ability to enable or disable automatic checking for updates or automatic updates’ is selected as part of FMT_SMF.1, all actions of enabling or disabling automatic checking for updates or automatic updates should be audited. Audit of management functions is intended to record both the issuing and the result of the command/administrative action. The corresponding audit event can

be recorded as either a single audit record or multiple audit records. In cases where a management function could conceivably fail, such as updating the TOE, there must exist an audit record indicating the outcome, such as the successful completion of the update process.

With respect to FAU_GEN.1.1, FMT_SMF.1 and FMT_MOF.1/Services the term ‘services’ refers to trusted path and trusted channel communications, on demand self-tests, trusted update and Administrator sessions (that exist under the trusted path) (e.g., netconf).

FAU_GEN.1.2

The TSF shall record within the audit data at least the following information:

1. Date and time of the auditable event, type of event, subject identity ~~(if applicable)~~, and the outcome (success or failure) of the event; and
2. For each auditable event type, based on the auditable event definitions of the functional components included in the **c**PP, PP-Module, functional package or ST, *information specified in column three of Table 2.*

Application Note: Application Note 3

The ST author replaces the cross-reference to the table of audit events with an appropriate cross-reference for the ST. If the TOE does not implement functionality that enables the administrator to configure local audit settings, then item FAU_STG_EXT.1 in Table 2 should be considered ‘trivially satisfied’ and the ST author should include an explanation that the local audit is not configurable in the TSS.

The date and time information for any audit event should be recorded as part of each audit record to ensure the timing of the event can be unambiguously determined from the data contained in the audit record. The representation of date and time information recorded for each event needs to allow unambiguous determination of at least day, month and year information for the date and hours, minutes and second information for the time.

Application Note 4

Additional audit events will apply to the TOE depending on the optional and selection-based requirements adopted from Annex A, Annex B, PP-Module(s), and functional package(s). For all SFRs included in the ST, the ST must include the relevant additional auditable events specified in Table 10 for optional SFRs, Table 11 for selection-based SFRs, the claimed PP-Module(s), and the claimed functional package(s). All audit events defined in Table 2 have to be included in the ST as they are mandatory.

Evaluation Activities

FAU_GEN.1

20. *The main reasons for collecting audit information are to detect and identify error conditions, security violations, etc. and to provide sufficient information to the Security Administrator to resolve the issue. The audit information to be collected according to FAU_GEN.1, and the failure conditions identified in tables 2, 11, and 12 need to enable the Security Administrator at least to detect and identify the problem and provide at least basic information to resolve the*

issue. Also for this level of detail, the other FAU requirements apply, in particular the need for local and remote storage of audit information according to FAU_STG_EXT.1.

21. *The level of detail that needs to be provided to the Security Administrator to actually resolve an issue usually depends on the complexity of the underlying use case. It is expected that a product provides additional levels of auditing to support resolution of error conditions, security violations, etc. beyond the level required by FAU_GEN.1, but it should also be clear that a high level of granularity cannot be maintained on most systems by default due to the high number of audit events that would be generated in such a configuration. It is expected that the TOE will be capable of auditing sufficient information to meet the requirements of FAU_GEN.1. If the TOE allows configuration of the level of auditing without taking the TOE out of the evaluated configuration, some of the audit events required by FAU_GEN.1 may only be recorded after corresponding configuration of the audit functionality.*
22. *The issue described above explicitly refers to the use of X.509v3 certificates. In the case that a certificate-based authentication fails, an error message telling the Security Administrator that 'something is wrong with the certificate' shall not be considered as sufficient information about the 'reason for failure' as a basic information to resolve the issue. The log message will inform the Security Administrator of at least the following:*

- *'Trust issue' with the certificate, e.g., due to failed path validation*
- *Use of an 'expired certificate'*
- *Absence of the basicConstraints extension o The CA flag is not set for a certificate presented as a CA*
- *Signature validation failure for any certificate in the certificate path; failure to establish revocation status; revoked certificate*

23. *As such for audit information related to the use of X.509v3 certificates, that it uniquely identifies the certificate that could not be successfully verified. For example, identification of a certificate could include Key Subject and Key ID (where Key Subject is an identifier contained in the CN or SAN and where Key ID is a certificate's serial number and issuer name) or Subject Key Identifier (SKI) and Authority Key Identifier (AKI). In general, when using open source libraries like OpenSSL, passing on error messages from such libraries to the Security Administrator is regarded as good practice.*

TSS

24. *For the administrative task of generating/import of, changing, or deleting of cryptographic keys as defined in FAU_GEN.1.1c, the TSS shall identify what information is logged to identify the relevant key.*
25. *For distributed TOEs, the evaluator shall examine the TSS to ensure that it describes which of the overall required auditable events defined in FAU_GEN.1.1 are generated and recorded by which TOE components. The evaluator shall ensure that this mapping of audit events to TOE components accounts for, and is consistent with, information provided in Table 1, as well as events in Tables 2, 11, and 12 (where applicable to the overall TOE). This includes that the evaluator shall confirm that all components defined as generating audit information for a particular SFR should also contribute to that SFR as defined in the mapping of SFRs to TOE components, and that the audit records generated by each component cover all the SFRs that it implements.*

Guidance

26. *The evaluator shall check the guidance documentation and ensure that it provides an example of each auditable event required by FAU_GEN.1 (i.e., at least one instance of each auditable event, comprising the mandatory, optional and selection-based SFR sections as applicable, shall be provided from the actual audit record).*

27. *The evaluator shall also make a determination of the administrative actions related to TSF data related to configuration changes. The evaluator shall examine the guidance documentation and make a determination of which administrative commands, including subcommands, scripts, and configuration files, are related to the configuration (including enabling or disabling) of the mechanisms implemented in the TOE that are necessary to enforce the requirements specified in the cPP. The evaluator shall document the methodology or approach taken while determining which actions in the administrative guide are related to TSF data related to configuration changes. The evaluator may perform this activity as part of the activities associated with ensuring that the corresponding guidance documentation satisfies the requirements related to it.*

Tests

28. *The evaluator shall test the TOE's ability to correctly generate audit records by having the TOE generate audit records for the events listed in the table of audit events and the administrative actions listed above. This should include all instances of an event; for instance, if there are several different Identity and Authentication (I&A) mechanisms for a system, the FIA_UIA_EXT.1 events must be generated for each mechanism. The evaluator shall test that audit records are generated for the establishment and termination of a channel for each of the cryptographic protocols contained in the ST. If HTTPS is implemented, the test demonstrating the establishment and termination of a TLS session can be combined with the test for an HTTPS session. When verifying the test results, the evaluator shall ensure the audit records generated during testing match the format specified in the guidance documentation, and that the fields in each audit record have the proper entries.*

29. *For distributed TOEs, the evaluator shall perform tests on all TOE components according to the mapping of auditable events to TOE components in the Security Target. For all events involving more than one TOE component when an audit event is triggered, the evaluator has to check that the event has been audited on both sides (e.g., failure of building up a secure communication channel between the two components). This is not limited to error cases but includes also events about successful actions like successful build up/tear down of a secure communication channel between TOE components.*

30. *Note that the testing here can be accomplished in conjunction with the testing of the security mechanisms directly.*

FAU_GEN.2 User identity association

FAU_GEN.2.1

For audit events resulting from actions of identified users, the TSF shall be able to associate each auditable event with the identity of the user that caused the event.

Application Note: Application Note 5

Where an auditable event is triggered by another component, the component that records the event must associate the event with the identity of the initiating component that caused the event (applies to distributed TOEs only).

Evaluation Activities

FAU_GEN.2

TSS

31. *The TSS requirements for FAU_GEN.2 are already covered by the TSS requirements for FAU_GEN.1.*

Guidance

The Guidance Documentation requirements for FAU_GEN.2 are already covered by the Guidance Documentation requirements for FAU_GEN.1.

Tests

32. *This activity should be accomplished in conjunction with the testing of FAU_GEN.1.1.*
33. *For distributed TOEs, the evaluator shall verify that where auditable events are instigated by another component, the component that records the event associates the event with the identity of the instigator. The evaluator shall perform at least one test on one component where another component instigates an auditable event. The evaluator shall verify that the event is recorded by the component as expected and the event is associated with the instigating component. It is assumed that an event instigated by another component can at least be generated for building up a secure channel between two TOE components. If for some reason (could be e.g., TSS or Guidance Documentation) the evaluator would come to the conclusion that the overall TOE does not generate any events instigated by other components, then this requirement shall be omitted.*

FAU_GEN_EXT.1 Security Audit Generation

This is a selection-based component. Its inclusion depends upon selection from .

FAU_GEN_EXT.1.1

The TSF shall be able to generate audit records for each TOE component. The audit records generated by the TSF of each TOE component shall **include the** subset of security relevant audit events which **can occur on the TOE component.**

Application Note: Application Note 48

The TOE must be able to generate audit records for each TOE component. Some TOE components of a distributed TOE might not implement the complete TSF of the overall TOE but only a subset of the TSF. The audit records for each TOE component need to cover all security relevant audit events according to the subset of the TSF implemented by this particular TOE component but not necessarily all security relevant audit events according to the TSF of the overall TOE. If a security-relevant event can occur on multiple TOE components, it needs to cause generation of an audit record uniquely identifying the component associated with the event. The ST author should identify for each TOE component which of the overall required

audit events defined in FAU_GEN.1.1 are logged. The ST author may decide to do this by providing a corresponding table. The information provided needs to be in agreement with Table 1. The overall TOE needs to cover all auditable events listed in Table 2 (and Table 10, Table 11, the claimed PP-Module(s), and the claimed functional package(s) as applicable to the overall TOE).

Evaluation Activities

FAU_GEN_EXT.1
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FAU_SAR.1 Audit Review

This is a selection-based component. Its inclusion depends upon selection from FAU_STG_EXT.1.6.

FAU_SAR.1.1

The TSF shall provide the *Security Administrator* with the capability to read *all audited events and record contents* from the audit data

FAU_SAR.1.2

The TSF shall provide the audit data in a manner suitable for the *Security Administrator* to interpret the information.

FAU_STG.2 Protected Audit Data Storage

This is an optional component. However, applied modules or packages might redefine it as mandatory.

FAU_STG.2.1

The TSF shall protect the stored audit data in the audit trail from unauthorised deletion.

FAU_STG.2.2

The TSF shall be able to prevent unauthorised modifications to the stored audit data in the audit trail.

Evaluation Activities

FAU_STG.2

TSS

256. The evaluator shall examine the TSS to ensure it describes the amount of audit data stored locally and how it is protected against unauthorised modification or deletion. The evaluator shall verify that the TSS describes the conditions that must be met for authorised deletion of audit records.

257. For distributed TOEs, the evaluator shall examine the TSS to ensure it describes to which TOE components this SFR applies and how local storage is implemented among the different TOE components (e.g., every TOE component either provides its own local storage, or the data is sent to another TOE component for central local storage of all audit events).

Guidance

258. The evaluator shall examine the guidance documentation to determine if it describes any configuration required for protection of the locally stored audit data against unauthorised modification or deletion.

Tests

259. The evaluator shall perform the following tests:

260. For distributed TOEs, the evaluator shall perform test 1 and test 2 for each component that is defined by the TSS to be covered by this SFR.

- *Test FAU_STG.2:1:*
  Test 1: The evaluator shall attempt to access the audit trail without authentication as a Security Administrator (either by authentication as a non-administrative user, if supported, or without authentication at all) and attempt to modify and delete the audit records. The evaluator shall verify that these attempts fail.

  In the case that no other users than the Security Administrator can be defined, without user authentication the user may not be able to get to the point where the attempt to access the audit trail can be executed. In this case it shall be demonstrated that access control mechanisms prevent execution up to the step that can be reached without authentication as Security Administrator.
- *Test FAU_STG.2:2:*
  Test 2: The evaluator shall access the audit trail as an authenticated Security Administrator and attempt to delete the audit records (if supported by the TOE, and to the extent described in the TSS). The evaluator shall verify that these attempts succeed. The evaluator shall verify that only the records authorised for deletion are deleted.

FAU_STG_EXT.1 Protected Audit Event Storage

FAU_STG_EXT.1.1

The TSF shall be able to transmit the generated audit data to an external IT entity using a trusted channel according to FTP_ITC.1.

Application Note: Application Note 6

For selecting the option of transmission of generated audit data to an external IT entity the TOE relies on a non-TOE audit server for storage and review of audit records. The storage of these audit records and the ability to allow the Administrator to review these audit records is provided by the operational environment in that case. Since the external audit server is not part of the TOE, there are no requirements on it

except the capabilities for FTP_ITC.1 transport for audit data. No requirements are placed upon the format or underlying protocol of the audit data being transferred. The TOE must be capable of being configured to transfer audit data to an external IT entity without Administrator intervention. Manual transfer would not meet the requirements. Transmission could be done in real-time or periodically. If the transmission is not done in real-time then the TSS describes what event stimulates the transmission to be made and what range of frequencies the TOE supports for making transfers of audit data to the audit server, the TSS also suggests typical acceptable frequencies for the transfer.

For distributed TOEs, each component must be able to export audit data across a protected channel external (FTP_ITC.1) or intercomponent (FPT_ITT.1 or FTP_ITC.1) as appropriate. At least one component of the TOE must be able to export audit records via FTP_ITC.1 such that all TOE audit records can be exported to an external IT entity.

An ‘external IT entity’ (physical or virtualized) is another device or computer on the network in which the TOE no longer has access to the audit records. This can be a physical or virtualized entity.

FAU_STG_EXT.1.2

The TSF shall be able to store generated audit data on the TOE itself. In addition

[**selection**: *The TOE shall consist of a single standalone component that stores audit data locally, The TOE shall be a distributed TOE that stores audit data on the following TOE components: [**assignment**: identification of TOE components], The TOE shall be a distributed TOE with storage of audit data provided externally for the following TOE components: [**assignment**: list of TOE components that do not store audit data locally and the other TOE components to which they transmit their generated audit data]*]

Application Note: Application Note 7

If the TOE is a standalone TOE (i.e., not a distributed TOE) the option 'The TOE should consist of a single standalone component that stores audit data locally' must be selected.

If the TOE is a distributed TOE, the option 'The TOE should be a distributed TOE that stores audit data on the following TOE components: [assignment: identification of TOE components]' must be selected and the TOE components which store audit data locally must be listed in the assignment. Since all TOEs are required to provide functions to store audit data locally this option needs to be selected for all distributed TOEs. In addition, FAU_GEN_EXT.1 and FAU_STG_EXT.4 must be claimed in the ST. If the distributed TOE consists only of components which are storing audit data locally, it is sufficient to select only the option 'The TOE should be a distributed TOE that stores audit data on the following TOE components: [assignment: identification of TOE components]' and add FAU_GEN_EXT.1 and FAU_STG_EXT.4.

If the TOE is a distributed TOE and some TOE components are not storing audit data locally, the option 'The TOE should be a distributed TOE with storage of audit data provided externally for the following TOE components: [assignment: list of TOE components that do not store audit data locally and the other TOE components to which they transmit their generated audit data]' must be selected in addition to the option 'The TOE should be a distributed TOE that stores audit data on the following TOE components: [assignment: identification of TOE components]'. In that case FAU_STG_EXT.5 must be claimed in the ST in addition to FAU_GEN_EXT.1 and FAU_STG_EXT.4. For the option 'The TOE should be a distributed TOE with

storage of audit data provided externally for the following TOE components: [assignment: list of TOE components that do not store audit data locally and the other TOE components to which they transmit their generated audit data]' the TOE components that do not store audit data locally should be mapped to the TOE components to which they transmit their generated audit data.

For distributed TOEs, this SFR can be fulfilled either by every TOE component storing its own security audit data locally or by one or more TOE components storing audit data locally and other TOE components which are not storing audit information locally sending security audit data to other TOE components for local storage. For the transfer of security audit data between TOE components a protected channel according to FTP_ITC.1 or FPT_ITT.1 must be used. The TSS describes which TOE components store security audit data locally and which TOE components do not store security audit data locally. For the latter, the TSS describes which other TOE component the audit data is stored locally.

For pNDs, ‘on the TOE itself’ or ‘locally’ means on storage inside or directly attached to the ND chassis and accessible by the networking functionality.

For vNDs, local storage is any storage accessible by TOE software. In a virtualized environment, ‘local’ storage is under the control of the VS and may be physically located on the local host, but it could also be located on a network drive or storage array.

FAU_STG_EXT.1.3

The TSF shall maintain a [**selection**: *log file, database, buffer, [**assignment**: other local logging method]*] of audit records in the event that an interruption of communication with the remote audit server occurs.

FAU_STG_EXT.1.4

The TSF shall be able to store [**selection**: *persistent, non-persistent*] audit records locally with a minimum storage size of [**assignment**: *number of records and/or file/buffer size(s)*]

Application Note: Application Note 8

Persistent logging is defined as any record(s) that is retained through power off, power failure, or reboot. This requirement allows for the TSF to implement logging either persistent log records or non-persistent log records that may be cleared on reboot of the TOE.

FAU_STG_EXT.1.5

The TSF shall [**selection**: *drop new audit data, overwrite previous audit records according to the following rule: [**assignment**: rule for overwriting previous audit records], [**assignment**: other action]*] when the local storage space for audit data is full.

Application Note: Application Note 9

The ST author may use the "other action" assignment to describe other measurable behaviour (e.g., frequency of log file rotation based on size and/or age of log files).

For distributed TOEs, each component is not required to store generated audit data locally, but the overall TOE needs to be able to store audit data locally. Each component must at least provide the ability to temporarily buffer audit information locally to ensure that audit records are preserved in case of network connectivity issues. Buffering audit information locally, does not necessarily involve non-volatile memory: audit information could be buffered in volatile memory. However, the local

storage of audit information in the sense of FAU_STG_EXT.1.5 needs to be done in non-volatile memory. For every component which performs local storage of audit information, the behaviour when local storage is exhausted needs to be described. For every component which is buffering audit information instead of storing audit information locally itself, it needs to be described what happens in case the buffer space is exhausted.

FAU_STG_EXT.1.6

The TSF shall provide the following mechanisms for administrative access to locally stored audit records [**selection**: *none*, *manual export*, *ability to view locally*].

Application Note: Application Note 10
If "ability to view locally" is selected in FAU_STG_EXT.1.6, then FAU_SAR.1 from Annex B must be included in the ST.

Evaluation Activities

FAU_STG_EXT.1

TSS

34. The evaluator shall examine the TSS to ensure it describes the means by which the audit data are transferred to the external audit server, and how the trusted channel is provided.

35. The evaluator shall examine the TSS to ensure it describes whether the TOE is a standalone TOE that stores audit data locally or a distributed TOE that stores audit data locally on each TOE component or a distributed TOE that contains TOE components that cannot store audit data locally on themselves but need to transfer audit data to other TOE components that can store audit data locally. The evaluator shall examine the TSS to ensure that for distributed TOEs it contains a list of TOE components that store audit data locally. The evaluator shall examine the TSS to ensure that for distributed TOEs that contain components which do not store audit data locally but transmit their generated audit data to other components it contains a mapping between the transmitting and storing TOE components.

36. The evaluator shall examine the TSS to ensure that it details whether the transmission of audit data to an external IT entity can be done in real-time, periodically, or both. In the case where the TOE is capable of performing transmission periodically, the evaluator shall verify that the TSS provides details about what event stimulates the transmission to be made as well as the possible acceptable frequency for the transfer of audit data.

37. For distributed TOEs, the evaluator shall examine the TSS to ensure it describes to which TOE components this SFR applies and how audit data transfer to the external audit server is implemented among the different TOE components (e.g., every TOE component does its own transfer or the data is sent to another TOE component for central transfer of all audit events to the external audit server).

38. The evaluator shall examine the TSS to ensure it describes the amount of audit data that can be stored locally and how these records are protected against unauthorised modification or deletion.

39. The evaluator shall examine the TSS to ensure it describes the method implemented for local logging, including format (e.g., buffer, log file, database) and whether the logs are persistent or non-persistent.

40. The evaluator shall examine the TSS to ensure it describes the conditions that must be met for authorised deletion of audit records.

41. The evaluator shall examine the TSS to ensure it details the behaviour of the TOE when the storage space for audit data is full. When the option 'overwrite previous audit record' is selected this description should include an outline of the rule for overwriting audit data. If 'other actions' are chosen such as sending the new audit data to an external IT entity, then the related behaviour of the TOE shall also be detailed in the TSS.

42. For distributed TOEs, the evaluator shall examine the TSS to ensure it describes which TOE components are storing audit information locally and which components are buffering audit information and forwarding the information to another TOE component for local storage. For every component the TSS shall describe the behaviour when local storage space or buffer space is exhausted.

Guidance

43. The evaluator shall also examine the guidance documentation to ensure it describes how to establish the trusted channel to the audit server, as well as describe any requirements on the audit server (particular audit server protocol, version of the protocol required, etc.), as well as configuration of the TOE needed to communicate with the audit server.

44. The evaluator shall also examine the guidance documentation to ensure it describes the relationship between the local audit data and the audit data that are sent to the audit log server. For example, when an audit event is generated, is it simultaneously sent to the external server and the local store, or is the local store used as a buffer and "cleared" periodically by sending the data to the audit server.

45. The evaluator shall examine the guidance documentation to ensure it describes any configuration required for protection of the locally stored audit data against unauthorised modification or deletion.

46. If the storage size is configurable, the evaluator shall review the Guidance Documentation to ensure it contains instructions on specifying the required parameters.

47. If more than one selection is made for FAU_STG_EXT.1.5, the evaluator shall review the Guidance Documentation to ensure it contains instructions on specifying which action is performed when the local storage space is full.

Tests

48. Testing of secure transmission of the audit data externally (FTP_ITC.1) and, where applicable, intercomponent (FPT_ITT.1 or FTP_ITC.1) shall be performed according to the assurance activities for the particular protocol(s).

49. The evaluator shall perform the following additional test for this requirement:

- *Test FAU_STG_EXT.1:1:*
  The evaluator shall establish a session between the TOE and the audit server according to the configuration guidance provided. The evaluator shall then examine the traffic that passes between the audit server and the TOE during several activities of the evaluator's choice designed to generate audit data to be transferred to the audit server. The evaluator shall observe that these data are not able to be viewed in the clear during this transfer, and that they are successfully received by the audit server. The evaluator shall record the particular software (name, version) used on the audit server during testing. The evaluator shall verify

that the TOE is capable of transferring audit data to an external audit server automatically without administrator intervention.

The following content should be included if:

- *the TOE implements "Distributed TOE"*

For distributed TOEs, Test 1 defined above shall be applicable to all TOE components that forward audit data to an external audit server.

- *Test FAU_STG_EXT.1:2:*
  The evaluator shall perform operations that generate audit data and verify that this data is stored locally. The evaluator shall then make note of whether the TSS claims persistent or non-persistent logging and perform one of the following actions:

  1. *If persistent logging is selected, the evaluator shall perform a power cycle of the TOE and ensure that following power on operations the log events generated are still maintained within the local audit storage.*
  2. *If non-persistent logging is selected, the evaluator shall perform a power cycle of the TOE and ensure that following power on operations the log events generated are no longer present within the local audit storage.*

- *Test FAU_STG_EXT.1:3:*
  The evaluator shall perform operations that generate audit data until the local storage space is exceeded and verifies that the TOE complies with the behaviour defined in FAU_STG_EXT.1.5. Depending on the configuration this means that the evaluator shall check the content of the audit data when the audit data is just filled to the maximum and then verifies that:

  1. *The audit data remains unchanged with every new auditable event that should be tracked but that the audit data is recorded again after the local storage for audit data is cleared (for the option 'drop new audit data' in FAU_STG_EXT.1.5).*
  2. *The existing audit data is overwritten with every new auditable event that should be tracked according to the specified rule (for the option 'overwrite previous audit records' in FAU_STG_EXT.1.5)*
  3. *The TOE behaves as specified (for the option 'other action' in FAU_STG_EXT.1.5).*

  71f55177-ba90-4ca2-ac4d-ae73c946de53

  1. *The audit data remains unchanged with every new auditable event that should be tracked but that the audit data is recorded again after the local storage for audit data is cleared.*

  The existing audit data is overwritten with every new auditable event that should be tracked according to the specified rule (for the option 'overwrite previous audit records' in FAU_STG_EXT.1.5)

  The TOE behaves as specified (for the option 'other action' in FAU_STG_EXT.1.5).

The following content should be included if:

- *the TOE implements "Distributed TOE"*

For distributed TOEs, for the local storage according to FAU_STG_EXT.1.4, Test 1 specified above shall be applied to all TOE components that store audit data locally. For all TOE components that store audit data locally and comply with FAU_STG_EXT.2, Test 2 specified above shall be applied. The evaluator shall verify that the transfer of audit data to an external audit server is implemented.

The following content should be included if:

- *manual export* is selected from *FAU_STG_EXT.1.6*
- *ability to view locally* is selected from *FAU_STG_EXT.1.6*

[Conditional]: In case manual export or ability to view locally is selected in FAU_STG_EXT.1.6, during interruption the evaluator shall perform a TSF-mediated action and verify the event is recorded in the audit trail.

Note: The intent of the test is to ensure that the local audit TSF (as specified by FAU_STG_EXT.1.3) operates independently from the ability to transmit the generated audit data to an external audit server (as specified in FAU_STG_EXT.1.1). There are no specific requirements on the interruption of the connection between the TOE and the external audit server (as for FTP_ITC.1).

FAU_STG_EXT.2 Counting Lost Audit Data

This is an optional component. However, applied modules or packages might redefine it as mandatory.

FAU_STG_EXT.2.1

The TSF shall provide information about the number of [**selection**: *dropped*, *overwritten*, *[**assignment**: other information]*] audit records in the case where the local storage has been filled and the TSF takes one of the actions defined in FAU_STG_EXT.1.5.

Application Note: Application Note 41
This option should be chosen if the TOE supports this functionality.

In case the local storage for audit records is cleared by the Administrator, the counters associated with the selection in the SFR should be reset to their initial value (most likely to 0). The guidance documentation should contain a warning for the Administrator about the loss of audit data when he clears the local storage for audit records.

For distributed TOEs, each component that implements counting of lost audit data has to provide a mechanism for Administrator access to, and management of, this information.

If FAU_STG_EXT.2 is added to the ST, the ST has to make clear any situations in which lost audit data is not counted.

Evaluation Activities

FAU_STG_EXT.2
261. This activity should be accomplished in conjunction with the testing of FAU_STG_EXT.1.4 and FAU_STG_EXT.1.5.

TSS
262. The evaluator shall examine the TSS to ensure that it details the possible options the TOE supports for information about the number of audit records that have been dropped, overwritten, etc. when the local storage for audit data is full.

263. For distributed TOEs, the evaluator shall examine the TSS to ensure it describes to which TOE components this SFR applies. Since this SFR is optional, it might only apply to some TOE components and not all. This might lead to the situation where all TOE components store their audit information themselves but FAU_STG_EXT.2 is supported only by one of the components.

Guidance

264. The evaluator shall also ensure that the guidance documentation describes all possible configuration options and the meaning of the result returned by the TOE for each possible configuration. The description of possible configuration options and explanation of the result shall correspond to those described in the TSS.

265. The evaluator shall verify that the guidance documentation contains a warning for the administrator about the loss of audit data when clearing the local storage for audit records.

Tests

- *Test FAU_STG_EXT.2:1:*
  266. The evaluator shall verify that the numbers provided by the TOE according to the selection for FAU_STG_EXT.2 are correct when performing the tests for FAU_STG_EXT.1.5.

  The following content should be included if:
  - *the TOE implements "Distributed TOE"*

  267. For distributed TOEs, the evaluator shall verify the correct implementation of counting of lost audit data for all TOE components that are supporting this feature, according to the description in the TSS.

FAU_STG_EXT.3 Action in Case of Possible Audit Data Loss

This is an optional component. However, applied modules or packages might redefine it as mandatory.

FAU_STG_EXT.3.1

The TSF shall generate a warning to inform the Administrator before the audit trail exceeds the local audit trail storage capacity.

Application Note: Application Note 42

This option should be chosen if the TOE generates a warning to inform the Administrator before the local storage space for audit data is used up. This SFR only applies to local storage of audit information.

It has to be ensured that the warning message required by FAU_STG_EXT.3.1 can be communicated to the Administrator. The communication should be done via the audit log itself because it cannot be guaranteed that an administrative session is active at the time the event occurs.

The warning should inform the Administrator when the local space to store audit data is used up and/or the TOE will lose audit data due to insufficient local space.

For distributed TOEs, that implement displaying a warning when local storage space for audit data is exhausted, it has to be described which TOE components support this feature (not necessarily all TOE components have to support this feature if selected for the overall TOE). Each component that supports this feature must either generate a warning itself or through another component.

If FAU_STG_EXT.3 is added to the ST, the ST has to make clear any situations in which audit records might be “invisibly lost”.

Evaluation Activities

FAU_STG_EXT.3

268. This activity should be accomplished in conjunction with the testing of FAU_STG_EXT.1.4 and FAU_STG_EXT.1.5.

TSS

269. The evaluator shall examine the TSS to ensure that it details how the Security Administrator is warned before the local storage for audit data is full.

270. For distributed TOEs, the evaluator shall examine the TSS to ensure it describes to which TOE components this SFR applies and how each TOE component implements this SFR. Since this SFR is optional, it might only apply to some TOE components and not all. This might lead to the situation where all TOE components store their audit information themselves but FAU_STG_EXT.3 is supported only by one of the components. In particular, the evaluator shall verify that the TSS describes for every component supporting this functionality, whether the warning is generated by the component itself or through another component and name the corresponding component in the latter case. The evaluator shall verify that the TSS makes clear any situations in which audit records might be 'invisibly lost'.

Guidance

271. The evaluator shall also verify that the guidance documentation describes how the Security Administrator is warned before the local storage for audit data is full and how this warning is displayed or stored (since there is no guarantee that an administrator session is running at the time the warning is issued, it is very likely that it will be stored in the log files). The description in the guidance documentation shall correspond to the description in the TSS.

Tests

- *Test FAU_STG_EXT.3:1:*
  272. The evaluator shall verify that a warning is issued by the TOE before the local storage space for audit data is full.

  The following content should be included if:
  - *the TOE implements "Distributed TOE"*

  273. For distributed TOEs, the evaluator shall verify the correct implementation of display warning for local storage space for all TOE components that are supporting this feature according to the description in the TSS. The evaluator shall verify that each component that supports this feature according to the description in the TSS is capable of generating a warning itself or through another component.

FAU_STG_EXT.4 Protected Local Audit Event Storage for Distributed TOEs

This is a selection-based component. Its inclusion depends upon selection from .

FAU_STG_EXT.4.1

The TSF of each TOE component which stores security audit data locally shall perform the following actions when the local storage space for audit data is full: [**assignment**: *table of components and for each component its action chosen according to the following:*] [**selection**: *drop new audit data, overwrite previous audit records according to the following rule: [**assignment**: rule for overwriting previous audit records], [**assignment**: other action]*].

Application Note: Application Note 49

If a component of a distributed TOE collects data from other components and then forwards it to another component or external IT entity (reference FAU_STG_EXT.1.1) then the operations in this SFR must be performed in a way to cover the storage space action(s) for all of the audit data that the TOE collects (i.e., not just for the data generated by the collecting component for itself).

It is acceptable for a TOE component to store audit information in multiple places (e.g., for redundancy), whether locally in the TOE component itself and in another TOE component, or in more than one other TOE component.

TOE components are not required to monitor or audit connectivity or network outages between TOE components. This aspect is covered by the assumption A.COMPONENTS_RUNNING

Evaluation Activities

FAU_STG_EXT.4
Something
TSS
ABC

Guidance

There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FAU_STG_EXT.5 Protected Remote Audit Event Storage for Distributed TOEs

This is a selection-based component. Its inclusion depends upon selection from .

FAU_STG_EXT.5.1

Each TOE component which does not store security audit data locally shall be able to buffer security audit data locally until it has been transferred to another TOE component that stores or forwards it. All transfer of audit records between TOE components shall use a protected channel according to [**selection**: *FPT_ITT.1*, *FTP_ITC.1*].

Application Note: Application Note 50

If a component of a distributed TOE collects data from other components and then forwards it to another component or external IT entity (cf. FAU_STG_EXT.1.1) then the operations in this SFR must be performed in a way to cover the storage space action(s) for all of the audit data that the TOE collects (i.e. not just for the data generated by the collecting component for itself).

It is acceptable for a TOE component to store audit information in multiple places (e.g., for redundancy), whether locally in the TOE component itself and in another TOE component, or in more than one other TOE component.

TOE components are not required to monitor or audit connectivity or network outages between TOE components. This aspect is covered by the assumption A.COMPONENTS_RUNNING.

Evaluation Activities

FAU_STG_EXT.5
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

5.1.3 Communication (FCO)

FCO_CPC_EXT.1 Component Registration Channel Definition

This is an optional component. However, applied modules or packages might redefine it as mandatory.

FCO_CPC_EXT.1.1

The TSF shall require a Security Administrator to enable communications between any pair of TOE components before such communication can take place.

FCO_CPC_EXT.1.2

The TSF shall implement a registration process in which components establish and use a communications channel that uses [**assignment**: *list of different types of channel given in the form of a selection*] for at least [**assignment**: *type of data for which the channel must be used*].

FCO_CPC_EXT.1.3

The TSF shall enable a Security Administrator to disable communications between any pair of TOE components.

Application Note: Application Note 46

This SFR is only applicable if the TOE is distributed and therefore has multiple components that need to communicate via an internal TSF channel. When creating the TSF from the initial pair of components, either of these components may be identified as the TSF for the purposes of satisfying the meaning of 'TSF' in this SFR.

The intention of this requirement is to ensure that there is a registration process that includes a positive enablement step by an Administrator before components joining a distributed TOE can communicate with the other components of the TOE and before the new component can act as part of the TSF. The registration process may itself involve communication with the joining component: many Network Devices use a bespoke process for this, and the security requirements for the 'registration communication' are then defined in FCO_CPC_EXT.1.2. Use of this 'registration communication' channel is not deemed inconsistent with the requirement of FCO_CPC_EXT.1.1 (i.e., the registration channel can be used before the enablement step, but only in order to complete the registration process).

The channel selection (for the registration channel) in FCO_CPC_EXT.1.2 is essentially a choice between the use of a normal secure channel that is equivalent to a channel used to communicate with external IT entities (FTP_ITC.1) or existing TOE components (FPT_ITT.1), or else a separate type of channel that is specific to registration (FTP_TRP.1/Join). If the TOE does not require a communications channel for registration (e.g., because the registration is achieved entirely by configuration actions by an Administrator at each of the components) then the main selection in FCO_CPC_EXT.1.2 is completed with the 'No channel' option.

If the ST author selects the FTP_ITC.1/FPT_ITT.1 channel type in the main selection in FCO_CPC_EXT.1.2 then the TSS identifies the relevant SFR iteration that specifies the channel used. If the ST author selects the FTP_TRP.1/Join channel type, then the TOE Summary Specification (possibly with support from the operational guidance) describes details of the channel and the mechanisms that it uses (and describes how the registration process ensures that the channel can only be used by the intended joiner and gatekeeper). Note: The FTP_TRP.1/Join channel type may require support from security measures in the operational environment (see the definition of FTP_TRP.1/Join for details).

If the ST author selects the FTP_ITC.1/FPT_ITT.1 channel type in the main selection in FCO_CPC_EXT.1.2 then the ST identifies the registration channel as a

separate iteration of FTP_ITC.1 or FPT_ITT.1 and gives the iteration identifier (e.g., “FPT_ITT.1/Join”) in an ST Application Note for FCO_CPC_EXT.1.

Note: The channel set up and used for registration may be adopted as a continuing internal communication channel (i.e., between different TOE components) provided that the channel meets the requirements of FTP_ITC.1 or FPT_ITT.1. Otherwise, the registration channel is closed after use, and a separate channel is used for the internal communications.

Specific requirements for Preparative Procedures relating to FCO_CPC_EXT.1 are defined in the Evaluation Activities in [SD].

Evaluation Activities

FCO_CPC_EXT.1
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

5.1.4 Cryptographic Support (FCS)

FCS_CKM.1/AKG Cryptographic Key Generation - Asymmetric Key

FCS_CKM.1.1/AKG

The TSF shall generate **asymmetric** cryptographic keys in accordance with a specified cryptographic key generation algorithm:

[**selection**: *Cryptographic Key Generation Algorithm*] and specified cryptographic algorithm parameters [**selection**: *Cryptographic Algorithm Parameters*] that meet the following: [**selection**: *List of Standards*] . The following table provides the allowed choices for completion of the selection operations of FCS_CKM.1.1/AKG.

Table 3: Allowed choices for FCS_CKM.1.1/AKG

Identifier	Cryptographic Key Generation Algorithm	Cryptographic Algorithm Parameters	List of Standards
RSA	RSA	Modulus of size [**selection**: *2048*, *3072*, *4096*, *6144*, *8192*] bits	NIST FIPS PUB 186-5 (Section A.1.1)
ECC-ERB	ECC-ERB - Extra Random Bits	Elliptic Curve [**selection**: *P-256*, *P-384*, *P-521*]	NIST FIPS PUB 186-5 (Section A.2.1), NIST SP 800-186 (Section 3) [NIST Curves]

ECC-RS	ECC-RS - Rejection Sampling	Elliptic Curve [**selection**: *P-256, P-384, P-521*]	NIST FIPS PUB 186-5 (Section A.2.2), NIST SP 800-186 (Section 3) [NIST Curves]
FFC-ERB	FFC-ERB - Extra Random Bits	Static domain parameters approved for [**selection**: • *IKE Groups [**selection**: MODP-2048, MODP-3072, MODP-4096, MODP-6144, MODP-8192]* • *TLS Groups [**selection**: ffdhe-2048, ffdhe-3072, ffdhe-4096, ffdhe-6144, ffdhe-8192]*]	NIST SP 800-56A Revision 3 (Section 5.6.1.1.3), [**selection**: *RFC 3526 [IKE groups], RFC 7919 [TLS groups]*]
FCC-RS	FCC-RS - Extra Random Bits	Static domain parameters approved for [**selection**: • *IKE Groups [**selection**: MODP-2048, MODP-3072, MODP-4096, MODP-6144, MODP-8192]* • *TLS Groups [**selection**: ffdhe-2048, ffdhe-3072, ffdhe-4096, ffdhe-6144, ffdhe-8192]*]	NIST SP 800-56A Revision 3 (Section 5.6.1.1.3), [**selection**: *RFC 3526 [IKE groups], RFC 7919 [TLS groups]*]
LMS	LMS	private key size [**selection**: • *192 bits with [**selection**: SHA-256/192, SHAKE256/192]* • *256 bits with [**selection**: SHA-256, SHAKE256]*] Winternitz parameter = [**selection**: *1, 2, 4, 8*],Tree height = [**selection**: *5, 10, 15, 20, 25*]	RFC 8554 [LMS], NIST SP 800-208 [parameters]
XMSS	XMSS	private key size [**selection**: • *192 bits with [**selection**: SHA-*	RFC 8391 [XMSS], NIST SP

		256/192, SHAKE256/192] • *256 bits with [**selection**: SHA-256, SHAKE256]*] Tree height = [**selection**: *10*, *16*, *20*]	800-208 [parameters]
ML-KEM	ML-KEM	Parameter set = ML-KEM-1024	NIST FIPS PUB 203
ML-DSA	ML-DSA	Parameter set = ML-DSA-87	NIST FIPS PUB 204

Application Note: Application Note 11

The ST author selects all key generation algorithms used for key agreement (including generation of ephemeral keys) and device authentication.

For RSA the choice of the modulus implies the resulting key sizes of the public and private keys generated using the specified standard methods.

When generating ECC keys pairs for key agreement and if “ECDH” is claimed in FCS_CKM_EXT.7, then “ECC–ERB” or “ECC–RS” must be claimed. The sizes of the private key, which is a scalar, and the public key, which is a point on the elliptic curve, are determined by the choice of the curve.

For Finite Field Cryptography (FFC), “FFC-ERB” or “FFC–RS” may be claimed only for generating private and public keys when “DH” is claimed in FCS_CKM_EXT.7.

The MODP Diffie-Hellman groups do not necessarily adhere to the protocol restrictions specified as IKE groups. MODP Diffie-Hellman groups may also be used in other protocols such as TLS 1.2.

When generating ECC key pairs for digital signature generation and if “ECDSA” is claimed in FCS_COP.1/SigGen, then “ECC–ERB” or “ECC–RS” must be claimed. The sizes of the private key, which is a scalar, and the public key, which is a point on the elliptic curve, are determined by the choice of the curve.

When key generation is used for device authentication, other than non-X.509 SSH authentication algorithm, the public key is expected to be associated with an X.509v3 certificate.

Evaluation Activities

FCS_CKM.1/AKG

TSS

50. The evaluator shall examine the TSS to verify that it describes how the TOE generates a key based on output from a random bit generator as specified in FCS_RBG.1. The evaluator shall review the TSS to verify that it describes how the functionality described by FCS_RBG.1 is invoked.

51. The evaluator shall examine the TSS to verify that it identifies the usage and key lifecycle for keys generated using each selected algorithm.

52. The evaluator shall examine the TSS to verify that any one-time values such as nonces or masks are constructed in accordance with the relevant standards.

53. If the TOE uses the generated key in a key chain or hierarchy then the evaluator shall verify that the TSS describes how the key is used as part of the key chain or hierarchy.

Guidance

54. The evaluator shall verify that the Guidance instructs the administrator how to configure the TOE to generate keys for the selected key generation algorithms for all key types and uses identified in the TSS.

Tests

55. The following tests are conditional based on the selections made in the SFR. The evaluator shall perform the following tests or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

- *Test FCS_CKM.1/AKG:1:*
  RSA Key Generation

  56. FIPS PUB 186-5 Key Pair generation specifies five methods for generating the primes p and q. These are:

  a. Random provable primes

  b. Random probable primes

  c. Provable primes with conditions based on auxiliary provable primes

  d. Probable primes with conditions based on auxiliary provable primes

  e. Probable primes with conditions based on auxiliary probable primes

  In addition to the key generation method, the input parameters are:

  - *Modulus [3072, 4096, 6144, 8192]*
  - *Hash algorithm [SHA-384, SHA-512] (methods 1, 3, 4 only)*
  - *Rabin-Miller prime test [2100, 2Security String] (methods 2, 4, 5 only)*
  - *p mod 8 value [0,1,3,5,7]*
  - *q mod 8 value [0,1,3,5,7]*

- *Private key format [standard, Chinese Remainder Theorem]*
- *Public exponent [fixed value, random]*

The evaluator shall verify the ability of the TSF to correctly produce values for the RSA key components, including the public verification exponent e, the private prime factors p and q, the public modulus n, and the calculation of the private signature exponent d.

- *Test FCS_CKM.1/AKG:2:*
  Testing for Random Provable Primes and Conditional Methods

  57. To test the key generation method for the random provable primes method and for all the primes with conditions methods (methods 1, 3-5), the evaluator must seed the TSF key generation routine with sufficient data to deterministically generate the RSA key pair. For each supported combination of the above input parameters, the evaluator shall have the TSF generate 25 key pairs. The evaluator shall verify the correctness of the TSF's implementation by comparing values generated by the TSF with those generated by a known good implementation using the same input parameters.

- *Test FCS_CKM.1/AKG:3:*
  Testing for Random Probable Primes Method

  58. If the TOE generates random probable primes (method 2) then, if possible, the random probable primes method should also be verified against a known good implementation as described above. If verification against a known good implementation is not possible, the evaluator shall have the TSF generate 25 key pairs for each supported key length nlen and verify that all of the following are true.

  - *n = p*q*
  - *p and q are probably prime according to Miller-Rabin tests with error probability <2(-125)*
  - *216 < e < 2256 and e is an odd integer*
  - *GCD(p-1,e) = 1*
  - *GCD(q-1,e) = 1*
  - *|p-q| > 2(nlen/2 – 100)*
  - *p ≥ squareroot(2)*(2(nlen/2 -1))*
  - *q ≥ squareroot(2)*(2(nlen/2 -1))*
  - *2(nlen/2) < d < LCM(p-1,q-1)*
  - *e*d = 1 mod LCM(p-1,q-1)*

- *Test FCS_CKM.1/AKG:4:*
  Elliptic Curve Key Generation

  59. To test the TOE's ability to generate asymmetric cryptographic keys using elliptic curves, the evaluator shall perform the ECC Key Generation Test and the ECC Key Validation Test using the following input parameters:

  a. Elliptic curve [P-256, P-384, P-521]

  b. Key pair generation method [extra random bits, rejection sampling]

- *Test FCS_CKM.1/AKG:5:*
  ECC Key Generation Test

60. For each supported combination of the above input parameters the evaluator shall require the implementation under test to generate 10 private/public key pairs (d, Q). The private key, d, shall be generated using a random bit generator as specified in FCS_RBG.1. The private key, d, is used to compute the public key, Q'. The evaluator shall confirm that 0<d<n (where n is the order of the group), and the computed value Q' is then compared to the generated public/private key pairs' public key, Q, to confirm that Q is equal to Q'.

- *Test FCS_CKM.1/AKG:6:*
  ECC Key Validation Test

  61. For each supported combination of the above parameters the evaluator shall generate 12 private/public key pairs using the key generation function of a known-good implementation. For each set of 12 public keys, the evaluator shall modify four public key values by shifting x or y out of range by adding the order of the field and modify four other public key values by shifting x or y so that they are still in bounds, but not on the curve. The remaining public key values are left unchanged (i.e., correct). To determine correctness, the evaluator shall submit the public keys to the public key validation (PKV) function of the TOE and shall confirm that the results correspond as expected for the modified and unmodified values.

- *Test FCS_CKM.1/AKG:7:*
  Finite Field Cryptography Key Generation

  62. To test the TOE's ability to generate asymmetric cryptographic keys using finite fields, the evaluator shall perform the Safe Primes Generation Test and the Safe Primes Validation Test using the following input parameter:

  a. Fields/Groups [MODP-3072, MODP-4096, MODP-6144, MODP-8192, ffdhe3072, ffdhe4096, ffdhe6144, ffdhe8192]

- *Test FCS_CKM.1/AKG:8:*
  Safe Primes Generation Test

  63. For each supported safe primes group, generate 10 key pairs. The evaluator shall verify the correctness of the TSF's implementation by comparing values generated by the TSF with those generated by a known good implementation using the same input parameters.

- *Test FCS_CKM.1/AKG:9:*
  Safe Primes Validation Test

  64. For each supported safe primes group, use a known good implementation to generate 10 key pairs. For each set of 10, the evaluator shall modify three such that they are incorrect. The remaining values are left unmodified (i.e. correct). To determine correctness, the evaluator shall submit the key pairs to the public key validation (PKV) function of the TOE and shall confirm that the results correspond as expected for the modified and unmodified values.

- *Test FCS_CKM.1/AKG:10:*
  LMS Key Generation

  65. To test the TOE's ability to generate asymmetric cryptographic keys using LMS, the evaluator shall perform the LMS Key Generation Test using the following input parameters:

  a. Hash algorithm [SHA-256/192, SHAKE256/192, SHA-256, SHAKE256]

  b. Winternitz [1, 2, 4, 8]

  c. Tree height [5, 10, 15, 20, 25]

- *Test FCS_CKM.1/AKG:11:*
  LMS Key Generation Test

66. For each supported combination of the hash algorithm, Winternitz parameter, and tree height, the evaluator shall generate one public key for each of the test cases. The number of test cases depends on the tree height, as follows:

- *For height of 5, use 5 test cases*
- *For height of 10, use 4 test cases*
- *For height of 15, use 3 test cases*
- *For height of 20, use 2 test cases*
- *For height of 25, use 1 test case*

The evaluator shall verify the correctness of the TSF's implementation by comparing the public key generated by the TSF with that generated by a known good implementation using the same input parameters.

- *Test FCS_CKM.1/AKG:12:*
  ML-KEM Key Generation

  67. To test the TOE's ability to generate asymmetric cryptographic keys using MLKEM, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

  - *Parameter set [ML-KEM-1024]*
  - *Random seed d [32 bytes]*
  - *Random seed z [32 bytes]*

  Algorithm Functional Test

  68. For each supported parameter set the evaluator shall require the implementation under test to generate 25 key pairs using 25 different randomly generated pairs of 32-byte seed values (d, z). To determine correctness, the evaluator shall compare the resulting key pairs (ek, dk) with those generated using a known-good implementation using the same inputs.

- *Test FCS_CKM.1/AKG:13:*
  ML-DSA Key Generation

  69. To test the TOE's ability to generate asymmetric cryptographic keys using MLDSA, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

  - *Parameter set [ML-DSA-87]*
  - *Random seed [32 bytes]*

  Algorithm Functional Test

  For each supported parameter set the evaluator shall require the implementation under test to generate 25 key pairs using 25 different randomly generated 32-byte seed values. To determine correctness, the evaluator shall compare the resulting key pairs with those generated using a known-good implementation using the same inputs.

- *Test FCS_CKM.1/AKG:14:*
  XMSS Key Generation

  70. To test the TOE's ability to generate asymmetric cryptographic keys using XMSS, the evaluator shall perform the XMSS Key Generation Test using the following input parameters:

  - *Hash algorithm [SHA-256/192, SHAKE256/192, SHA-256, SHAKE256]*
  - *Tree height [10, 16, 20] (XMSS only)*

  XMSS Key Generation Test

For each supported combination of hash algorithm and tree height, the evaluator shall generate one public key for each test case. The number of test cases depends on the tree height as specified as follows:

- *For height of 10, use 5 test cases*
- *For height of 16, use 4 test cases*
- *For height of 20, use 3 test cases*
- *For height of 40, use 2 test cases*
- *For height of 60, use 1 test case*

The evaluator shall verify the correctness of the TSF's implementation by comparing values generated by the TSF with those generated by a known good implementation using the same input parameters.

Note: The number of test cases is limited due to the extreme amount of time it can take to generate XMSS trees.

FCS_CKM.2 Cryptographic Key Distribution

This is an optional component. However, applied modules or packages might redefine it as mandatory.

FCS_CKM.2.1

The TSF shall **perform** cryptographic **key establishment** in accordance with a specified cryptographic key **establishment** method:

[**selection**: *key encapsulation*, *key wrapping*, *encrypted channels*] that meets the following: none.

Application Note: Application Note 43

This requirement specifies key transport schemes. For key agreement see FCS_CKM_EXT.7. Key transport schemes refer to cases in which one party has a key to share with another party. Key encapsulation is used when ML-KEM is used as the method of key establishment. Key wrapping and encrypted channels are used in support of wireless LAN communications. Key wrapping is also used in support of MACsec.

If "key encapsulation" is selected, FCS_COP.1/KeyEncap from Annex B must be claimed, which specifies the relevant list of standards.

If "key wrapping" is selected, FCS_COP.1/KeyWrap from Annex B must be claimed, which specifies the relevant list of standards.

FCS_CKM.6 Timing and Event of Cryptographic Key Destruction

FCS_CKM.6.1

The TSF shall destroy *plaintext cryptographic keys (including keying material)* when [**selection**: *no longer needed*, *[**assignment**: other circumstances for key or keying material destruction]*].

Application Note: Application Note 12

The TOE will have mechanisms to destroy keys, including intermediate keys and key material, by using an approved method as specified in FCS_CKM.6.2. Examples of keys include intermediate keys, leaf keys, encryption keys, and signing keys. Key material includes seeds, authentication secrets, passwords, PINs, and other secret values used to derive keys.

This SFR does not apply to the public component of asymmetric key pairs or to keys that are permitted to remain stored, such as device identification keys.

FCS_CKM.6.2

The TSF shall destroy **plaintext** cryptographic keys and keying material specified by FCS_CKM.6.1 in accordance with a specified cryptographic key destruction method

[**selection**:

- *For volatile storage, the destruction shall be executed by a [**selection**:*
  - *single overwrite consisting of [**selection**: a pseudo-random pattern using the TSF's RBG (as specified in FCS_RBG.1), zeros, ones, a new value of a key, [**assignment**: some value that does not contain any CSP]]*
  - *removal of power to the memory*
  - *removal of all references to the key directly followed by a request for garbage collection*

  ];
- *For non-volatile storage [**selection**:*
  - *that consists of an invocation of an interface provided by a part of the TSF, the destruction shall be executed by: [**selection**:*
    - *logically addressing the storage location of the key and performing a [**selection**: single, [**assignment**: number of passes] - pass] overwrite consisting of [**selection**: a pseudo-random pattern using the TSF's RBG (as specified in FCS_RBG.1), zeroes, ones, a new value of the key, [**assignment**: a static or dynamic value that does not contain any CSP]]*
    - *instructing a part of the TSF to destroy the abstraction that represents the key*

    ]
  - *that employs a wear-leveling algorithm, the destruction shall be executed by a [**selection**:*
    - *single overwrite consisting of [**selection**: zeroes, ones, pseudo-random pattern, a new value of a key of the same size, [**assignment**: some value that does not contain any CSP]]*
    - *block erase*

    ];
  - *that does not employ a wear-leveling algorithm, the destruction shall be executed by a [**selection**:*
    - *[**selection**: single, [**assignment**: ST author defined multi-pass]] overwrite consisting of [**selection**: zeros, ones, pseudo-random pattern, a new value of a key of the same size, [**assignment**: some value that does not contain any CSP]] followed by a read-verify. If the read-verification of the overwritten data fails, the process shall be repeated up to [**assignment**: number of times to attempt overwrite] times, whereupon an error is returned.*

- *block erase*

]

]

]
] that meets the following: [*no standard]*.

Application Note: In the case of volatile memory, the selection “removal of all references to the key directly followed by a request for garbage collection” is used in a situation where the TSF cannot address the specific physical memory locations holding the data to be erased and therefore relies on addressing logical addresses (which frees the relevant physical addresses holding the old data) and then requesting the platform to ensure that the data in the physical addresses is no longer available for reading (i.e., the “garbage collection” referred to in the SFR text).

In parts of the selections where keys are identified as being destroyed by “a part of the TSF”, the TSS identifies the relevant part and the interface involved. The interface referenced in the requirement could take different forms for different TOEs, the most likely of which is an application programming interface to an OS kernel. There may be various levels of abstraction visible. For instance, in a given implementation the application may have access to the file system details and may be able to logically address specific memory locations. In another implementation the application may simply have a handle to a resource and can only ask another part of the TSF such as the interpreter or OS to delete the resource.

Where different key destruction methods are used for different keys and/or different destruction situations then the different methods and the keys/situations they apply to are described in the TSS (and the ST may use separate iterations of the SFR to aid clarity). The TSS describes all relevant keys used in the implementation of SFRs, including cases where the keys are stored in a non-plaintext form. In the case of non-plaintext storage, the encryption method and relevant key-encrypting-key are identified in the TSS.

The selection for destruction of data in non-volatile memory includes block erase as an option, and this option applies only to flash memory. A block erase does not require a read verify, since the mappings of logical addresses to the erased memory locations are erased, as well as the data itself.

Some selections allow the assignment of “some value that does not contain any CSP.” This means that the TOE uses some specified data not drawn from an RBG meeting FCS_RBG requirements and not being any of the values listed as other selection options. The point of the phrase “does not contain any CSP” is to ensure that the overwritten data is carefully selected and not taken from a general pool that might contain data that itself requires confidentiality protection.

Evaluation Activities

FCS_CKM.6
TSS
71. The evaluator shall examine the TSS to ensure it lists all relevant keys (describing the origin and storage location of each), all relevant key destruction situations (e.g., factory reset or device wipe function, disconnection of trusted channels, key change as part of a secure channel protocol), and the destruction method used in each case. For the purpose of this Evaluation Activity the relevant keys are those keys that are relied upon to support any of the SFRs in the Security Target. The

evaluator shall confirm that the description of keys and storage locations is consistent with the functions carried out by the TOE (e.g., that all keys for the TOE-specific secure channels and protocols, or that support FPT_APW.EXT.1 and FPT_SKP_EXT.1, are accounted for[2]). In particular, if a TOE claims not to store plaintext keys in non-volatile memory then the evaluator shall check that this is consistent with the operation of the TOE.

72. The evaluator shall check to ensure the TSS identifies how the TOE destroys keys stored as plaintext in non-volatile memory, and that the description includes identification and description of the interfaces that the TOE uses to destroy keys (e.g., file system APIs, key store APIs).

73.Note that where selections involve 'destruction of reference' (for volatile memory) or 'invocation of an interface' (for non-volatile memory) then the relevant interface definition is examined by the evaluator to ensure that the interface supports the selection(s) and description in the TSS. In the case of nonvolatile memory, the evaluator includes in their examination the relevant interface description for each media type on which plaintext keys are stored. The presence of OS-level and storage device-level swap and cache files is not examined in the current version of the Evaluation Activity.

74. Where the TSS identifies keys that are stored in a non-plaintext form, the evaluator shall check that the TSS identifies the encryption method and the keyencrypting-key used, and that the key-encrypting-key is either itself stored in an encrypted form or that it is destroyed by a method included under FCS_CKM.6.

75. The evaluator shall check that the TSS identifies any configurations or circumstances that may not conform to the key destruction requirement (see further discussion in the Guidance Documentation section below). Note that reference may be made to the Guidance Documentation for description of the detail of such cases where destruction may be prevented or delayed.

76. Where the ST specifies the use of "a static or dynamic value that does not contain any CSP" to overwrite keys, the evaluator shall examine the TSS to ensure that it describes how that pattern is obtained and used, and that this justifies the claim that the pattern does not contain any CSPs.

Guidance

77. A TOE may be subject to situations that could prevent or delay key destruction in some cases. The evaluator shall check that the guidance documentation identifies configurations or circumstances that may not strictly conform to the key destruction requirement, and that this description is consistent with the relevant parts of the TSS (and any other supporting information used). The evaluator shall check that the guidance documentation provides guidance on situations where key destruction may be delayed at the physical layer.

78. For example, when the TOE does not have full access to the physical memory, it is possible that the storage may be implementing wear-levelling and garbage collection. This may result in additional copies of the key that are logically inaccessible but persist physically. Where available, the TOE might then describe use of the TRIM command[3] and garbage collection to destroy these persistent copies upon their deletion (this would be explained in the TSS and Operational Guidance).

Tests

None

FCS_CKM_EXT.7 Cryptographic Key Agreement

FCS_CKM_EXT.7.1

The TSF shall derive shared cryptographic keys with input from multiple parties in accordance with specified cryptographic key agreement algorithms

[**selection**: *Cryptographic Key Generation Algorithm*] and specified cryptographic parameters [**selection**: *Cryptographic Algorithm Parameters*] that meet the following: [**selection**: *List of Standards*] The following table provides the allowed choices for completion of the selection operations of FCS_CKM_EXT.7.1.

Table 4: Allowed choices for FCS_CKM_EXT.7.1

Identifier	Cryptographic Key Generation Algorithm	Cryptographic Algorithm Parameters	List of Standards
DH	Finite Field Cryptography Diffie-Hellman	Static domain parameters approved for [**selection**: • *IKE Groups [**selection**: MODP-2048, MODP-3072, MODP-4096, MODP-6144, MODP-8192]* • *TLS Groups [**selection**: ffdhe-2048, ffdhe-3072, ffdhe-4096, ffdhe-6144, ffdhe-8192]*]	NIST SP 800-56A Revision 3 (Section 5.7.1.1), [**selection**: *RFC 3526 [IKE groups], RFC 7919 [TLS groups]*]
ECDH	Elliptic Curve Diffie-Hellman	Elliptic Curve [**selection**: *P-256, P-384, P-521*]	NIST SP 800-56A Revision 3 (Section 5.7.1.2) [ECDH]NIST SP 800-186 (Section 3.2.1) [NIST Curves]

Application Note: Application Note 14

This requirement specifies key transport schemes. Key agreement schemes refer to cases in which two or more parties want to establish a single key between them, and all parties contribute to the entropy of the agreed-upon key.

The ST author selects all key agreement schemes used for the selected cryptographic protocols.

The elliptic curves used for the key agreement scheme correlate with the curves specified in FCS_CKM.1.1/AKG.

The static domain parameters approved for the finite field-based key agreement scheme are specified by the key generation according to FCS_CKM.1.1/AKG.

For Key Transport, see FCS_CKM.2 in Annex A.

Evaluation Activities

FCS_CKM_EXT.7

TSS

80. The evaluator shall ensure that the TSS documents that the security strength of the material contributed by the TOE is sufficient for the security strength of the key and the agreement method.

81. The intent of this activity is to be able to identify the scheme being used by each service. This would mean, for example, one way to document scheme usage could be as shown in the table below. The information provided in this example does not necessarily have to be included as a table but can be presented in other ways as long as the necessary data is available. The relevant SFR citation may come from a functional package if the TOE boundary includes any functional packages that define uses of key establishment schemes. For example, FCS_TLSS_EXT.1 is referenced below, which would be appropriate if the TOE includes that SFR claim as part of conformance to the Functional Package for TLS.

<table>

Guidance

82. The evaluator shall verify that the AGD guidance instructs the administrator how to configure the TOE to use the selected key agreement method(s).

Tests

83. The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

- *Test FCS_CKM_EXT.7:1:*

  FFC Diffie-Hellman Key Agreement

  84. To test the TOE's implementation of FFC Diffie-Hellman Key Agreement, the evaluator shall perform the Algorithm Functional Test and Validation Test using the following input parameters:

  - *Domain Parameter Group [MODP-2048, MODP-3072, MODP-4096, MODP-6144, MODP-8192, ffdhe3072, ffdhe4096, ffdhe6144, ffdhe8192]*

- *Test FCS_CKM_EXT.7:2:*

  Algorithm Functional Test

  85. For each supported domain parameter group, the evaluator shall generate 10 test cases by generating the initiator and responder secret keys using random data, calculating the responder public key, and creating the shared secret. The resulting shared secrets shall be compared with those generated by a knowngood implementation using the same inputs.

- *Test FCS_CKM_EXT.7:3:*

  Validation Test

  86. For each supported combination of the above parameters the evaluator shall generate 15 Diffie Hellman initiator/responder key pairs using the key generation function of a known-

good implementation. For each set of key pairs, the evaluator shall modify five initiator private key values. The remaining key values are left unchanged (i.e., correct). To determine correctness, the evaluator shall confirm that the 15 shared secrets correspond as expected for both the modified and unmodified inputs.

- *Test FCS_CKM_EXT.7:4:*
  Elliptic Curve Diffie-Hellman Key Agreement

  87. To test the TOE's implementation of Elliptic Curve Diffie-Hellman Key Agreement, the evaluator shall perform the Algorithm Functional Test and Validation Test using the following input parameters:

  - *Elliptic Curve [P-256, P-384, P-521]*

- *Test FCS_CKM_EXT.7:5:*
  Algorithm Functional Test

  88. For each supported Elliptic Curve the evaluator shall generate 10 test cases by generating the initiator and responder secret keys using random data, calculating the responder public key, and creating the shared secret. The resulting shared secrets shall be compared with those generated by a knowngood implementation using the same inputs.

- *Test FCS_CKM_EXT.7:6:*
  Validation Test

  89. For each supported Elliptic Curve the evaluator shall generate 15 Diffie Hellman initiator/responder key pairs using the key generation function of a knowngood implementation. For each set of key pairs, the evaluator shall modify five initiator private key values. The remaining key values are left unchanged (i.e., correct). To determine correctness, the evaluator shall confirm that the 15 shared secrets correspond as expected for the modified and unmodified values.

FCS_COP.1/AEAD Cryptographic Operation - Authenticated Encryption with Associated Data

This is a selection-based component. Its inclusion depends upon selection from FCS_COP.1.1/DataEncryption.

FCS_COP.1.1/AEAD

The TSF shall perform *authenticated encryption with associated data* in accordance with a specified cryptographic algorithm

[**selection**: *Cryptographic algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*] .

the following table provides the allowed choices for completion of the selection operations of FCS_COP.1/AEAD.

Table 5: Allowed choices for FCS_COP.1/AEAD

Identifier	Cryptographic algorithm	Cryptographic key sizes	List of standards

AES-CCM	AES in CCM mode with unpredictable, non-repeating nonce, minimum size of 64 bits	[**selection**: *128, 256*] bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (Clause 7), NIST SP 800-38C*] [CCM]
AES-GCM	AES in GCM mode with non-repeating IVs using [**selection**: *deterministic, RBG-based*],IV construction; the tag must be of length [**selection**: *96, 104, 112, 120, 128*] bits.	[**selection**: *128, 256*] bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (Clause 10), NIST SP 800-38D*] [GCM]

Application Note: Application Note 51
The ST author should choose the cryptographic algorithms in which AES operates, parameters, and standards implemented to perform symmetric-key authenticated encryption/decryption.

Evaluation Activities

FCS_COP.1/AEAD
TSS
The evaluator shall examine the TSS to ensure that it describes the construction of any IVs, nonces, and tags in conformance with the relevant specifications.

If a CCM mode algorithm is selected, then the evaluator shall examine the TOE summary specification to confirm that it describes how the nonce is generated and that the same nonce is never reused to encrypt different plaintext pairs under the same key.

If a GCM mode algorithm is selected, then the evaluator shall examine the TOE summary specification to confirm that it describes how the IV is generated and that the same IV is never reused to encrypt different plaintext pairs under the same key. The evaluator shall also confirm that for each invocation of GCM, the length of the plaintext is at most (2^{32})-2 blocks.

Guidance
There are no additional Guidance evaluation activities for this component.

Tests

The following tests may require the developer to provide access to a test platform that provides the evaluator with tools that are typically not found on factory products. The following tests are conditional based on the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

AES-CCM

Identifier	***Cryptographic Algorithm***	***Cryptographic Key Sizes***	***List of Standards***
AES-CCM	*AES in CCM mode with nonrepeating nonce, minimum size of 64 bits*	*256 bits*	*[**selection:** ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES]* *[**selection:** ISO/IEC 19772:2020 (Clause 7), NIST SP 800-38C] [CCM]*

To test the TOE's implementation of AES-CCM authenticated encryption functionality the evaluator shall perform the Algorithm Functional Tests described below using the following input parameters:

- *Key Size [256] bits*
- *Associated data size [0-65536] bits in increments of 8*
- *Payload size [0-256] bits in increments of 8*
- *IV/Nonce size [64-104] bits in increments of 8*
- *Tag size [32-128] bits in increments of 16*

Algorithm Functional Tests *Unless otherwise specified, the following tests should use random data, a tag size of 128 bits, IV/Nonce size of 104 bits, payload size of 256 bits, and associated data size of 256 bits. If any of these values are not supported, any supported value may be used. The evaluator shall compare the output from each test case against results generated by a known-good implementation with the same input parameters.*

Variable Associated Data Test *For each claimed key size, and for each supported associated data size from 0 through 256 bits in increments of 8 bits, the TOE must be tested by encrypting 10 test cases using all random data. In addition, for each key size, the TOE must be tested by encrypting 10 cases with associated data lengths of 65536 bits, if supported.*

Variable Payload Test *For each claimed key size, and for each supported payload size from 0 through 256 bits in increments of 8 bits, the TOE must be tested by encrypting 10 test cases using all random data.*

Variable Nonce Test *For each claimed key size, and for each supported IV/Nonce size from 64 through 104 bits in increments of 8 bits, the TOE must be tested by encrypting 10 test cases using all random data.*

Variable Tag Test *For each claimed key size, and for each supported tag size from 32 through 128 bits in increments of 16 bits, the TOE must be tested by encrypting 10 test cases using all random data.*

Decryption Verification Test *For each claimed key size, for each supported associated data size from 0 through 256 bits in increments of 8 bits, for each supported payload size from 0 through 256 bits in increments of 8 bits, for each supported IV/Nonce size from 64 through 104 bits in increments of 8 bits, and for each supported tag size from 32 through 128 bits in increments of 16 bits, the TOE must be tested by decrypting 10 test cases using all random data.*

AES-GCM

Identifier	*Cryptographic Algorithm*	*Cryptographic Key Sizes*	*List of Standards*
AES-GCM	*AES in GCM mode with nonrepeating IVs using [**selection:** deterministic, DRBG-based] IV construction; the tag must be of length [**selection:** 96, 104, 112, 120, or 128] bits.*	*256 bits*	*[**selection:** ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES]* *[**selection:** ISO/IEC 19772:2020 (Clause 10), NIST SP 800-38D] [GCM]*

To test the TOE's implementation of AES-GCM authenticated encryption functionality the evaluator shall perform the Encryption Algorithm Functional Tests and Decryption Algorithm Functional Tests as described below using the following input parameters.

- *Key Size [256] bits*
- *Associated data size [0-65536] bits*
- *Payload size [0-65536] bits*
- *IV size [96] bits*
- *Tag size [96, 104, 112, 120, 128] bits*

***Encryption Algorithm Functional Tests** The evaluator shall generate 15 test cases using random data for each combination of the above parameters as follows:*

- *Each claimed key size,*
- *Each supported tag size,*
- *Four supported non-zero payload sizes, such that two are multiples of 128 bits and two are not multiples of 128 bits,*
- *Four supported non-zero associated data sizes, such that two are multiples of 128 bits and two are not multiples of 128 bits, and*
- *An associated data size of zero, if supported.*

Note that the IV size is always 96 bits. The evaluator shall compare the output from each test case against results generated by a known- good implementation with the same input parameters.
***Decryption Algorithm Functional Tests** The evaluator shall test the authenticated decrypt functionality of AES-GCM by supplying 15 test cases for the supported combinations of the parameters as described above. For each parameter combination the evaluator shall introduce an error into either the ciphertext or the Tag such that approximately half of the cases are correct and half the cases contain errors.*

FCS_COP.1/CMAC Cryptographic Operation - CMAC

This is a selection-based component. Its inclusion depends upon selection from FCS_NTP_EXT.1.2.

FCS_COP.1.1/CMAC

The TSF shall perform *CMAC* in accordance with a specified cryptographic algorithm *AES using CMAC mode* and cryptographic key sizes *128 bits* that meet the following:

[**selection**: *ISO/IEC 9797-1:2011 subclause 7.6 (CMAC) and ISO/IEC 18033-3:2010 subclause 5.2 (AES), NIST SP 800-38B (CMAC) and NIST FIPS 197 (AES)*].

FCS_COP.1/DataEncryption Cryptographic Operation (AES Data Encryption/Decryption)

FCS_COP.1.1/DataEncryption

The TSF shall perform *encryption/decryption* in accordance with a specified cryptographic algorithm AES **operating in**

[**selection**: *CBC mode as defined in FCS_COP.1/SKC, CTR mode as defined in FCS_COP.1/SKC, XTS mode as defined in FCS_COP.1/SKC, CCM mode as defined in FCS_COP.1/AEAD, GCM mode as defined in FCS_COP.1/AEAD*].

Application Note: Application Note 15

The ST author selects the mode or modes in which AES operates. If CBC mode, CTR mode, or XTS mode is selected then FCS_COP.1/SKC from Annex B must be included. If CCM mode or GCM mode is selected, then FCS_COP.1/AEAD from Annex B must be included.

Evaluation Activities

FCS_COP.1/DataEncryption

TSS

90. The evaluator shall examine the TSS to ensure it identifies the mode or modes in which AES operates.

Guidance

91. There are no additional Guidance evaluation activities for this component.

Tests

92. There are no additional Test evaluation activities for this component.

FCS_COP.1/Hash Cryptographic Operation (Hash Algorithm)

FCS_COP.1.1/Hash

The TSF shall perform *cryptographic hashing* in accordance with a specified cryptographic algorithm

[**selection**: *SHA-256, SHA-384, SHA-512, SHA3-256, SHA3-384, SHA3-512*] that meets the following: [**selection**: *ISO/IEC 10118-3:2018 [SHA, SHA3], FIPS PUB 180-4 [SHA], FIPS PUB 202 [SHA3]*].

Application Note: Application Note 18

The hash function selection should have an output length that is the same or greater than the security strength of the algorithm used for signature generation. For example, the TOE should choose SHA-384 for 3072-bit RSA, 4096-bit RSA, or ECC with P-384; and SHA-512 for ECC with P-521. The ST author selects the standard based on the algorithms selected. For FCS_COP.1.1/Hash, SHA3 hashes may be used only for image signing or boot integrity verification.

Evaluation Activities

FCS_COP.1/Hash

TSS

135. The evaluator shall check that the association of the hash function with other TSF cryptographic functions (for example, the digital signature verification function) is documented in the TSS.

Guidance

136. The evaluator shall check the AGD documents to determine that any configuration that is required to configure the required hash sizes is present.

Tests

137. The following tests may require the developer to provide access to a test platform that provides the evaluator with tools that are typically not found on factory products.

138. The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

- *Test FCS_COP.1/Hash:1:*
  SHA-256, SHA-384, SHA-512

  139. To test the TOE's ability to generate hash digests using SHA2 the evaluator shall perform the Algorithm Functional Test, Monte Carlo Test, and Large Data Test for each claimed SHA2 algorithm.

  Algorithm Functional Test

  140. The evaluator shall generate a number of test cases equal to the block size of the hash (512 for SHA2-256; 1024 for the other SHA2 algorithms).

  141. Each test case is to consist of random data of a random length between 0 and 65536 bits, or the largest size supported.

  142. Each test case is to consist of random data of a random length between 0 and 65536 bits, or the largest size supported.

  Monte Carlo Test

  143. Monte Carlo tests begin with a single seed and run 100 iterations of the chained computation.

  144. There are two versions of the Monte Carlo test for SHA-1 and SHA-2. Either one is acceptable. For the Standard Monte Carlo test the message hashed is always three times the length of the initial seed.

```
For j = 0 to 99
       A = B = C = SEED
       For i = 0 to 999
                MSG = A || B || C
                MD = SHA(MSG)
```

```
                A = B
                B = C
                C = MD
        Output MD
        SEED = MD
```

145. For the alternate version of the Monte Carlo Test, the hashed message is always the same length as the seed.

```
INITIAL_SEED_LENGTH = LEN(SEED)
For j = 0 to 99
       A = B = C = SEED
       For i = 0 to 999
                MSG = A || B || C
                if LEN(MSG) >= INITIAL_SEED_LENGTH:
                        MSG = leftmost INITIAL_SEED_LENGTH bits of MSG
                else:
                        MSG = MSG || INITIAL_SEED_LENGTH - LEN(MSG) 0 bits
                MD = SHA(MSG)
                A = B
                B = C
                C = MD
        Output MD
        SEED = MD
```

146. The evaluator shall compare the output against results generated by a known-good implementation with the same input

Large Data Test

147. The implementation must be tested against one test case each on large data messages of 1GB, 2GB, 4GB, and 8GB of data as supported. The data need not be random. It may, for example, consist of a repeated pattern of 64 bits.

148. The evaluator shall compare the output against results generated by a known-good implementation with the same input.

- *Test FCS_COP.1/Hash:2:*
  SHA3-384, SHA3-512

  149. To test the TOE's ability to generate hash digests using SHA3 the evaluator shall perform the Algorithm Functional Test, Monte Carlo Test, and Large Data Tests for each claimed SHA3 algorithm.

  Algorithm Functional Test

  150. Generate a test case consisting of random data for every message length from 0 bits (or the smallest supported message size) to rate bits, where rate equals

  - *832 for SHA3-384 and*
  - *576 for SHA3-512.*

  151. Additionally, generate tests cases of random data for messages of every multiple of (rate+1) bits starting at length rate, and continuing until 65535 is exceeded.

  152. The evaluator shall compare the output against results generated by a known-good implementation with the same input.

  Monte Carlo Test

  153. Monte Carlo tests begin with a single seed and run 100 iterations of the chained computation.

  154. For this Monte Carlo Test, the hashed message is always the same length as the seed.

```
MD[0] = SEED
INITIAL_SEED_LENGTH = LEN(SEED)
```

```
For 100 iterations
       For i = 1 to 1000
                MSG = MD[i-1];
                if LEN(MSG) >= INITIAL_SEED_LENGTH:
                        MSG = leftmost INITIAL_SEED_LENGTH bits of MSG
                else:
                        MSG = MSG || INITIAL_SEED_LENGTH - LEN(MSG) 0 bits
                MD[i] = SHA3(MSG)
         MD[0] = MD[1000]
         Output MD[0]
```

155. The evaluator shall compare the output against results generated by a known-good implementation with the same input.

Large Data Test

156. The implementation must be tested against one test case each on large data messages of 1GB, 2GB, 4GB, and 8GB of data as supported. The data need not be random. It may, for example, consist of a repeated pattern of 64 bits.

157. The evaluator shall compare the output against results generated by a known-good implementation with the same input.

FCS_COP.1/KeyedHash Cryptographic Operation (Keyed Hash Algorithm)

FCS_COP.1.1/KeyedHash

The TSF shall perform [*keyed hash message authentication*] in accordance with a specified cryptographic algorithm [**selection**: *Keyed Hash Algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*] .

provides the allowable choices for completion of the selection operations of FCS_COP.1/KeyedHash.

Table 6: Allowable choices for FCS_COP.1/KeyedHash

Keyed Hash Algorithm	Cryptographic key sizes	List of standards
HMAC-SHA-384	[**selection**: *384 (ISO, FIPS), 256 (FIPS)*] bits	[**selection**: *ISO/IEC 9797-2:2021 (Section 7 "MAC Algorithm 2"), FIPS PUB 198-1*]
HMAC-SHA-512	[**selection**: *512 (ISO, FIPS), 384 (FIPS), 256 (FIPS)*] bits	[**selection**: *ISO/IEC 9797-2:2021 (Section 7 "MAC Algorithm 2"), FIPS PUB 198-1*]
HMAC-SHA-256	256 bits	[**selection**: *ISO/IEC 9797-2:2021 (Section 7 "MAC Algorithm 2"), FIPS PUB 198-1*]

Application Note: Application Note 19

The HMAC minimum key sizes in the table are specified in ISO/IEC 9797-2:2021, which requires that the minimum key size be equal to the digest size. The FIPS standard specifies no minimum or maximum key sizes, so if FIPS PUB 198-1 is selected, larger or smaller key sizes may be used. This is indicted by the parenthesized annotations in the Cryptographic Key Sizes column. Select 'implicit' in

cases where keyed-hash message authentication is done implicitly (e.g., SSH using AES in GCM mode).

Evaluation Activities

FCS_COP.1/KeyedHash
TSS
158. The evaluator shall examine the TSS to ensure that the size of the key is sufficient for the desired security strength of the output.

Guidance
159. There are no additional Guidance evaluation activities for this component.

Tests
160. The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

- *Test FCS_COP.1/KeyedHash:1:*
  HMAC

  161. To test the TOE's ability to generate keyed hashes using HMAC the evaluator shall perform the Algorithm Functional Test for each combination of claimed HMAC algorithm the following parameters:

  - *Hash function [SHA-256, SHA-384, SHA-512]*
  - *Key length [8-65536] bits by 8s*
  - *MAC length [32-[digest size of hash function (256, 384, 512)]] bits*

  Algorithm Functional Test

  162. For each supported Hash function the evaluator shall generate 150 test cases using random input messages of 128 bits, random supported key lengths, random keys, and random supported MAC lengths such that across the 150 test cases:

  - *The key length includes the minimum, the maximum, a key length equal to the block size, and key lengths that are both larger and smaller than the block size.*
  - *The MAC size includes the minimum, the maximum, and two other random values.*

  163. The evaluator shall compare the output against results generated by a known-good implementation with the same input.

FCS_COP.1/KeyEncap Cryptographic Operation - Key Encapsulation

This is a selection-based component. Its inclusion depends upon selection from FCS_CKM.2.1.

FCS_COP.1.1/KeyEncap

The TSF shall perform *key encapsulation* in accordance with a specified cryptographic algorithm

[**selection**: *Cryptographic Algorithm*] and cryptographic key sizes [**selection**: *Cryptographic Key Sizes*] that meet the following: [**selection**: *List of Standards*] .
The following table provides the allowed choices for completion of the selection operations of FCS_COP.1/KeyEncap.

Table 7: Allowed choices for FCS_COP.1/KeyEncap

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
ML-KEM	ML-KEM	Parameter set = ML-KEM-1024	NIST FIPS 203

Application Note: Application Note 52
The only anticipated use of key encapsulation is the use of ML-KEM as part of key establishment for trusted communication.

FCS_COP.1/KeyWrap Cryptographic Operation - Key Wrapping

This is a selection-based component. Its inclusion depends upon selection from FCS_CKM.2.1.

FCS_COP.1.1/KeyWrap

The TSF shall perform *key wrapping* in accordance with a specified cryptographic algorithm

[**selection**: *Cryptographic Algorithm*] and cryptographic key sizes [**selection**: *Cryptographic Key Sizes*] that meet the following: [**selection**: *List of Standards*] .

The following table provides the allowed choices for completion of the selection operations of FCS_COP.1/KeyWrap.

Table 8: Allowed choices for FCS_COP.1/KeyWrap

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
AES-KW	AES in KW mode	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (clause 6), NIST SP 800-38F (Section 6.2)*] [KW mode]

AES-KWP	AES in KWP mode	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] NIST SP 800-38F (Section 6.3) [KWP mode]
AES-CCM	AES in CCM mode with unpredictable, non-repeating nonce, minimum size of 64 bits	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (Clause 7), NIST SP 800-38C*] [CCM]
AES-GCM	AES in GCM mode with non-repeating IVs using [**selection**: *deterministic*, *RBG-based*],IV construction; the tag must be of length [**selection**: *96*, *104*, *112*, *120*, *128*] bits.	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (Clause 10), NIST SP 800-38D*] [GCM]

Application Note: Application Note 53
NIST SP 800-57 Part 1 Revision 5, Section 5.6.2 specifies that the size of key used to protect the key being transported should be at least the security strength of the key it is protecting.

Evaluation Activities

FCS_COP.1/KeyWrap
TSS
The evaluator shall ensure that the TSS documents that the selection of the key size is sufficient for the security strength of the key wrapped.

The evaluator shall examine the TSS to ensure that it describes the construction of any IVs, nonces, and MACs in conformance with the relevant specifications.

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
For tests of AES-GCM and AES-CCM, see testing for FCS_COP.1/AEAD. The following tests are conditional based on the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.
AES-KW

Identifier	***Cryptographic Algorithm***	***Cryptographic Key Sizes***	***List of Standards***
AES-KW	*AES in KW mode*	*256 bits*	*[**selection:** ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES]* *[**selection:** ISO/IEC 19772:2020 (clause 6), NIST SP 800-38F (Section 6.2)] [KW mode]*

To test the TOE's ability to wrap keys using AES in Key Wrap mode the evaluator shall perform the Algorithm Functional Tests using the following input parameters.

- *Key size [256] bits*
- *Keyword cipher type [cipher, inverse]*
- *Payload sizes [128-4096] bits by 64s*

Algorithm Functional Test *The evaluator shall generate 100 encryption test cases using random data for each combination of claimed key size, keyword cipher type, and six supported payload sizes such that the payload sizes include the minimum, the maximum, two that are divisible by 128, and two that are not divisible by 128. The results shall be compared with those generated by a known-good implementation using the same inputs. The evaluator shall generate 100 decryption test cases using the same parameters as above, but with 20 of each 100 test cases having modified ciphertext to produce an incorrect result. To determine correctness, the evaluator shall confirm that the results correspond as expected for both the modified and unmodified values.*
AES-KWP

Identifier	***Cryptographic Algorithm***	***Cryptographic Key Sizes***	***List of Standards***
AES-KWP	*AES in KWP mode*	*256 bits*	*[**selection:** ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES]* *NIST SP 800-38F (Section 6.3) [KWP mode]*

To test the TOE's ability to wrap keys using AES in Key Wrap with Padding mode with padding the evaluator shall perform the Algorithm Functional Tests using the following input parameters.

- *Key size [256] bits*
- *Keyword cipher type [cipher, inverse]*
- *Payload sizes [8-4096] bits by 8s*

Algorithm Functional Test *The evaluator shall generate 100 encryption test cases using random data for each combination of claimed key size, keyword cipher type, and six supported payload sizes such that the payload sizes include the minimum, the maximum, two that are divisible by 128, and*

two that are not divisible by 128. The results shall be compared with those generated by a known-good implementation using the same inputs. The evaluator shall generate 100 decryption test cases using the same parameters as above, but with 20 of each 100 test cases having modified ciphertext to produce an incorrect result. To determine correctness, the evaluator shall confirm that the results correspond as expected for both the modified and unmodified values.

FCS_COP.1/SigGen Cryptographic Operation - Signature Generation

FCS_COP.1.1/SigGen

The TSF shall perform [*digital signature generation*] in accordance with a specified cryptographic algorithm [**selection**: *Cryptographic algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*] .

The following table provides the allowable choices for completion of the selection operations of FCS_COP.1/SigGen.

Table 9: Allowable choices for FCS_COP.1/SigGen

Identifier	Cryptographic algorithm	Cryptographic key sizes	List of standards
RSA-PKCS	RSASSA-PKCS1-v1_5	Modulus of size [**selection**: *2048, 3072, 4096, 6144, 8192*] bits and hash [**selection**: *SHA-256, SHA-384, SHA-512*]	RFC 8017 (Section 8.2) [PKCS #1 v2.2] FIPS PUB 186-5 (Section 5.4) [RSASSA-PKCS1-v1_5]
RSA-PSS	RSASSA-PSS	Modulus of size [**selection**: *2048, 3072, 4096, 6144, 8192*] bits and hash [**selection**: *SHA-256, SHA-384, SHA-512*],Salt Length (*sLen*) such that [**assignment**: *0 ≤ sLen ≤ hLen (Hash Output Length)*] and Mask Generation Function = MGF1	RFC 8017 (Section 8.1) [PKCS#1 v2.2] FIPS PUB 186-5 (Section 5.4) [RSASSA-PSS]
ECDSA	ECDSA	Elliptic Curve [**selection**: *P-256, P-384, P-521*],per-message secret number generation [**selection**: *extra random bits, rejection sampling, deterministic*] and hash function using [**selection**: *SHA-256, SHA-384, SHA-512*]	[**selection**: *ISO/IEC 14888-3:2018 (Subclause 6.6), FIPS PUB 186-5 (Sections 6.3.1, 6.4.1)*] [ECDSA] NIST SP-800 186 (Section 4) [NIST Curves]

ML-DSA	ML-DSA Signature Generation	Parameter set = ML-DSA-87	NIST FIPS 204 (Section 5.2)

Application Note: Application Note 16

The ST author should choose the cryptographic algorithms, parameters, and standards implemented to perform digital signature generation. For the algorithm chosen, the ST author should make the appropriate assignments/selections to specify the parameters that are implemented for that algorithm.

Evaluation Activities

FCS_COP.1/SigGen

TSS

93. The evaluator shall examine the TSS and verify that any hash function is the appropriate security strength for the signing algorithm.

94. The evaluator shall examine the TSS to verify that any one-time values such as nonces or masks are constructed and used in accordance with the relevant standards.

95. The evaluator shall examine the TSS to verify that the TOE has appropriate measures in place to ensure that hash-based signature algorithms do not reuse private keys.

96. The evaluator shall examine the TSS to determine that it specifies the cryptographic algorithm(s) and key size(s) supported by the TOE for signature generation services.

Guidance

97. There are no additional Guidance evaluation activities for this component.

Tests

98. The following tests are conditional based upon the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

- *Test FCS_COP.1/SigGen:1:*
  RSA-PKCS Signature Generation

  99. To test the TOE's ability to perform RSA Digital Signature Generation using PKCS1-v1,5 signature type, the evaluator shall perform the Generated Data Test using the following input parameters:

  - *Modulus size [2048, 3072, 4096, 6144, 8192] bits*

- *Hash algorithm [SHA-256, SHA-384, SHA-512]*

Generated Data Test

100. For each supported combination of the above parameters, the evaluator shall cause the TOE to generate three test cases using random data. The evaluator shall compare the results against those from a known-good implementation.

- *Test FCS_COP.1/SigGen:2:*
  RSA-PSS Signature Generation

  101. To test the TOE's ability to perform RSA Digital Signature Generation using PSS signature type, the evaluator shall perform the Generated Data Test using the following input parameters:

  - *Modulus size [2048, 3072, 4096, 6144, 8192] bits*
  - *Hash algorithm [SHA-256, SHA-384, SHA-512]*
  - *Salt length [Fixed based on implementation]*
  - *Mask function [MGF1]*

  Generated Data Test

  102. For each supported combination of the above parameters, the evaluator shall cause the TOE to generate three test cases using random data. The evaluator shall compare the results against those from a known-good implementation.

- *Test FCS_COP.1/SigGen:3:*
  ECDSA Signature Generation

  103. To test the TOE's ability to perform ECDSA Digital Signature Generation using extra random bits or rejection sampling for secret number generation, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

  - *Elliptic Curve [P-256, P-384, P-521]*
  - *Hash algorithm [SHA-256, SHA-384, SHA-512]*

  104. To test the TOE's ability to perform ECDSA Digital Signature Generation using deterministic secret number generation, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

  - *Elliptic Curve [P-256, P-384, P-521]*
  - *Hash algorithm [SHA-256, SHA-384, SHA-512]*

  Algorithm Functional Test

  105. For each supported combination of the above parameters, the evaluator shall cause the TOE to generate 10 test cases using random data. The evaluator shall compare the results against those from a known-good implementation.

- *Test FCS_COP.1/SigGen:4:*
  LMS Signature Generation

  106. To test the TOE's ability to generate cryptographic digital signature using LMS, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

  - *Hash algorithm [SHA-256/192, SHAKE256/192, SHA-256, SHAKE256]*
  - *Winternitz [1, 2, 4, 8]*
  - *Tree height [5, 10, 15, 20, 25]*

Algorithm Functional Test

107. For each supported combination of the above parameters, the evaluator shall generate 10 signatures. The evaluator shall verify the correctness of the implementation by comparing values generated by the TOE with those generated by a known good implementation using the same input parameters.

- *Test FCS_COP.1/SigGen:5:*
  ML-DSA Signature Generation

  108. To test the TOE's ability to generate digital signatures using ML-DSA, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

  - *Parameter set [ML-DSA-87]*
  - *Seed [32 random bytes] (for non-deterministic signature testing), or*
  - *Seed [32 zero bytes] (for deterministic signature testing)*
  - *Message to sign [8-65535] bytes*
  - *Mu value (if generated externally)*
  - *Previously generated private key (sk)*
  - *Context (for external interface testing)*

  Algorithm Functional Test

  109. For each combination of supported parameter set and capabilities, the evaluator shall require the implementation under test to generate 15 signatures pairs using 15 different randomly generated 32-byte seed values. To determine correctness, the evaluator shall compare the resulting key pairs with those generated using a known-good implementation using the same inputs.

- *Test FCS_COP.1/SigGen:6:*
  Known Answer Test for Rejection Cases

  110. For each supported parameter set, the evaluator shall cause the TOE to generate signatures using the data below and a deterministic seed of all 0's. Correctness is determined by comparing the hash of the resulting signature with the hash in the fourth row for each corresponding test case below.

  The test values are defined as follows:

  - *Seed is the seed to generate the key pair (pk, sk)*
  - *Hash of keys is computed by SHA-256(pk||sk)*
  - *Message is the message to be signed*
  - *Hash of sig is computed by SHA-256(sig)*

  ML-DSA-87 Test Cases for Rejection Cases

```
Test case 87-RC-01
Seed: E4F5AFCF697E0EC3C1BDEB66FAA903221E803902F9C3F716E1056A63D77DC250
Hash of Keys: 61618E8DDA6998072C8EB36974E03880D741CAF0BD523356DFC161E7C9E63934
Message: F4F1C05004D5B946F69EAFE104C4020519086ADDB9582A20FDE887D13DFC36B1
Hash of sig: B584E38FA442FC3C81A147D4BDBF058D73C822CAF5CA4C06B0110867F60A8001
Test case 87-RC-02
Seed: 8B828D871254D6C57384A8E7025AA3F7160CAD1D2C754499DF3844426062C3DD
Hash of Keys: BB64481317D6C0DBAD20C0C7EF11078AD54E5D574F4A07652115A95F77C655FA
Message: 0F9409C5A4930C25B83FC5B77FDB5BB49C75372DE724D9C1A77DB700CF0CF154
Hash of sig: F86B49BE9DEB2B209BDEB4E922E5939E92D38E562C44BB09AFBD67323C345192
Test case 87-RC-03
```

Seed: E693D282CACB8CE65FD4D108DA7A373F097F0AA9713550BE242AAD5BD3E2E452
Hash of Keys: B0BEAF56713A69BD4AB2CBEE006FA5001E7B41F3AE541E05F088933AA0CC78DF
Message: 24DABB9D57ADEBD560ED65D9451C5106D437061708F849BA53F3543CDF9AAAE0
Hash of sig: DBF65CEFF9F96A74AAF6F3AB27B043231BE6AA04FBA2EEC987A24A00BDD6A08E
Test case 87-RC-04
Seed: 4002163EB8EED01A8E0919BA8C07D291341EDCAE25B02B9779A2CFFE50561AF0
Hash of Keys: FED1BE685C20ECB322FC40D41DEE7E0E98D0409FBF989CAE71B8AD2D58AD645E
Message: EE316BB5EBED53325B4A55571C60657B53E353B51B831F4A0BBB28107EBA4BA8
Hash of sig: 3BE9B5545FDCED92547B3409C83B3312CCB5792A8EC3A4DA63BA692C79BEF17C
Test case 87-RC-05
Seed: 9C7AD524F65854C27E565BCEDF8E86D650F13A40D0448F9AE10C05F10F777120
Hash of Keys: 0EA872CA5A4BEA94F4E8EF7ED31800727899A51059FDEE111E5CB15F0233B534
Message: CE09831294AA96CAF684B9E667947B021C57B24C138EC7D4DA270694C82F2E08
Hash of sig: 3B9526CEE6587F2418BFE603ADB0F7DF0D69EBA31C9F9F005C60C993945EBD33
Test case 87-RC-06
Seed: 2EB7676D4A28700DA7772A7A035EB495CAA6F842352A74824EF5FD891BC38B2A
Hash of Keys: D5B73703A1DDC5BCB0D14AE39B193A25D6ADA6535827973181ADB0BE70435A5B
Message: C2B3A0AC483A5517682285C205974B2A506946448A8F7D3E1934C155EFDFE922
Hash of sig: 375D598704B722C8A1FEF1626FD7738A532C06329AA4217357460E3B729660F8
Test case 87-RC-07
Seed: E4E80CCE8B26DF1B02B99949851EE2F907FE4F0CC34790352C76D5D91634D073
Hash of Keys: 84B7E61684A12698400B09EA332EA3C4FBCFA47FE37FD6AE725CBC5FA8A99D3F
Message: 89E6AB43C9CB1CC59C3986D53217A558357E62102A26F666F2B64CD1DBB7A536
Hash of sig: 7C4AABD163CAEF8F6EBFDA3E3EEBC0A9604675B0E991ABAFD284F1AE8BA07B2A
Test case 87-RC-08
Seed: 5787262B803499223D4E5A8C1EE572E89F7A69B359B3F8505355B0BDEAB95E5C
Hash of Keys: 85AE1DE605A7B479C02730BF4B7DD6D0FD8FFE5C980893CA6DAD00BD8BD1CE68
Message: D3230C4E061964BBFB17702432D5D36FC1EB3D1068F8CCAA84044776E3B5CC55
Hash of sig: D3ABE460EE2DD9595F413CFE2780A319E4E4DFD6592995298A7AB0B82A5E2815
Test case 87-RC-09
Seed: CE099B99330537DD153052243FC32ACAD509A126AB982410258858567D410D79
Hash of Keys: E04A9F15EDF8F078EB336CE624249EF2A8EDF2CDBF6A8276E9F5E92ED9B0BAE8
Message: 0035931762665F561A1B22176567E3B10FDE2441521F77030733A8E39312EEEE
Hash of sig: 3EEF413CB5EB179896ECA172D0DBFB9B251545DC561D61580BD5BBC8B6D734E1
Test case 87-RC-10
Seed: FC8F2929878CBD81E1CCC23913F290380120C043A4A8A251AEEBF09705B8E590
Hash of Keys: 7E2ECCA86F532E8E8092FEBB6E0007F92E7909AD2BCBE2E02AB375DAC9969E5E
Message: D3C28875D2671C0EF23BFDC8869E8ECF8868D3F0561C3134D254F7479D0CE0E5
Hash of sig: EB69A908EDCC04320A0B61AD57E21B044465F2037698636B64229CF2DB259789

- *Test FCS_COP.1/SigGen:7:*

  Known Answer Test for Large Number of Rejection Cases (Total Rejection Count)

  111. For each supported parameter set, the evaluator shall cause the TOE to generate signatures using the data below and a deterministic seed of all 0's. Correctness is determined by comparing the hash of the resulting signature with the hash in the fourth row of the corresponding test case below.

  ML-DSA-87 Test Cases for Total Rejection Count

  Test case 87-LN-01
  Seed: 98B6298051D92BF37293C93C97370747BF527B87B71F6C4264182F45155ADE4C
  Hash of Keys: 04A135B5C9B7020332C7B16E7108E8FF7FC1EAE1C23C5FA0B5D5CED0FEEE7424
  Message: D7B0341269259083ABF3C8DC47559A19D57669B4486E0224F376DC43E577A3D8
  Hash of sig: 58D72D76EC0FB65BFB9893C4479366B79DD7B8B7577E4291D13514FCC76C26DD
  Test case 87-LN-02
  Seed: DFB5BDD90F58571DCA962426C623F13D046BBE814D183886AC90D143EAD725A7
  Hash of Keys: 2B6AB8CFCCCC41F759CAF01932E9413F5DC6D949BC827F739866929683FB155E
  Message: 21005DB2B583CC826A9684BFFD0EE00AB97E0479FE4A1D266699337540145778
  Hash of sig: C93EA34E00FFFFC3ECEA072D5FB038A83B5539CAF7B831AEDCFA785E50B3CA5E
  Test case 87-LN-03
  Seed: 5AD414E0DD0EF2FE685F342871875FDF06F503717A86C3B3466565ADD2096417
  Hash of Keys: BD9C2D52F3FC78DB17E682DA2E78947ECFC0898333838D60C892700B2B0DDA9F
  Message: 29139C279816B25F2D6BB52C8247D163544F7BA332C3CF63359B9E23FBC56515
  Hash of sig: DB4BE2DE19FB40437BDB7E9B6578D665DB05B4E88C16907DF4546EBA9BE03AEA
  Test case 87-LN-04
  Seed: 484DD2F406A4D15F49A91AD5FC3BDC1D0FF253622EB68F83D6E1C870D0E89E29
  Hash of Keys: A719DC9A77C91C46295555C2353BA0CBEA513DA9A92A5C34D2E949EFF46A12D8
  Message: 6AD6E959F0EA60126364FB7C95FA71133F246A9265A11B4965EE78AB0CB5AF0E

Hash of sig: 5050D7A665074EC63D9F3966C1F01A1BFB18F9E83AE0B09F838BC1E2342ED6F4
Test case 87-LN-05
Seed: B25C1816F82D59940D5CB829BAC364AAD013C4C16415CE1CF6DCC2F15199B391
Hash of Keys: ADBB2CD43F222640BD9FF4E61C80E63853E8DC1F759C581B7447C9C166EAA38E
Message: 824E47322895BFFE37B6B4AFC41CF6115C07EEC0C24EB81076C87A1B01AE8617
Hash of sig: 667ADA46073BC69D64DC47BB9A76DD0D78302E7415D87D5E816B05FB95F9E84D
Test case 87-LN-06
Seed: B2CE72B3560AF07E06465881F56ADA00262BA708D87B73F39E04E310F3B8A3E9
Hash of Keys: FD9C4AC53AE803242A62DF933B8E8BAD6CE5207AC4A73683B6D9383B5E70B17A
Message: A1501CC84C917E0D2D7C27C2AC382220BD8FFFE807DB38E37A9E429EC2781911
Hash of sig: 779553B195E11558EE59EF3942F5F6B446A2144600D1F4F50B300C6C56504760
Test case 87-LN-07
Seed: AB01D0E591B7DDCD3C03395AED808FA2763C0A486D44119D621BE0FD0B022B25
Hash of Keys: 93B6ADE34F78A4ADB36B2F6D2C51DB793E659E1243E80488AE1C03B65125D6D7
Message: 8DE8122D89D15FE84A4C34F6B59B2C4B11F33B6A053154D199B634F557FDF5F6
Hash of sig: 0483045999A79B583F403DB96A736F0F0B24E2DFBC4E5CFA9B50E3D910786F07
Test case 87-LN-08
Seed: 15D60D3693762F82C9AC1DCB0576936651AC81D863842EDB91109C8EE83AE705
Hash of Keys: 2DF544E2E939AA717741C2437288FAEB308DEB8FF37A2652FAE34BAE8B84D779
Message: F05946A6113905C34163AEF2246FD69016CE24A7BA40F8E7E42EDAC2D0A44605
Hash of sig: F8383917AF79C8E540D2356AB05F08B465BF32DFEC444B787CE31BF48CC6C3DD
Test case 87-LN-09
Seed: 21212285BED53B3411705DAF5F3BDDB6F0618EB571B36EE11A74053407A269F5
Hash of Keys: 737061155A9A03F11F9FEBBB940BED4DD54542C4A6212F89A5EB4EC2BE542782
Message: FFE38246BF3DEFD9CAD15CC17CEA511C067D582E04227B479E32F9197CF91482
Hash of sig: C4C12C58032052FB2D21F0C6A7388A63154FB85B74287D2859DE6C1C6F7F277B
Test case 87-LN-10
Seed: A2744470587C71BA43EC26DC390CE3531978F315993C653E5D3EFD2849D5D9F1
Hash of Keys: B1BF37BFFB11531B6ADD697870D7DB2E2462D0A97A63F09C1D0038457C6D795A
Message: 9831A830231A160B9847203341A5F30BF3E87A2A482AEEA6886315C92B5C4E4C
Hash of sig: 46C669D2FEB643A38E54FF87B790CC33F44043A1B6B31DB9474D301328CA2A7F

- *Test FCS_COP.1/SigGen:8:*

  XMSS Signature Generation

  112. To test the TOE's ability to generate digital signatures using XMSS, the evaluator shall perform the XMSS Key Generation Test using the following input parameters:

  - *Hash algorithm [SHA-256/192, SHAKE256/192, SHA-256, SHAKE256]*
  - *Tree height [10, 16, 20]*

  XMSS Key Generation Test

  113. For each supported combination of the above parameters, the evaluator shall generate 10 signatures. The evaluator shall verify the correctness of the implementation by comparing values generated by the TOE with those generated by a known-good implementation using the same input parameters.

FCS_COP.1/SigVer Cryptographic Operation - Signature Verification

FCS_COP.1.1/SigVer

The TSF shall perform [*digital signature verification*] in accordance with a specified cryptographic algorithm [**selection**: *Cryptographic algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*] .

The following table provides the allowable choices for completion of the selection operations of FCS_COP.1/SigVer.

Table 10: Allowable choices for FCS_COP.1/SigVer

Identifier	Cryptographic algorithm	Cryptographic key sizes	List of standards
RSA-PKCS	RSASSA-PKCS1-v1_5	Modulus of size [**selection**: *2048*, *3072*, *4096*, *6144*, *8192*] bits and hash [**selection**: *SHA-256, SHA-384, SHA-512*]	RFC 8017 (Section 8.2) [PKCS #1 v2.2] FIPS PUB 186-5 (Section 5.4) [RSASSA-PKCS1-v1_5]
RSA-PSS	RSASSA-PSS	Modulus of size [**selection**: *2048*, *3072*, *4096*, *6144*, *8192*] bits and hash [**selection**: *SHA-256, SHA-384, SHA-512*]	RFC 8017 (Section 8.1) [PKCS#1 v2.2] FIPS PUB 186-5 (Section 5.4) [RSASSA-PSS]
ECDSA	ECDSA	Elliptic Curve [**selection**: *P-256*, *P-384*, *P-521*] using hash [**selection**: *SHA-256, SHA-384*, *SHA-512*]	[**selection**: *ISO/IEC 14888-3:2018 (Subclause 6.6), FIPS PUB 186-5 (Section 6.4.2)*] [ECDSA] NIST SP-800 186 (Section 4) [NIST Curves]
LMS	LMS	Private key size = [**selection**: • *192 bits with [**selection**: SHA-256/192, SHAKE256/192]* • *256 bits with [**selection**: SHA-256, SHAKE256]*] Winternitz parameter = [**selection**: *1*, *2*, *4*, *8*] Tree height = [**selection**: *5*, *10*, *15*, *20*, *25*]	RFC 8554 [LMS] NIST SP 800-208 [parameters]
XMSS	XMSS	Private key size = [**selection**: • *192 bits with [**selection**: SHA-256/192, SHAKE256/192]* • *256 bits with [**selection**: SHA-256, SHAKE256]*	RFC 8391 [XMSS] NIST SP 800-208 [parameters]

		] Tree height = [**selection**: *10, 16, 20*]	
ML-DSA	ML-DSA Signature Verification	Parameter set = ML-DSA-87	NIST FIPS 204 (Section 5.3)

Application Note: Application Note 17

The ST Author should choose the algorithm implemented to perform verification of digital signatures. For the algorithm chosen, the ST Author should make the appropriate assignments/selections to specify the parameters that are implemented for that algorithm. In particular, if ECDSA is selected as one of the signature algorithms, the key size specified must match the selection for the curve used in the algorithm.

If LMS or XMSS is selected, then FCS_COP.1/XOF from Annex B must be included.

Evaluation Activities

FCS_COP.1/SigVer
TSS

114. The evaluator shall examine the TSS to verify that any one-time values such as nonces or masks are constructed and used in accordance with the relevant standards.

Guidance

115. There are no additional Guidance evaluation activities for this component.

Tests

116. The following tests are conditional based on the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.

- *Test FCS_COP.1/SigVer:1:*
  RSA-PKCS Signature Verification

  117. To test the TOE's ability to perform RSA Digital Signature Verification using PKCS1-v1,5 signature type, the evaluator shall perform the Generated Data Test using the following input parameters:

- *Modulus size [2048, 3072, 4096, 6144, 8192] bits*
- *Hash algorithm [SHA-256, SHA-384, SHA-512]*

Generated Data Test

118. For each supported combination of the above parameters, the evaluator shall cause the TOE to generate six test cases using a random message and its signature such that the test cases are modified as follows:

- *One test case is left unmodified*
- *For one test case the Message is modified*
- *For one test case the Signature is modified*
- *For one test case the exponent (e) is modified*
- *For one test case the IR is moved*
- *For one test case the Trailer is moved*

119. The TOE must correctly verify the unmodified signatures and fail to verify the modified signatures.

- *Test FCS_COP.1/SigVer:2:*
  RSA-PSS Signature Verification

  120. To test the TOE's ability to perform RSA Digital Signature Verification using PSS signature type, the evaluator shall perform the Generated Data Test using the following input parameters:

  - *Modulus size [2048, 3072, 4096, 6144, 8192] bits*
  - *Hash algorithm [SHA-256, SHA-384, SHA-512]*
  - *Salt length [Fixed based on implementation]*
  - *Mask function [MGF1]*

  Generated Data Test

  121. For each supported combination of the above parameters, the evaluator shall cause the TOE to generate six test cases using random data such that the test cases are modified as follows:

  - *One test case is left unmodified*
  - *For one test case the Message is modified*
  - *For one test case the Signature is modified*
  - *For one test case the exponent (e) is modified*
  - *For one test case the IR is moved*
  - *For one test case the Trailer is moved*

  122. The TOE must correctly verify the unmodified signatures and fail to verify the modified signatures.

- *Test FCS_COP.1/SigVer:3:*
  ECDSA Signature Verification

  123. To test the TOE's ability to perform ECDSA Digital Signature Verification, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

- *Elliptic Curve [P-256, P-384, P-521]*
- *Hash algorithm [SHA-256, SHA-384, SHA-512]*

Algorithm Functional Test

124. For each supported combination of the above parameters, the evaluator shall cause the TOE to generate test cases consisting of messages and signatures such that the 21 test cases are modified as follows:

- *Three test cases are left unmodified*
- *For three test cases the Message is modified*
- *For three test cases the key is modified*
- *For three test cases the r value is modified*
- *For three test cases the s value is modified*
- *For three test cases the value r is zeroed*
- *For three test cases the value s is zeroed*

125. The TOE must correctly verify the unmodified signatures and fail to verify the modified signatures.

- *Test FCS_COP.1/SigVer:4:*
  LMS Signature Verification

  126. To test the TOE's ability to verify cryptographic digital signature using LMS, the evaluator shall perform the Algorithm Functional Test using the following input parameters

  - *Hash algorithm [SHA-256/192, SHAKE256/192, SHA-256, SHAKE256]*
  - *Winternitz[1, 2, 4, 8]*
  - *Tree height [5, 10, 15, 20, 25]*

  Algorithm Functional Test

  127. For each supported combination of the above parameters, the evaluator shall generate 4 test cases consisting of signed messages and keys, such that

  - *One test case is unmodified (i.e. correct)*
  - *For one test case modify the message, i.e. the message is different*
  - *For one test case modify the signature, i.e. signature is different*
  - *For one test case modify the signature header so that it is a valid header for a different LMS parameter set.*

  128. The TOE must correctly verify the unmodified test case and fail to verify the modified test cases.

- *Test FCS_COP.1/SigVer:5:*
  XMSS Signature Verification

  129. To test the TOE's ability to verify digital signatures using XMSS or XMSS MT, the evaluator shall perform the XMSS digital signature verification test using the following input parameters:

  - *Hash algorithm [SHA-256/192, SHAKE256/192, SHA-256, SHAKE256]*
  - *Tree height [10, 16, 20]*

XMSS Digital Signature Verification Test

130. For each supported combination of the above parameters, the evaluator shall generate four test cases consisting of signed messages and keys, such that

- *One test case is unmodified (i.e. correct)*
- *For one test case modify the message, i.e. the message is different*
- *For one test case modify the signature, i.e. signature is different*
- *For one test case modify the signature header so that it is a valid header for a different XMSS parameter set*

131. The evaluator shall verify the correctness of the implementation by verifying that the TOE correctly verifies the unmodified test case and fails to verify the modified test cases

- *Test FCS_COP.1/SigVer:6:*
  ML-DSA Signature Verification

  132. To test the TOE's ability to validate digital signatures using ML-DSA, the evaluator shall perform the Algorithm Functional Test using the following input parameters:

  - *Parameter set [ML-DSA-87]*
  - *Previously generated signed Message [8-65535] bytes*
  - *Mu value (if generated externally)*
  - *Context (for external interface testing)*
  - *Previously generated Public key (pk)*
  - *Previously generated Signature*

  Algorithm Functional Test

  133. For each combination of supported parameter set and capabilities, the evaluator shall require the implementation under test to validate 15 signatures. Each group of 15 test cases is modified as follows:

  - *Three test cases are left unmodified*
  - *For three test cases the Signed message is modified*
  - *For three test cases the component of the signature that commits the signer to the message is modified*
  - *For three test cases the component of the signature that allows the verifier to construct the vector z is modified*
  - *For three test cases the component of the signature that allows the verifier to construct the hint array is modified*

  134. The TOE must correctly verify the unmodified signatures and fail to verify the modified signatures.

FCS_COP.1/SKC Cryptographic Operation - Symmetric Key Cryptography

This is a selection-based component. Its inclusion depends upon selection from

***FCS_COP.1.1/DataEncryption*.**

FCS_COP.1.1/SKC

The TSF shall perform *symmetric-key encryption/decryption* in accordance with a specified cryptographic algorithm

[**selection**: *Cryptographic Algorithm*] and cryptographic key sizes [**selection**: *Cryptographic Key Sizes*] that meet the following: [**selection**: *List of Standards*] .

The following table provides the allowed choices for completion of the selection operations of FCS_COP.1/SKC.

Table 11: Allowed choices for FCS_COP.1/SKC

Identifier	Cryptographic Algorithm	Cryptographic Key Sizes	List of Standards
AES-CBC	AES in CBC mode with non-repeating and unpredictable IVs	[**selection**: *128, 256*]	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 10116:2017 (Clause 7), NIST SP 800-38A*] [CBC]
XTS-AES	AES in XTS mode with unique tweak values that are consecutive non-negative integers starting at an arbitrary non-negative integer	[**selection**: *256, 512*]	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *IEEE Std. 1619-2018, NIST SP 800-38E*] [XTS]
AES-CTR	AES in Counter Mode with a non-repeating initial counter and with no repeated use of counter values across multiple messages with the same secret key	[**selection**: *128, 256*]	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *: ISO/IEC 10116:2017 (Clause 10), NIST SP 800-38A*] [CTR]

Application Note: Application Note 54
The ST author should choose the cryptographic algorithms in which AES operates, parameters, and standards implemented to perform symmetric-key encryption/decryption without built-in authentication.

Evaluation Activities

FCS_COP.1/SKC
TSS
The evaluator shall examine the TSS to ensure that it describes the construction of any IVs, tweak values, and counters in conformance with the relevant specifications.

If XTS-AES is claimed then the evaluator shall examine the TSS to verify that the TOE creates full-length keys by methods that ensure that the two key halves are distinct.

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
The following tests require the developer to provide access to a test platform that provides the evaluator with tools that are typically not found on factory products. The following tests are conditional based on the selections made in the SFR. The evaluator shall perform the following test or witness respective tests executed by the developer. The tests must be executed on a platform that is as close as practically possible to the operational platform (but which may be instrumented in terms of, for example, use of a debug mode). Where the test is not carried out on the TOE itself, the test platform shall be identified and the differences between test environment and TOE execution environment shall be described.
AES-CBC

Identifier	***Cryptographic Algorithm***	***Cryptographic Key Sizes***	***List of Standards***
AES-CBC	*AES in CBC mode with non-repeating and unpredictable IVs*	*256 bits*	*[**selection:** ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES]* *[**selection:** ISO/IEC 10116:2017 (Clause 7), NIST SP 800-38A] [CBC]*

To test the TOE's ability to encrypt/decrypt data using AES in CBC mode, the evaluator shall perform Algorithm Functional Tests and Monte Carlo Tests using the following input parameters:

- *Key size [256] bits*
- *Direction [encryption, decryption]*

Algorithm Functional Tests *Algorithm Functional Tests are designed to verify the correct operation of the logical components of the algorithm implementation under normal operation using different block sizes. For AES-CBC, there are two types of AFTs:*
Known-Answer Tests *For each combination of direction and claimed key size, the TOE must be tested using the GFSBox, KeySbox, VarTxt, and VarKey test cases listed in Appendixes B through E of The Advanced Encryption Standard Algorithm Validation Suite (AESAVS), NIST, 15 November 2002.*
Multi-Block Message Tests *For each combination of direction and claimed key size, the TOE must*

be tested against 10 test cases consisting of a random IV, random key, and random plaintext/ciphertext. The plaintext/ciphertext starts with a length of 16 bytes and increases by 16 bytes for each test case until reaching 160 bytes.

Monte Carlo Tests *Monte Carlo tests are intended to test the implementation under strenuous conditions. The TOE must process the test cases according to the following algorithm once for each combination of direction and key size:*

```
Key[0] = Key
IV[0] = IV
PT[0] = PT
for i = 0 to 99 {
        Output Key[i], IV[i], PT[0]
        for j = 0 to 999 {
                if (j == 0) {
                        CT[j] = AES-CBC-Encrypt(Key[i], IV[i], PT[j])
                        PT[j+1] = IV[i]
                } else {
                        CT[j] = AES-CBC-Encrypt(Key[i], PT[j])
                        PT[j+1] = CT[j-1]
                }
        }
        Output CT[j]
        AES_KEY_SHUFFLE(Key, CT)
        IV[i+1] = CT[j]
        PT[0] = CT[j-1]
}
```

where

`AES_KEY_SHUFFLE`

is defined as:

```
If ( keylen = 128 )
        Key[i+1] = Key[i] xor MSB(CT[j], 128)
If ( keylen = 192 )
        Key[i+1] = Key[i] xor (LSB(CT[j-1], 64) || MSB(CT[j], 128))
If ( keylen = 256 )
        Key[i+1] = Key[i] xor (MSB(CT[j-1], 128) || MSB(CT[j], 128))
```

The above pseudocode is for encryption. For decryption, swap all instances of CT and PT. The initial IV, key, and plaintext/ciphertext should be random. The evaluator shall test the decrypt functionality using the same test as above, exchanging CT and PT, and replacing AES-CBC-Encrypt with AES-CBC-Decrypt.

XTS-AES

Identifier	***Cryptographic Algorithm***	***Cryptographic Key Sizes***	***List of Standards***
XTS-AES	*AES in XTS mode with unique tweak values that are consecutive non-negative integers starting at an arbitrary non-negative integer*	*512 bits*	*[**selection:** ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES]* *[**selection:** IEEE Std. 1619-2018, NIST SP 800-38E] [XTS]*

To test the TOE's ability to encrypt/decrypt data using AES in XTS mode, the evaluator shall perform the Single Data Unit Test and the Multiple Data Unit Test using the following input parameters:

- *Direction [encryption, decryption]*
- *Key size [512] bits*
- *Tweak value format [128-bit hex string, data unit sequence number]*

Single Data Unit Test *For each combination of claimed key size, direction, and supported tweak value format, the evaluator shall generate 50 test cases consisting of random payload data. The payload data size is determined randomly for each test case from supported values within the range [128-65536] bits. The payload size and data unit size must be equal.*
Multiple Data Unit Test *For each combination of claimed key size, direction, and supported tweak value format, the evaluator shall generate 50 test cases consisting of random payload data. The payload data size is determined randomly for each test case from supported values within the range [128-65536] bits. Likewise, the data unit size is determined randomly for each test case from supported values within the range [128-65535] bits. The payload size and data unit size must not be equal. The evaluator shall verify the correctness of the TSF's implementation by comparing values generated by the TSF with those generated by a known good implementation using the same input parameters.*
AES-CTR

Identifier	***Cryptographic Algorithm***	***Cryptographic Key Sizes***	***List of Standards***
AES-CTR	*AES in Counter Mode with a non-repeating initial counter and with no repeated use of counter values across multiple messages with the same secret key.*	*256 bits*	*[**selection:** ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197] [AES]* *[**selection:** ISO/IEC 10116:2017 (Clause 10), NIST SP 800-38A] [CTR]*

To test the TOE's ability to encrypt/decrypt data using AES in CTR mode, the evaluator shall perform the Algorithm Functional Test and the Counter Test using the following input parameters:

- *Direction [encryption, decryption]*
- *Key size [256] bits*

Algorithm Functional Tests *Algorithm Functional Tests are designed to verify the correct operation of the logical components of the algorithm implementation under normal operation using different block sizes. For AES-CTR, there are three types of AFTs:*
Known-Answer Tests *For each combination of direction and claimed key size, the TOE must be tested using the GFSBox, KeySbox, VarTxt, and VarKey test cases listed in Appendixes B through E of The Advanced Encryption Standard Algorithm Validation Suite (AESAVS), NIST, 15 November 2002.*
Single Block Message Tests *For each combination of direction and claimed key, the evaluator shall generate 10 test cases with a data size of 128 bits.*
Partial Block Message Tests *Monte Carlo tests are intended to test the implementation under strenuous conditions. The TOE must process the test cases according to the following algorithm once for each combination of direction and key size: For each combination of direction and claimed key, the evaluator shall generate five test cases such that the data size is not a multiple of 128 bits. The evaluator shall verify the correctness of the TSF's implementation by comparing values generated by the TSF with those generated by a known good implementation using the same input parameters.*
Counter Test *The evaluator shall generate a single message of 1000 blocks (128000 bits) and either encrypt or decrypt it. Back-compute the IVs used. Verify that they are unique and increasing (encryption) or decreasing (decryption).*

FCS_COP.1/XOF Cryptographic Operation - Extendable-Output Function

This is a selection-based component. Its inclusion depends upon selection from .

FCS_COP.1.1/XOF

The TSF shall perform *extendable-output function* in accordance with a specified cryptographic algorithm

[**selection**: *Cryptographic Algorithm*] and [**selection**: *Parameters*] that meet the following: [**selection**: *List of Standards*] . The following table provides the allowed choices for completion of the selection operations of FCS_COP.1.1/XOF.

Table 12: Allowed choices for FCS_COP.1.1/XOF

Cryptographic Algorithm	Parameters	List of Standards
SHAKE	Functions = [SHAKE128, SHAKE256]	NIST FIPS PUB 202 (Section 6.2)

Application Note: Since LMS and XMSS use both SHAKE128 and SHAKE256 internally, claiming and testing of both Functions is mandatory

FCS_IPSEC_EXT.1 IPsec Protocol

This is a selection-based component. Its inclusion depends upon selection from .

FCS_IPSEC_EXT.1.1

The TSF shall implement the IPsec architecture as specified in RFC 4301.

Application Note: Application Note 60

RFC 4301 calls for an IPsec implementation to protect IP traffic through the use of a Security Policy Database (SPD). The SPD is used to define how IP packets are to be handled: PROTECT the packet (e.g., encrypt the packet), BYPASS the IPsec services (e.g., no encryption), or DISCARD the packet (e.g., drop the packet). The SPD can be implemented in various ways, including router access control lists, firewall rulesets, a ‘traditional’ SPD, etc. Regardless of the implementation details, there is a notion of a ‘rule’ that a packet is ‘matched’ against and a resulting action that takes place.

While there must be a means to order the rules, a general approach to ordering is not mandated, as long as the SPD can distinguish the IP packets and apply the rules accordingly. There may be multiple SPDs (one for each network interface), but this is not required.

FCS_IPSEC_EXT.1.2

The TSF shall have a nominal, final entry in the SPD that matches anything that is otherwise unmatched and discards it.

FCS_IPSEC_EXT.1.3

The TSF shall implement [**selection**: *tunnel mode, transport mode*].

Application Note: Application Note 61

The ST author selects the supported modes of operation for IPsec.

FCS_IPSEC_EXT.1.4

The TSF shall implement the IPsec protocol ESP as defined by RFC 4303 using the cryptographic algorithms [**selection**: *AES-CBC-128 (RFC 3602), AES-CBC-192 (RFC 3602), AES-CBC-256 (RFC 3602), AES-GCM-128 (RFC 4106), AES-GCM-192 (RFC 4106), AES-GCM-256 (RFC 4106)*] together with a Secure Hash Algorithm (SHA)-based HMAC [**selection**: *HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512, no HMAC algorithm*].

Application Note: Application Note 62

When an AES-CBC algorithm is selected, at least one SHA-based HMAC must also be chosen. If only an AES-GCM algorithm is selected, then a SHA-based HMAC is not required since AES-GCM satisfies both confidentiality and integrity functions. IPsec may utilise a truncated version of the SHA-based HMAC functions contained in the selections. Where a truncated output is utilised, it should be highlighted in the TSS.

FCS_IPSEC_EXT.1.5

The TSF shall implement the protocol: [**selection**:

- *IKEv1, using Main Mode for Phase 1 exchanges, as defined in RFCs 2407, 2408, 2409, RFC 4109, [**selection**: no other RFCs for extended sequence numbers, RFC 4304 for extended sequence numbers],and [**selection**: no other RFCs for hash functions, RFC 4868 for hash functions]*
- *IKEv2 as defined in RFC 7296 [**selection**: with no support for NAT traversal, with mandatory support for NAT traversal as specified in RFC 7296 (Section 2.23)],and [**selection**: no other RFCs for hash functions, RFC 4868 for hash functions]*

].

Application Note: Application Note 63

If the TOE implements SHA-2 hash algorithms for IKEv1 or IKEv2, the ST author selects RFC 4868. If the TOE implements the use of truncated SHA-based HMACs as described in RFC 4868, they should be highlighted in the TSS.

FCS_IPSEC_EXT.1.6

The TSF shall ensure the encrypted payload in the [**selection**: *IKEv1, IKEv2*] protocol uses the cryptographic algorithms [**selection**: *AES-CBC-128, AES-CBC-192, AES-CBC-256 (specified in RFC 3602), AES-GCM-128, AES-GCM-192, AES-GCM-256 (specified in RFC 5282)*].

Application Note: Application Note 64

AES-GCM-128, AES-GCM-192 and AES-GCM-256 may only be selected if IKEv2 is also selected, as there is no RFC defining AES-GCM for IKEv1.

FCS_IPSEC_EXT.1.7

The TSF shall ensure that [**selection**:

- *IKEv1 Phase 1 SA lifetimes can be configured by a Security Administrator based on [**selection**: number of bytes, length of time, where the time values can be configured between [**assignment**: minimum configurable rekey time] and [**assignment**: maximum configurable rekey time]]*
- *IKEv2 SA lifetimes can be configured by a Security Administrator based on [**selection**: number of bytes, length of time, where the time values can be*

*configured between [**assignment**: minimum configurable rekey time] and [**assignment**: maximum configurable rekey time]]*

].

Application Note: Application Note 65

The ST author chooses either the IKEv1 requirements or IKEv2 requirements (or both, depending on the selection in FCS_IPSEC_EXT.1.5). The ST author chooses either volume-based lifetimes or time-based lifetimes (or a combination). The range between the minimum and maximum rekeys time must include a rekey time that causes a rekey to occur at or slightly before 24 hours. The exact values supported may vary by TOE implementation. Some TOEs might require the administrators to ensure rekeying prior to the desired time (e.g., configure a time value of 23h 59min to ensure the actual rekey is performed no later than 24h), while other TOEs might automatically ensure rekeying is performed prior to the configured time.

This requirement must be accomplished by providing Security Administrator-configurable lifetimes. Hardcoded limits do not meet this requirement.

FCS_IPSEC_EXT.1.8

The TSF shall ensure that [**selection**:

- *IKEv1 Phase 2 SA lifetimes can be configured by a Security Administrator based on [**selection**: number of bytes, length of time, where the time values can be configured between [**assignment**: minimum configurable rekey time] and [**assignment**: maximum configurable rekey time]]*
- *IKEv2 Child SA lifetimes can be configured by a Security Administrator based on [**selection**: number of bytes, length of time, where the time values can be configured between [**assignment**: minimum configurable rekey time] and [**assignment**: maximum configurable rekey time]]*

].

Application Note: Application Note 66

The ST author chooses either the IKEv1 requirements or IKEv2 requirements (or both, depending on the selection in FCS_IPSEC_EXT.1.5). The ST author chooses either volume-based lifetimes or time-based lifetimes (or a combination). The range between the minimum and maximum rekeys time must include a rekey time that causes a rekey to occur at or slightly before 8 hours. The exact values supported may vary by TOE implementation. Some TOEs might require the administrators to ensure rekeying prior to the desired time (e.g., configure a time value of 7h 59min to ensure the actual rekey is performed no later than 8h), while other TOEs might automatically ensure rekeying is performed prior to the configured time.

This requirement must be accomplished by providing Security Administrator-configurable lifetimes. Hardcoded limits do not meet this requirement.

FCS_IPSEC_EXT.1.9

The TSF shall generate the secret value x used in the IKE Diffie-Hellman key exchange (“x” in g^x mod p) using the random bit generator specified in FCS_RBG.1, and having a length of at least [**assignment**: *(one or more) number(s) of bits that is at least twice the security strength of the negotiated Diffie-Hellman group*] bits.

Application Note: Application Note 67

For DH groups 19 and 20, the ‘x’ value is the point multiplier for the generator point G.

Since the implementation may allow different Diffie-Hellman groups to be negotiated for use in forming the SAs, the assignment in FCS_IPSEC_EXT.1.9 may contain multiple values. For each DH group supported, the ST author consults Table 2 in NIST SP 800-57 “Recommendation for Key Management –Part 1: General” to determine the security strength (‘bits of security’) associated with the DH group. Each unique value is then used to fill in the assignment for this element. For example, suppose the implementation supports DH group 14 (2048-bit MODP) and group 20 (ECDH using NIST curve P-384). From Table 2, the bits of security value for group 14 is 112, and for group 20 is 192.

FCS_IPSEC_EXT.1.10

The TSF shall generate nonces used in [**selection**: *IKEv1, IKEv2*] protocol exchanges of length [**selection**: *according to the security strength associated with the negotiated Diffie-Hellman group, at least 128 bits in size and at least half the output size of the negotiated pseudorandom function (PRF) hash*].

Application Note: Application Note 68

This SFR is for the IKEv1 (phase 1 and phase 2) and IKEv2 (IKE_AUTH and CREATE_CHILD_SA) protocol exchanges.

The ST author must select the second option for nonce lengths if IKEv2 is selected (as this is mandated in RFC 7296). The ST author may select either option for IKEv1.

The security strengths of DH groups are defined in NIST SP 800-57.

Because nonces may be exchanged before the DH group is negotiated, the nonce used should be large enough to support all TOE-chosen proposals in the exchange.

FCS_IPSEC_EXT.1.11

The TSF shall ensure that IKE protocols implement DH Group(s) [**selection**:

- *[**selection**: 14 (2048-bit MODP), 15 (3072-bit MODP), 16 (4096-bit MODP), 17 (6144-bit MODP), 18 (8192-bit MODP)] according to RFC 3526*
- *[**selection**: 19 (256-bit Random ECP), 20 (384-bit Random ECP), 21 (521-bit Random ECP)] according to RFC 5114*

].

Application Note: Application Note 69

The selections are used to specify additional DH groups supported. This applies to IKEv1 and IKEv2 exchanges.

FCS_IPSEC_EXT.1.12

The TSF shall be able to ensure that the strength of the symmetric algorithm (in terms of the number of bits in the key) negotiated to protect the [**selection**: *IKEv1 Phase 1*, *IKEv2 IKE_SA*] connection is greater than or equal to the strength of the symmetric algorithm (in terms of the number of bits in the key) negotiated to protect the [**selection**: *IKEv1 Phase 2*, *IKEv2 CHILD_SA*] connection.

Application Note: Application Note 70

The ST author chooses either or both of the IKE selections based on what is implemented by the TOE. Obviously, the IKE version(s) chosen should be consistent not only in this element, but with other choices for other elements in this component. While it is acceptable for this capability to be configurable, the default configuration in the evaluated configuration (either ‘out of the box’ or by configuration guidance in the AGD documentation) must enable this functionality.

FCS_IPSEC_EXT.1.13

The TSF shall ensure that all IKE protocols perform peer authentication using [**selection**: *RSA, ECDSA*] that use X.509v3 certificates that conform to RFC 4945 and [**selection**: *Pre-shared Keys that conform to RFC 8784, no other method*].

Application Note: Application Note 71

At least one public-key-based Peer Authentication method is required in order to conform to this cPP; one or more of the public key schemes is chosen by the ST author to reflect what is implemented. The ST author also ensures that appropriate FCS requirements reflecting the algorithms used (and key generation capabilities, if provided) are listed to support those methods. Note: The TSS will elaborate on the way in which these algorithms are to be used (for example, RFC 2409 specifies three authentication methods using public keys; each one supported will be described in the TSS).

If the selection "Pre-shared Keys that conform to RFC 8784" is chosen, the selection-based requirement FIA_PSK_EXT.1 in Annex B must be claimed.

FCS_IPSEC_EXT.1.14

The TSF shall only establish a trusted channel if the presented identifier in the received certificate matches the configured reference identifier, where the presented and reference identifiers are of the following fields and types: [**selection**: *SAN: IP address, SAN: Fully Qualified Domain Name (FQDN), SAN: user FQDN, CN: IP address, CN: Fully Qualified Domain Name (FQDN), CN: user FQDN, Distinguished Name (DN)*] and [**selection**: *no other reference identifier type, [**assignment**: other supported reference identifier types]*].

Application Note: Application Note 72

When using RSA or ECDSA certificates for peer authentication, the reference and presented identifiers take the form of either a DN, IP address, FQDN or user FQDN. The reference identifier is the identifier the TOE expects to receive from the peer during IKE authentication. The presented identifier is the identifier that is contained within the peer certificate body. The ST author should select the presented and reference identifier types supported and may optionally assign additional supported identifier types in the second selection. Excluding the DN identifier type (which is necessarily the Subject DN in the peer certificate), the TOE may support the identifier in either the Common Name or Subject Alternative Name (SAN) or both.

The critical requirement of X.509 identifiers is the ability to bind the public key uniquely to an identity. This can be achieved by using strongly-typed identifiers or controlling the CA and certificate issuance. One recommended method for identity verification is supporting the use of the Subject Alternative Name (SAN) extension using DNS names, URI names, or Service Names. However, the support for a SAN extension is optional as long as identifier uniqueness can be achieved by other means.

Supported peer certificate algorithms are the same as FCS_IPSEC_EXT.1.13

Evaluation Activities

FCS_IPSEC_EXT.1
Something
TSS

ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FCS_NTP_EXT.1 NTP Protocol

This is a selection-based component. Its inclusion depends upon selection from FPT_STM_EXT.1.2.

FCS_NTP_EXT.1.1

The TSF shall use only the following NTP version(s) [**selection**: *NTP v3 (RFC 1305), NTP v4 (RFC 5905)*].

FCS_NTP_EXT.1.2

The TSF shall update its system time using [**selection**:

- *Authentication using [**selection**: SHA256, SHA384, SHA512, AES-CMAC-128 (RFC 8573), AES-CBC-128, AES-CBC-256] as the message digest algorithm(s);*
- *[**selection**: IPsec, DTLS as defined in the Functional Package for TLS] to provide trusted communication between itself and an NTP time source.*

].

FCS_NTP_EXT.1.3

The TSF shall not update NTP timestamp from broadcast and/or multicast addresses.

FCS_NTP_EXT.1.4

The TSF shall support configuration of at least three (3) NTP time sources in the Operational Environment.

Application Note: Application Note 73
The TOE has to support configuration of at least three time sources though it is not mandated that the TOE is configured to always use at least three time sources.

Evaluation Activities

FCS_NTP_EXT.1
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FCS_RBG.1 Random Bit Generation

FCS_RBG.1.1

The TSF shall perform deterministic random bit generation services using [**selection**: *DRBG Algorithm*] in accordance with [**selection**: *List of Standards*] after initialization.

The following table provides the allowable choices for completion of the selection operations of FCS_RBG.1.1.

Table 13: Allowable choices for FCS_RBG.1.1

Identifier	DRBG Algorithm	List of Standards
HASH_DRBG	Hash_DRBG with [**selection**: *SHA-256, SHA-384, SHA-512, SHA3-256, SHA3-384, SHA3-512*]	[**selection**: *ISO/IEC 18031: 2011 (Section C.2.2), NIST SP 800-90A Revision 1 Section 10.1.1*]
HMAC_DRBG	HMAC_DRBG with [**selection**: *SHA-256, SHA-384, SHA-512, SHA3-256, SHA3-384, SHA3-512*]	[**selection**: *ISO/IEC 18031: 2011 (Section C.2.3), NIST SP 800-90A Revision 1 Section 10.1.2*]
CTR_DRBG	CTR_DRBG with [**selection**: *AES-128, AES-192, AES-256*]	[**selection**: *ISO/IEC 18031: 2011 (Section C.3.2), NIST SP800-90A Revision 1 Section 10.2.1*]

FCS_RBG.1.2

The TSF shall use a [**selection**: *TSF entropy source [**assignment**: name of entropy source], multiple TSF entropy sources [**assignment**: name of entropy sources], TSF interface for seeding*] for initialized seeding.

Application Note: Application Note 20

For the selection in this requirement, the ST author selects "TSF entropy source" if a single entropy source is used as input to the DRBG. The ST author selects "multiple TSF entropy sources" if a seed is formed from a combination of two or more entropy sources within the TOE boundary. If the TSF implements two or more separate DRBGs that are seeded in separate manners, this SFR should be iterated for each DRBG. If multiple distinct entropy sources exist such that each DRBG only uses one of them, then each iteration would select "TSF entropy source"; "multiple TSF entropy sources" is only selected if a single DRBG uses multiple entropy sources for its seed. The ST author selects "TSF interface for seeding" if entropy source data is generated outside the TOE boundary.

If "TSF entropy source" is selected in FCS_RBG.1.2, FCS_RBG.3 must be claimed from Annex B.

If "multiple TSF entropy sources" is selected in FCS_RBG.1.2, FCS_RBG.4 and FCS_RBG.5 must be claimed from Annex B.

If "TSF interface for seeding" is selected in FCS_RBG.1.2, FCS_RBG.2 must be claimed from Annex B.

FCS_RBG.1.3

The TSF shall update the DRBG state by [**selection**: *reseeding, uninstantiating and re-instantiating*] using a [**selection**: *TSF entropy source [**assignment**: name of entropy source], multiple TSF entropy sources [**assignment**: name of entropy sources], TSF interface for obtaining entropy [**assignment**: name of the interface]*] in the following situations: [**selection**: *never, on demand, on the condition: [**assignment**: condition], after [**assignment**: time]*] in accordance with [**assignment**: *list of standards*].

Application Note: Application Note 21

If a reseeding is selected in the first selection of FCS_RBG.1.2 and something other than “never” is selected in the third selection of FCS_RBG.1.3, but reseeding is not feasible, the TSF will uninstantiate RBGs, rather than produce output that is of insufficient quality. The listed standards should specify the reseed interval and procedure for uninstantiating and reseeding. The remaining selection allows the PP Author to require application-specific conditions for reseeding.

“Uninstantiate” means that the internal state of the DRBG is no longer available for use. In the second selection of FCS_RBG.1.3, “on demand” means that a TOE presents an interface to reseed as a TSFI (e.g., an API call). The interface causes the DRBG to reseed at the request of an authorised user, either with an internal source, an external source, or from input provided through the TSFI (e.g., the API call).

The list of standards selected in the last assignment should be consistent with the standards selected in FCS_RBG.1.1

Evaluation Activities

FCS_RBG.1

164. Documentation shall be produced—and the evaluator shall perform the activities—in accordance with Annex D of [NDcPP].

TSS

There are no additional TSS evaluation activities for this component.

Guidance

There are no additional Guidance evaluation activities for this component.

Tests

There are no test activities for this component.

FCS_RBG.1.1

TSS

165. The evaluator shall examine the TSS to determine it identifies the DRBGs used by the TOE.

Guidance

168. If the DRBG functionality is configurable, the evaluator shall verify that the operational guidance includes instructions on how to configure this behaviour.

Tests

171. The evaluator shall perform 15 trials for the RNG implementation. If the RNG is configurable, the evaluator shall perform 15 trials for each configuration.

172. If the RNG has prediction resistance enabled, each trial consists of (1) instantiate DRBG, (2) generate the first block of random bits (3) generate a second block of random bits (4) uninstantiate. The evaluator shall verify that the second block of random bits is the expected value. The evaluator

shall generate eight input values for each trial. The first is a count (0 – 14). The next three are entropy input, nonce, and personalization string for the instantiate operation. The next two are additional input and entropy input for the first call to generate. The final two are additional input and entropy input for the second call to generate. These values are randomly generated. "generate one block of random bits" means to generate random bits with number of returned bits equal to the Output Block Length (as defined in NIST SP800-90A).

173. If the RNG does not have prediction resistance, each trial consists of (1) instantiate DRBG, (2) generate the first block of random bits (3) reseed, (4) generate a second block of random bits (5) uninstantiate. The evaluator shall verify that the second block of random bits is the expected value. The evaluator shall generate eight input values for each trial. The first is a count (0 – 14). The next three are entropy input, nonce, and personalization string for the instantiate operation. The fifth value is additional input to the first call to generate. The sixth and seventh are additional input and entropy input to the call to reseed. The final value is additional input to the second generate call.

174. The following paragraphs contain more information on some of the input values to be generated/selected by the evaluator.

- ***Entropy input:*** *the length of the entropy input value must equal the seed length.*
- ***Nonce:*** *If a nonce is supported (CTR_DRBG with no Derivation Function does not use a nonce), the nonce bit length is one-half the seed length.*
- ***Personalization string:*** *The length of the personalization string must be ≤ seed length. If the implementation only supports one personalization string length, then the same length can be used for both values. If more than one string length is support, the evaluator shall use personalization strings of two different lengths. If the implementation does not use a personalization string, no value needs to be supplied.*
- ***Additional input:*** *the additional input bit lengths have the same defaults and restrictions as the personalization string lengths.*

FCS_RBG.1.2

TSS

166. There are no additional TSS evaluation activities for this element.

Guidance

169. There are no additional Guidance evaluation activities for this element.

Tests

175. There are no test activities for this element.

FCS_RBG.1.3

TSS

167. The evaluator shall verify that the TSS identifies how the DRBG state is updated, and the situations under which this may occur.

Guidance

170. If the ST claims that the DRBG state can be updated on demand, the evaluator shall verify that the operational guidance has instructions for how to perform this operation.

Tests

176. There are no test activities for this element.

FCS_RBG.2 Random Bit Generation (External Seeding)

This is a selection-based component. Its inclusion depends upon selection from FCS_RBG.1.2.

FCS_RBG.2.1

The TSF shall be able to accept a minimum input of [**assignment**: *minimum input length greater than zero*] from a TSF interface for obtaining entropy.

Application Note: Application Note 55

This requirement is claimed when a TOE uses one or more external sources of entropy to initialize or reseed a DRBG that is outside the TOE boundary. Seeding a DRBG is the same as initializing a DRBG. In the case of a network device this would only occur with a vND where the entropy source is provided by the underlying virtualization System (VS) platform. The ST author ensures that the assignment is completed with the minimum length of the input sufficient to initialize or reseed a DRBG.

The TSF interface for the purpose of seeding here is the interface used to gather entropy for initializing or reseed.

FCS_RBG.3 Random Bit Generation (Internal Seeding - Single Source)

This is a selection-based component. Its inclusion depends upon selection from .

FCS_RBG.3.1

The TSF shall be able to seed the DRBG using a [**selection, choose one of**: *TSF software-based entropy source*, *TSF hardware-based entropy source*] [**assignment**: *name of entropy source*] with [**assignment**: *number of bits*] bits of min-entropy.

Application Note: Application Note 56

This requirement is claimed when a TOE uses a single internal source of entropy to initialize or reseed a DRBG that is within the TOE boundary. Seeding a DRBG is the same as initializing a DRBG.

Hardware-based noise sources are entropy sources whose primary function is noise generation, such as ring oscillators, diodes, and thermal noise. While a TOE may use software to collect the noise from these hardware sources, these are not software-based.

Software-based noise sources generate noise as a byproduct of their normal operation. Examples of software-based noise sources can be user or system-based events such as reading the least significant bits from an event timer, etc.

The TOE collects enough input from the internal noise source such that the total measured entropy of the input is sufficient to initialize or reseed a DRBG. The ST author ensures that the assignment is completed with the number of bits of the input sufficient to initialize or reseed a DRBG.

FCS_RBG.4 Random Bit Generation (Internal Seeding - Multiple Sources)

This is a selection-based component. Its inclusion depends upon selection from .

FCS_RBG.4.1

The TSF shall be able to seed the DRBG using [**selection**: *[**assignment**: number] TSF software-based entropy source(s), [**assignment**: number] TSF hardware-based entropy source(s)*].

Application Note: Application Note 57

This requirement is claimed when a TOE uses two or more internal sources of entropy to initialize or reseed a DRBG. Seeding a DRBG is the same as initializing a DRBG. FCS_RBG.5 defines the mechanism by which these sources are combined to ensure sufficient minimum entropy.

FCS_RBG.5 Random Bit Generation (Combining Entropy Sources)

This is a selection-based component. Its inclusion depends upon selection from .

FCS_RBG.5.1

TSF shall [**selection**: *hash, concatenate and hash, XOR, input into a linear feedback shift register, [**assignment**: combining operation]*] [**selection**: *output from TSF entropy source(s), input from TSF interface(s) for obtaining entropy*] resulting in a minimum of [**assignment**: *number of bits*] bits of min-entropy to create the entropy input into the derivation function as defined in [**selection**: *ISO/IEC 18031:2011*, *NIST SP 800-90A Revision 1*].

Application Note: Application Note 58

This requirement is claimed when a TOE combines two or more sources of entropy to initialize or reseed a DRBG. Seeding a DRBG is the same as initializing a DRBG. The ST author ensures that the assignment is completed with the number of bits of the combined entropy sufficient to initialize or reseed a DRBG.

One can apply NIST SP 800-90B (or AIS-31) statistical tests against internal noise sources (a.k.a. raw entropy) to confirm the min-entropy of the noise sources either in aggregate or individually. One should not apply NIST SP 800-90B (or AIS-31) statistical tests against external noise sources since the TOE is unable to enforce entropy requirements or conditioning requirements against external sources of entropy. However, the TSS may include estimates for min-entropy from external sources that contribute to the overall entropy requirements for the DRBG.

5.1.5 Identification and Authentication (FIA)

FIA_AFL.1 Authentication Failure Management (Refinement)

This is an implementation-based component. Its inclusion in depends on whether the TOE implements one or more of the following features:

- ***Password-based remote administration***

as described in Appendix A.3: Implementation-based Requirements.

FIA_AFL.1.1

The TSF shall detect when an Administrator configurable positive integer within

[**assignment**: *range of acceptable values*]

unsuccessful authentication attempts occur related to *Administrators attempting to authenticate remotely using a password.*

FIA_AFL.1.2

When the defined number of unsuccessful authentication attempts has been met, the TSF shall

[**selection**: *prevent the offending Administrator from successfully establishing a remote session using any authentication method that involves a password until [**assignment**: action to unlock] is taken by an Administrator, prevent the offending Administrator from successfully establishing a remote session using any authentication method that involves a password until an Administrator defined time period has elapsed*].

Application Note: Application Note 74

This requirement applies to a defined number of successive unsuccessful remote password-based authentication attempts and does not apply to local Administrative access, since it does not make sense to lock a local Administrator's account in this fashion. Compliant TOEs may optionally include cryptographic authentication failures and/or local authentication failures in the number of unsuccessful authentication attempts. This could be addressed by (for example) requiring a separate account for local Administrators or having the authentication mechanism implementation distinguish local and remote login attempts. The 'action' taken by a local Administrator is implementation specific and would be defined in the Administrator guidance (for example, lockout reset, or password reset). The ST author chooses one or both of the selections for handling of authentication failures depending on how the TOE has implemented this handler.

The TSS describes how the TOE ensures that authentication failures by remote Administrators cannot lead to a situation where no Administrator access is available, either permanently or temporarily (e.g., by providing local logon, which is not subject to blocking, or by allowing a reboot to clear the lockout status and restore administrator access). The Operational Guidance describes, and identifies the importance of, any actions that are required in order to ensure that Administrator access will always be maintained, even if remote administration is made permanently or temporarily unavailable due to blocking of accounts as a result of FIA_AFL.1.

Evaluation Activities

FIA_AFL.1
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FIA_PMG_EXT.1 Password Management

This is an implementation-based component. Its inclusion in depends on whether the TOE implements one or more of the following features:

- ***Password-based authentication***

as described in Appendix A.3: Implementation-based Requirements.

FIA_PMG_EXT.1.1

The TSF shall provide the following password management capabilities for administrative passwords:

1. Passwords shall be able to be composed of any combination of upper and lower case letters, numbers and the following special characters:

[**selection**: *"!", "@", "#", "$", "%", "^", "&", "*", "(", ")", [**assignment**: other characters]*]

1. Minimum password length shall be *configurable to between*

[**assignment**: *minimum number of characters supported by the TOE*] and [**assignment**: *number of characters greater than or equal to 15*] *characters.*

Application Note: Application Note 76

The ST author selects the special characters that are supported by the TOE. They may optionally list additional special characters supported using the assignment. "Administrative passwords" refers to passwords used by Administrators at the local console, over protocols that support passwords, such as SSH and HTTPS, or to grant configuration data that supports other SFRs in the Security Target.

The second assignment should be configured with the largest minimum password length the Security Administrator can configure.

Evaluation Activities

FIA_PMG_EXT.1
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FIA_PSK_EXT.1 Pre-Shared Key Composition

This is an implementation-based component. Its inclusion in depends on whether the TOE implements one or more of the following features:

- ***Pre-shared keys for IPsec***

as described in Appendix A.3: Implementation-based Requirements.

FIA_PSK_EXT.1.1

The TSF shall be able to use pre-shared keys that conform to RFC 8784 for IPsec.

FIA_PSK_EXT.1.2

The TSF shall be able to [**selection**: *accept externally generated pre-shared keys*, *generate 256 bit-based pre-shared keys via FCS_RBG.1*].

Application Note: Application Note 77
Generated PSKs are expected to be shared between components via an out-of-band mechanism.

FIA_UAU.7 Protected Authentication Feedback

This is an implementation-based component. Its inclusion in depends on whether the TOE implements one or more of the following features:

- ***Password-based local authentication***

as described in Appendix A.3: Implementation-based Requirements.

FIA_UAU.7.1

The TSF shall provide only obscured feedback to the **administrative** user while the authentication is in progress **at the local console**.

Application Note: Application Note 75
'Obscured feedback' implies the TSF does not produce a visible display of any authentication data entered by an administrator (such as the echoing of a password), although an obscured indication of progress may be provided (such as an asterisk for each character). It also implies that the TSF does not return any information during the authentication process to the administrator that may provide any indication of the authentication data.

Evaluation Activities

FIA_UAU.7
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FIA_UIA_EXT.1 User Identification and Authentication

FIA_UIA_EXT.1.1

The TSF shall allow the following actions prior to requiring the non-TOE entity to initiate the identification and authentication process:

- Display the warning banner in accordance with FTA_TAB.1;
- [**selection**: *no other actions, automated generation of cryptographic keys, [**assignment**: list of services, actions performed by the TSF in response to non-TOE requests]*].

FIA_UIA_EXT.1.2

The TSF shall require each administrative user to be successfully identified and authenticated before allowing any other TSF-mediated actions on behalf of that administrative user.

Application Note: Application Note 22

This requirement applies to Administrators and external IT entities of services available from the TOE directly and not services available by connecting through the TOE. While it should be the case that few or no services are available to external entities prior to identification and authentication, if there are some available (perhaps ICMP echo) these should be listed in the assignment statement; if automated generation of cryptographic keys is supported without administrator authentication, the option "automated generation of cryptographic keys" should be selected; otherwise, the option “no other actions” should be selected.

FIA_UIA_EXT.1.3

The TSF shall provide the following remote authentication mechanisms [**selection**: *Web GUI password, SSH password, SSH public key, X.509 certificate*] and [**selection**: *no other mechanism, external authentication server*].The TSF shall provide the following local authentication mechanisms: [**selection**: *none, password-based, [**assignment**: other authentication mechanism]*].

Application Note: Application Note 23

An authentication process consists of two basic steps: identification step (presenting the claimed attribute value (e.g., a user identifier) to the authentication subsystem); verification step (presenting or generating authentication information (e.g., a value signed with a private key) that acts as evidence to prove the binding between the attribute and that for which it is claimed).

Remote authentication is when a user associated with the Security Administrator role remotely communicates with the TOE for the purpose of security management over a cryptographic protocol specified in FTP_TRP.1/Admin. Local authentication mechanisms are defined as those that occur at a local administrative interface using a console. If no local authentication mechanism is supported by the TOE, the ST author should select "none" from the final selection. See Application Note 26 for examples of compliant local administrative interfaces.

Local administration is defined as administration using a dedicated physical interface that (from the TOE’s point of view) is directly connected to the device(s) the administrator interacts with and therefore falls under the physical protection (OE.PHYSICAL). Any administrator choice to extend a local console so it is remotely accessible (e.g., console server or remote KVM) is outside the scope of the NDcPP. The following are examples of compliant local administrative interfaces:

a. RS-232 terminal.

b. Peripherals (e.g., keyboard, monitor, mouse).

The TOE must support at least one authentication mechanism where the verification step is processed locally, as such “external authentication server” should not be the only available authentication method.

The ST author selects the authentication mechanisms necessary to support remote administration. If "Web GUI password" or "SSH password" is selected for remote authentication mechanism the ST author specifies an appropriate cryptographic protocol in FTP_TRP.1/Admin (e.g., "HTTPS" or "SSH") and includes FIA_AFL.1, FIA_PMG_EXT.1, FPT_APW_EXT.1 from Annex B.

If integration with an external X.500 Directory is supported and enabled, the "external authentication server" must be selected and an appropriate cryptographic protocol with each "authentication server" must be selected in FTP_ITC.1. Since the identity verification step is performed remotely, FIA_AFL.1, FIA_PMG_EXT.1, FPT_APW_EXT.1 requirements are not enforced by the TOE and therefore are not applicable to the “external authentication server” selection.

FIA_UIA_EXT.1.4

The TSF shall authenticate any administrative user’s claimed identity according to each authentication mechanism specified in FIA_UIA_EXT.1.3.

Application Note: Application Note 24

According to the application note for FMT_SMR.2, for distributed TOEs at least one TOE component has to support the authentication of Security Administrators according to FIA_UIA_EXT.1.3 and FIA_UIA_EXT.1.4 but not necessarily all TOE components. In case not all TOE components support this way of authentication for Security Administrators the TSS must describe how Security Administrators are authenticated and identified.

Evaluation Activities

FIA_UIA_EXT.1
Something
TSS
177. The evaluator shall examine the TSS to determine that it describes the logon process for remote authentication mechanism (e.g., SSH public key, Web GUI password, etc.) and optional local authentication mechanisms supported by the TOE. This description shall contain information pertaining to the credentials allowed/used, any protocol transactions that take place, and what constitutes a “successful logon”.

178. The evaluator shall examine the TSS to determine that it describes which actions are allowed before administrator identification and authentication. The description shall cover authentication and identification for local and remote TOE administration.

179. For distributed TOEs, the evaluator shall examine that the TSS details how Security Administrators are authenticated and identified by all TOE components. If not all TOE components support authentication of Security Administrators according to FIA_UIA_EXT.1, the TSS shall describe how the overall TOE functionality is split between TOE components including how it is ensured that no unauthorised access to any TOE component can occur.

180. For distributed TOEs, the evaluator shall examine the TSS to determine that it describes, for each TOE component, which actions are allowed before administrator identification and authentication. The description shall cover authentication and identification for remote TOE administration and optionally for local TOE administration if claimed by the ST author. For each TOE component that does not support authentication of Security Administrators according to

FIA_UIA_EXT.1, the TSS shall describe any unauthenticated services/services that are supported by the component.

Guidance

181. The evaluator shall examine the guidance documentation to determine that any necessary preparatory steps (e.g., establishing credential material such as pre- shared keys, tunnels, certificates, etc.) to logging in are described. For each supported login method, the evaluator shall ensure the guidance documentation provides clear instructions for successfully logging on. If configuration is necessary to ensure the services provided before login are limited, the evaluator shall determine that the guidance documentation provides sufficient instruction on limiting the allowed services.

Tests

182. The evaluator shall perform the following tests for each method by which administrators access the TOE (local and remote), as well as for each type of credential supported by the login method:

- *Test FIA_UIA_EXT.1:1:*
  Test 1: The evaluator shall use the guidance documentation to configure the appropriate credential supported for the login method. For all combinations of supported credentials and login methods, the evaluator shall show that providing correct I&A information results in the ability to access the system, while providing incorrect information results in denial of access.
- *Test FIA_UIA_EXT.1:2:*
  Test 2: The evaluator shall configure the services allowed (if any) according to the guidance documentation, and then determine the services available to an external remote entity. The evaluator shall determine that the list of services available is limited to those specified in the requirement.
- *Test FIA_UIA_EXT.1:3:*
  Test 3: For local access, the evaluator shall determine what services are available to a local administrator prior to logging in, and make sure this list is consistent with the requirement.
- *Test FIA_UIA_EXT.1:4:*
  Test 4: For distributed TOEs, where not all TOE components support the authentication of Security Administrators according to FIA_UIA_EXT.1, the evaluator shall test that the components authenticate Security Administrators as described in the TSS.

5.1.6 Security Management (FMT)

General Requirements for Distributed TOEs

For distributed TOEs, the evaluation activities defined in this chapter shall be performed.

TSS

183. For distributed TOEs, the evaluator shall verify that the TSS describes how every function related to security management is realized for every TOE component and shared between different TOE components. The evaluator shall confirm that all relevant aspects of each TOE component are covered by the FMT SFRs.

Guidance Documentation

184. For distributed TOEs, the evaluator shall verify that the Guidance Documentation describes management of each TOE component. The evaluator shall confirm that all relevant aspects of each TOE component are covered by the FMT SFRs.

Tests

185. Tests defined to verify the correct implementation of security management functions shall be performed for every TOE component. For security management functions that are implemented centrally, sampling should be applied when defining the evaluator's tests (ensuring that all components are covered by the sample).

FMT_MOF.1/AutoUpdate Management of Security Functions Behaviour

This is a selection-based component. Its inclusion depends upon selection from .

FMT_MOF.1.1/AutoUpdate

The TSF shall restrict the ability to [**selection**: *enable*, *disable*] the functions [**selection**: *automatic checking for updates*, *automatic update*]

to *Security Administrators*.

Application Note: Application Note 81

FMT_MOF.1/AutoUpdate is only applicable and should be included if the TOE supports automatic checking for updates and/or automatic updates and allows them to be enabled and disabled. Enable and disable of automatic checking for updates and/or automatic updates is restricted to Security Administrators. The option "automatic update" may only be selected if digital signatures are used to validate the trusted update.

Evaluation Activities

FMT_MOF.1/AutoUpdate
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FMT_MOF.1/Functions Management of Security Functions Behaviour

This is a selection-based component. Its inclusion depends upon selection from .

FMT_MOF.1.1/Functions

The TSF shall restrict the ability to [**selection**: *determine the behaviour of*, *modify the behaviour of*] the functions [**selection**: *transmission of audit data to an external IT entity*, *handling of audit data*, *audit functionality when Local Audit Storage Space is full*]

to *Security Administrators*.

Application Note: Application Note 82
FMT_MOF.1/Functions should be chosen if one or more of the following scenarios apply:

- If the transmission protocol for transmission of audit data to an external IT entity as defined in FAU_STG_EXT.1.1 is configurable, “transmission of audit data to an external IT entity” should be chosen.
- If the handling of audit data is configurable, “handling of audit data” must be chosen. The term “handling of audit data” refers to any administratively configurable selection or assignments in any FAU_STG_EXT.x SFR.
- If the behaviour of the audit functionality is configurable when Local Audit Storage Space is full, “audit functionality when Local Audit Storage Space is full” must be chosen.

The first selection for ‘determine the behaviour of’ and ‘modify the behaviour of’ should be done as appropriate. It might be necessary to have different selections for the first selection depending on the second selection (e.g., “handling of audit data” might require “determine the behaviour of” and “modify the behaviour of” for the first selection on the one hand and “audit functionality when Local Audit Storage Space is full” might require “modify the behaviour of” only). In that case FMT_MOF.1/Functions should be iterated with increasing number appended (i.e., FMT_MOF.1/Functions1, FMT_MOF.1/Functions2, etc.).

Evaluation Activities

FMT_MOF.1/Functions
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FMT_MOF.1/ManualUpdate Management of Security Functions Behaviour

FMT_MOF.1.1/ManualUpdate

The TSF shall restrict the ability to enable the functions *to perform manual updates* to *Security Administrators*.

Application Note: Application Note 25
FMT_MOF.1/ManualUpdate restricts the initiation of manual updates to Security Administrators.

Evaluation Activities

FMT_MOF.1/ManualUpdate

TSS

186. For distributed TOEs, see FMT introduction. There are no specific requirements for non-distributed TOEs.

Guidance

187. The evaluator shall examine the guidance documentation to determine that any necessary steps to perform manual update are described. The guidance documentation shall also provide warnings regarding functions that may cease to operate during the update (if applicable).

188. For distributed TOEs, the guidance documentation shall describe all steps for how to update all TOE components. This shall contain a description of the order in which components need to be updated if the order is relevant to the update process. The guidance documentation shall also provide warnings regarding functions of TOE components and the overall TOE that may cease to operate during the update (if applicable).

Tests

189. The evaluator shall perform the following tests:

- *Test FMT_MOF.1/ManualUpdate:1:*
  Test 1: The evaluator shall try to perform the update using a legitimate update image without prior authentication as Security Administrator (either by authentication as a user with no administrator privileges or without user authentication at all – depending on the configuration of the TOE). The attempt to update the TOE shall fail.
- *Test FMT_MOF.1/ManualUpdate:2:*
  Test 2: The evaluator shall try to perform the update with prior authentication as Security Administrator using a legitimate update image. This attempt should be successful. This test case is covered by Test 1 for FPT_TUD_EXT.1.

FMT_MOF.1/Services Management of Security Functions Behaviour

This is a selection-based component. Its inclusion depends upon selection from .

FMT_MOF.1.1/Services

The TSF shall restrict the ability to **start and stop** ~~the functions~~ **services** to *Security Administrators*.

Application Note: Application Note 80

FMT_MOF.1/Services should only be chosen if the Security Administrator has the ability to start and stop services and the corresponding option has been selected in FMT_SMF.1.

In FMT_MOF.1.1/Services 'enable and disable' have been refined to 'start and stop' and 'the functions: [assignment: list of functions]' has been refined to 'services'.

With respect to FAU_GEN.1.1, FMT_SMF.1 and FMT_MOF.1/Services the term ‘services’ refers to trusted path and trusted channel communications, on demand self-tests, trusted update and Administrator sessions (that exist under the trusted path) (e.g., netconf).

Evaluation Activities

FMT_MOF.1/Services
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FMT_MTD.1/CoreData Management of TSF Data

FMT_MTD.1.1/CoreData

The TSF shall restrict the ability to manage the *TSF data* to *Security Administrators*.

Application Note: Application Note 26
The word 'manage' includes but is not limited to create, initialize, view, change default, modify, delete, clear, and append. This SFR includes also the resetting of administrative passwords by the Security Administrator. The identifier 'CoreData' has been added here to separate this iteration of FMT_MTD.1 from the optional iteration of FMT_MTD.1 defined in Annex A.4.2.1 (FMT_MTD.1/CryptoKeys).

Evaluation Activities

FMT_MTD.1/CoreData

TSS
190. For each administrative function identified in the guidance documentation that is accessible through an interface prior to administrator log-in, the evaluator shall confirm that the TSS details how the ability to manipulate the TSF data through these interfaces is disallowed for non-administrative users.

191. If the TOE supports handling of X.509v3 certificates and implements a trust store, the evaluator shall examine the TSS to determine that it contains sufficient information to describe how the ability to manage the TOE's trust store is restricted.

Guidance
192. The evaluator shall review the guidance documentation to determine that each of the TSF-data-manipulating functions implemented in response to the requirements of the cPP is identified, and that configuration information is provided to ensure that only administrators have access to the functions.

193. If the TOE supports handling of X.509v3 certificates and provides a trust store, the evaluator shall review the guidance documentation to determine that it provides sufficient information for the administrator to configure and maintain the trust store in a secure way. If the TOE supports loading of CA certificates, the evaluator shall review the guidance documentation to determine that it provides sufficient information for the administrator to securely load CA certificates into the trust

store. The evaluator shall also review the guidance documentation to determine that it explains how to designate a CA certificate a trust anchor.

Tests
194. No separate testing for FMT_MTD.1/CoreData is required unless one of the management functions has not already been exercised under any other SFR.

FMT_MTD.1/CryptoKeys Management of TSF Data

This is an implementation-based component. Its inclusion in depends on whether the TOE implements one or more of the following features:

- ***Admin management of crypto keys***

as described in Appendix A.3: Implementation-based Requirements.

FMT_MTD.1.1/CryptoKeys

The TSF shall restrict the ability to *manage* the *cryptographic keys* to *Security Administrators*.

Application Note: Application Note 83
FMT_MTD.1.1/CryptoKeys restricts management of cryptographic keys to Security Administrators. It should be included if cryptographic keys can be managed (e.g., modified, deleted or generated/imported) by the Security Administrator. The identifier 'CryptoKeys' has been added here to separate this iteration of FMT_MTD.1 from the mandatory iteration of FMT_MTD.1 defined in Section 6.6.2.1 (FMT_MTD.1/CoreData).

Evaluation Activities

FMT_MTD.1/CryptoKeys
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FMT_SMF.1 Specification of Management Functions

FMT_SMF.1.1

The TSF shall be capable of performing the following management functions:

- Ability to administer the TOE remotely;
- Ability to configure the access banner;

- Ability to configure the remote session inactivity time before session termination;
- Ability to update the TOE, and to verify the updates using digital signature capability prior to installing those updates;

[**selection**: *Ability to start and stop services, Ability to configure local audit behaviour (e.g. changes to storage locations for audit; changes to behaviour when local audit storage space is full; changes to local audit storage size), Ability to modify the behaviour of the transmission of audit data to an external IT entity, Ability to configure the list of TOE-provided services available before an entity is identified and authenticated, as specified in FIA_UIA_EXT.1, Ability to manage the cryptographic keys, Ability to configure the cryptographic functionality, Ability to configure thresholds for SSH rekeying, Ability to configure the lifetime for IPsec SAs, Ability to configure the list of supported (D)TLS ciphers, Ability to configure the interaction between TOE components, Ability to enable or disable automatic checking for updates or automatic updates, Ability to re-enable an Administrator account, Ability to set the time which is used for time-stamps, Ability to configure NTP, Ability to configure the reference identifier for the peer, Ability to manage the TOE's trust store and designate X509.v3 certificates as trust anchors, Ability to generate Certificate Signing Request (CSR) and process CA certificate response, Ability to administer the TOE locally, Ability to configure the local session inactivity time before session termination or locking, Ability to configure the authentication failure parameters for FIA_AFL.1, Ability to manage the trusted public keys database, Ability to manage the public key or certificate used to validate the digital update, No other capabilities*] Status Markers:
- undefined

#	Management Function	Status

Application Note: Application Note 27

FMT_SMF.1.1 Management Function	Management Function Guidance
Ability to administer the TOE remotely	The TOE must provide functionality for remote administration. Local administration is optional. This cPP does not mandate a specific security management function to be available either through the local administration interface, the remote administration interface or both. Remote administrative sessions are specified in FTP_TRP.1/Admin.
Ability to configure the access banner	The TOE must provide functionality to configure the access banner for FTA_TAB.1 and the session inactivity time(s) for FTA_SSL.3 and (if included) FTA_SSL_EXT.1, though an access banner is only required for each interactive (human-computer) interface (HCI), not for any programmatic interface [application programming interface (API), e.g., REST API].

Ability to configure the remote session inactivity time before session termination	The TOE must provide functionality to configure the access banner for FTA_TAB.1 and the session inactivity time(s) for FTA_SSL.3 and (if included) FTA_SSL_EXT.1, though an access banner is only required for each interactive (human-computer) interface (HCI), not for any programmatic interface [application programming interface (API), e.g., REST API].
Ability to update the TOE, and to verify the updates using digital signature capability prior to installing those updates	The option “Ability to update the TOE, and to verify the updates using digital signature capability prior to installing those updates” includes the relevant management functions from FMT_MOF.1/ManualUpdate and FPT_TUD_EXT.1. Based on selections in FPT_TUD_EXT.1.2, FMT_MOF.1/AutoUpdate must be included if the option “Ability to enable or disable automatic checking for updates or automatic updates” is included in the ST.
Ability to start and stop services	The selection "Ability to start and stop services" should be included in the ST if the TOE supports starting and stopping services of the TOE. If this selection is included in the ST, FMT_MOF.1/Services must be claimed in the ST.
Ability to configure local audit behaviour (e.g., changes to storage locations for audit; changes to behaviour when local audit storage space is full; changes to local audit storage size)	The selection “Ability to configure local audit behaviour” includes the relevant management functions from FMT_MOF.1/Services and FMT_MOF.1/Functions, (for all of these SFRs that are included in the ST) and is intended to cover security relevant configuration options (if any) to the audit behaviour (like changes to the behaviour when the local audit storage space is full). The option "Ability to modify the behaviour of the transmission of audit data to an external IT entity" is intended to cover the management functionalities related to the transmission of local audit information to an external IT entity.
Ability to modify the behaviour of the transmission of audit data to an external IT entity	The option "Ability to modify the behaviour of the transmission of audit data to an external IT entity" is intended to cover the management functionalities related to the transmission of local audit information to an external IT entity.
Ability to configure the list of TOE-provided services available before an entity is identified and authenticated, as specified in FIA_UIA_EXT.1	The selection "Ability to configure the list of TOEprovided services available before an entity is identified and authenticated, as specified in FIA_UIA_EXT.1" should be included in the ST if the TOE supports configuration of the list of TOE-provided services which are available before any entity is identified and authenticated. The term 'list' refers to the resulting list of available services as a result of the configuration activities. The configuration activity itself does not necessarily have to be modification of a list

	but could be any type of activation and deactivation procedure.
Ability to manage the cryptographic keys	The selection "Ability to manage the cryptographic keys" should be included in the ST if the TOE supports management of cryptographic keys (e.g., generation of cryptographic keys). If this selection is included in the ST, FMT_MTD.1/CryptoKeys must be claimed in the ST.
Ability to configure the cryptographic functionality	For distributed TOEs, that implement a registration channel (as described in FCO_CPC_EXT.1.2), the ST author uses the selection “Ability to configure the cryptographic functionality” in this SFR, and its corresponding mapping in the TSS, to describe the configuration of any cryptographic aspects of the registration channel that can be modified by the operational environment in order to improve the channel security (reference the description of the content of Preparative Procedures in [SD, 3.4.1]).
Ability to configure thresholds for SSH rekeying	The selection "Ability to configure thresholds for SSH rekeying" may only be selected if SSH is selected within FTP_ITC.1, FTP_TRP.1 or FPT_ITT.1. This only applies if the TOE claims conformance to the Functional Package for SSH and the rekey threshold is configurable.
Ability to configure the lifetime for IPsec SAs	The selection “Ability to configure lifetime for IPsec SAs” must be included in the ST if the TOE supports secure communication via IPsec and the FCS_IPSEC_EXT.1 requirements are included in the ST. The configuration of the lifetime for IPsec SAs needs to be in line with the selection in FCS_IPSEC_EXT.1.7.
Ability to configure the list of supported (D)TLS ciphers	The selection "Ability to configure the list of supported (D)TLS ciphers" must be included in the ST if the TOE implements TLS or DTLS and the supported ciphersuites are configurable. This only applies if the TOE claims conformance to the Functional Package for TLS, and only if such a configuration option exists.
Ability to configure the interaction between TOE components	For distributed TOEs, the interaction between TOE components will be configurable (see FCO_CPC_EXT.1). Therefore, the ST author includes the selection "Ability to configure the interaction between TOE components" for distributed TOEs. A simple example would be the change of communication protocol according to FPT_ITT.1. Another example would be changing the management of a TOE component from direct remote administration to remote administration through another TOE component. A more complex use case would be if the realization of an SFR is achieved through two or more TOE components

	and the responsibilities between the two or more components could be modified.
Ability to enable or disable automatic checking for updates or automatic updates	Based on selections in FPT_TUD_EXT.1.2, FMT_MOF.1/AutoUpdate must be included if the option “Ability to enable or disable automatic checking for updates or automatic updates” is included in the ST.
Ability to re-enable an Administrator account	If the TOE offers the ability for a remote Administrator account to be disabled in line with FIA_AFL.1, then the ST author must select the option “Ability to re-enable an Administrator account” to allow the account to be re-enabled by a local Administrator.
Ability to set the time which is used for time-stamps	The selection “Ability to set the time which is used for time-stamps” should be included in the ST if the TOE allows the Administrator to set the time of the device which is then used in time stamps. This option should not be selected if the TOE does not allow manual time setting but only relies on synchronization with external time sources like NTP servers.
Ability to configure NTP	The selection “Ability to configure NTP” should be included in the ST if the TOE uses NTP for timestamp configuration. If selected, FCS_NTP_EXT.1 must be included in the ST as well.
Ability to configure the reference identifier for the peer	The selection “Ability to configure the reference identifier for the peer” should be included in the ST if the TOE allows the Administrator to specify the expected identity of a remote peer when establishing secure communications using a protocol included in the ST. For TOEs that support only IP address and FQDN identifier types, configuration of the reference identifier may be the same as configuration of the peer’s name for the purposes of connection.
Ability to manage the TOE’s trust store and designate X509.v3 certificates as trust anchors	The selection “Ability to manage the TOE’s trust store and designate X509.v3 certificates as trust anchors" should be included in the ST if the TOE supports management and configuration of the TOE’s trust store. This means the TOE supports X.509v3 certificates for some security functions. This only applies if the TOE claims conformance to the Functional Package for X.509.
Ability to generate Certificate Signing Request (CSR) and process CA certificate response	The selection "Ability to generate Certificate Signing Request (CSR) and process CA certificate response" must be included in the ST if the TOE implements Certificate Request or Enrollment Request processes. This only applies if the TOE claims conformance to the Functional Package for X.509, and only when FIA_X509_EXT.3 from the functional package is claimed.

Ability to administer the TOE locally	The TOE must provide functionality for remote administration. Local administration is optional. This cPP does not mandate a specific security management function to be available either through the local administration interface, the remote administration interface or both. Remote administrative sessions are specified in FTP_TRP.1/Admin.
Ability to configure the local session inactivity time before session termination or locking	The TOE must provide functionality to configure the access banner for FTA_TAB.1 and the session inactivity time(s) for FTA_SSL.3 and (if included) FTA_SSL_EXT.1, though an access banner is only required for each interactive (human-computer) interface (HCI), not for any programmatic interface [application programming interface (API), e.g., REST API].
Ability to configure the authentication failure parameters for FIA_AFL.1	This management function enables administrators to configure parameters related to authentication failure, such as the threshold for unsuccessful login attempts and the actions the TOE takes when that threshold is reached (e.g., account lockout, notification, or timed delays).
Ability to manage the trusted public keys database	If the TOE offers ability for a remote authorised IT entities or authorised remote Administrators to connect via an interface secured with SSH, then the ST author must select the option “Ability to manage the trusted public keys database” to account for management of public key authentication. It is acceptable for this management function to be implemented as part of general TOE management functionality or as a standalone management function.
Ability to manage the public key or certificate used to validate the digital update	If the TOE offers the ability to modify the public key used to validate the digital update, then the ST author must select the option “Ability to manage the public key or certificate used to validate the digital update”. There is no requirement to implement this as a standalone management function, it is acceptable for this management function to be implemented as part of the trusted update (FPT_TUD_EXT.1) functionality.
No other capabilities	If the TOE offers the ability for the Security Administrator to configure the audit behaviour, configure the services available prior to identification or authentication, or if any of the cryptographic functionality on the TOE can be configured, or if the ST is describing a distributed TOE, then the ST author makes the appropriate choice or choices in the second selection, otherwise select the option "No other capabilities" (in the latter case the selection may alternatively be left blank in the ST).

Table 9: FMT_SMF.1.1 Management Function Guidance

With respect to FAU_GEN.1.1, FMT_SMF.1 and FMT_MOF.1/Services the term ‘services’ refers to trusted path and trusted channel communications, on demand self-tests, trusted update and Administrator sessions (that exist under the trusted path) (e.g., netconf).

Evaluation Activities

FMT_SMF.1

195. The security management functions for FMT_SMF.1 are distributed throughout the cPP and are included as part of the requirements in FTA_SSL_EXT.1, FTA_SSL.3, FTA_TAB.1, FMT_MOF.1/ManualUpdate, FMT_MOF.1/AutoUpdate (if included in the ST), FIA_AFL.1, FPT_TUD_EXT.1.2 and FPT_TUD_EXT.2.2 (if included in the ST and if an administrator-configurable action is included as per Application Note 65 of FPT_TUD_EXT.2.4 in the cPP), FMT_MOF.1/Services, and FMT_MOF.1/Functions (for all of these SFRs that are included in the ST), FMT_MTD, FPT_TST_EXT, and any cryptographic management functions specified in the reference standards. If the TOE claims conformance to the Functional Package for X.509, any management functions defined there are relevant to this SFR as well. Compliance to these requirements satisfies compliance with FMT_SMF.1.

TSS

196. The evaluator shall examine the TSS and Guidance Documentation and confirm that each management function specified in FMT_SMF.1 is adequately described. The evaluator shall confirm that the TSS details which security management functions are available through the local and/or remote administration interfaces.

197. [Conditional] The evaluator shall examine the TSS and Guidance Documentation to verify they both describe the local administrative interface. The evaluator shall ensure the Guidance Documentation includes appropriate warnings for the administrator to ensure the interface is local.

198. For distributed TOEs, with the option 'ability to configure the interaction between TOE components' the evaluator shall examine that the ways to configure the interaction between TOE components is detailed in the TSS and Guidance Documentation. The evaluator shall check that the TOE behaviour observed during testing of the configured SFRs is as described in the TSS and Guidance Documentation.

The following content should be included if:

- *Ability to configure local audit behaviour (e.g. changes to storage locations for audit; changes to behaviour when local audit storage space is full; changes to local audit storage size) is selected from FMT_SMF.1.1*

199. (If 'configure local audit' is selected) The evaluator shall examine the TSS and Guidance Documentation to ensure that a description of the logging implementation is described in enough detail to determine how log files are maintained on the TOE.

Guidance

200. Guidance activities are covered in the TSS section.

Tests

201. The evaluator shall test management functions as part of testing the SFRs identified in the introduction to this section. No separate testing for FMT_SMF.1 is required unless one of the

management functions in FMT_SMF.1.1 has not already been exercised under any other SFR.

FMT_SMR.2 Restrictions on security roles

FMT_SMR.2.1

The TSF shall maintain the roles:

- *Security Administrator.*

FMT_SMR.2.2

The TSF shall be able to associate users with roles.

FMT_SMR.2.3

The TSF shall ensure that the conditions

- *The Security Administrator role shall be able to administer the TOE remotely*

are satisfied.

Application Note: Application Note 28

FMT_SMR.2.3 requires that a Security Administrator be able to administer the TOE through a remote mechanism. See Application Note 23 for the definition of remote administration.

For distributed TOEs, not every TOE component is required to implement its own user management to fulfil this SFR. At least one component has to support authentication and identification of Security Administrators according to FIA_UIA_EXT.1. For the other TOE components authentication as Security Administrator can be realized through the use of a trusted channel (either according to FTP_ITC.1 or FPT_ITT.1) from a component that supports the authentication of Security Administrators according to FIA_UIA_EXT.1. The identification of users according to FIA_UIA_EXT.1.2 and the association of users with roles according to FMT_SMR.2.2 is done through the components that support the authentication of Security Administrators according to FIA_UIA_EXT.1.4. TOE components that authenticate Security Administrators through the use of a trusted channel are not required to support local administration of the component.

A single user associated with the Security Administrator role does not necessarily have to be able to perform all security management functions defined in FMT_SMF.1 and does not necessarily have to able to perform local administration. All users associated with the Security Administrator role together need to be able to perform all security management functions defined in FMT_SMF.1 (mandatory and selected ones) and need to be able to perform remote administration.

This implies that a user that can perform only a single security management function defined in FMT_SMF.1 needs to be regarded as Security Administrator of the TOE.

Evaluation Activities

FMT_SMR.2

TSS

202. The evaluator shall examine the TSS to determine that it details the TOE supported roles and any restrictions of the roles involving administration of the TOE (e.g., if local administrators and

remote administrators have different privileges, or if several types of administrators with different privileges are supported by the TOE).

Guidance

203. The evaluator shall review the guidance documentation to ensure that it contains instructions for administering the TOE both locally and remotely, including any configuration that needs to be performed on the client for remote administration.

Tests

204. In the course of performing the testing activities for the evaluation, the evaluator shall use all supported interfaces, although it is not necessary to repeat each test involving an administrative action with each interface. The evaluator shall ensure, however, that each supported method of administering the TOE that conforms to the requirements of this cPP be tested.

205. For example, if the following are possible:

- *direct connection if the TOE can be administered through a local hardware interface*
- *SSH if the TSF shall be validated against the Functional Package for Secure Shell referenced in Section 2.2 of the cPP*
- *TLS/HTTPS if the TSF has TLS implemented as defined in the Functional Package for TLS*

then all three methods of administration must be exercised during the evaluation team's test activities.

5.1.7 Protection of the TSF (FPT)

FPT_APW_EXT.1 Protection of Administrator Passwords

This is an implementation-based component. Its inclusion in depends on whether the TOE implements one or more of the following features:

- ***Password-based authentication***

as described in Appendix A.3: Implementation-based Requirements.

FPT_APW_EXT.1.1

The TSF shall store administrative passwords in non-plaintext form.

FPT_APW_EXT.1.2

The TSF shall prevent the reading of plaintext administrative passwords.

Application Note: Application Note 78

The intent of the requirement is that raw password authentication data of Security Administrators is not stored in the clear, and that no Administrator is able to read the plaintext password of a Security Administrator through "normal" interfaces. An all-powerful Administrator could directly read memory to capture a password but is trusted not to do so. Passwords should be obscured during entry on the local console in accordance with FIA_UAU.7.

Although this is out-of-scope of this cPP, it is strongly advised to protect all authentication data of the device the same way and/or with similar strength as

administrative passwords to reduce the risk of attacks like privilege escalation, etc.

Evaluation Activities

FPT_APW_EXT.1
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FPT_ITT.1 Basic internal TSF data transfer protection (Refinement)

This is an optional component. However, applied modules or packages might redefine it as mandatory.

FPT_ITT.1.1

The TSF shall protect TSF data from disclosure **and detect its modification** when it is transmitted between separate parts of the TOE **through the use of**

[**selection**: *IPsec, SSH as defined in the Functional Package for SSH, TLS as defined in the Functional Package for TLS, DTLS as defined in the Functional Package for TLS, HTTPS*].

Application Note: Application Note 44
This requirement is only applicable to distributed TOEs and ensures that all communications between components of the distributed TOE are protected through the use of an encrypted communications channel. The data passed in this trusted communication channel are encrypted as defined by the protocol chosen in the selection. The ST author should identify the channels and protocols used by each pair of communicating components in a distributed TOE, iterating this SFR as appropriate.

This channel may also be used as the registration channel for the registration process, as described in Section 3.3 and FCO_CPC_EXT.1.2.

If "TLS" or "DTLS" is selected, then the TSF is validated against the applicable requirements of the Functional Package for TLS. Additionally, the reference identifier established for the server (FCS_DTLSC_EXT.1.5 or FCS_TLSC_EXT.1.5 in the Functional Package for TLS) may be established through a “gatekeeper” discovery process. The TSS should describe the discovery process and highlight how the reference identifier is supplied to the “joining” component.

If "SSH" is selected, then the TSF is validated against the applicable requirements of the Functional Package for SSH.

See Section B.4.1 for additional requirements.

Evaluation Activities

FPT_ITT.1
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FPT_SKP_EXT.1 Protection of TSF Data (for reading of all pre-shared, symmetric and private keys)

FPT_SKP_EXT.1.1

The TSF shall prevent reading of all pre-shared keys, symmetric keys, and private keys.

Application Note: The intent of this requirement is for the device to protect keys, key material, and authentication credentials from unauthorised disclosure. This data should only be accessed for the purposes of their assigned security functionality, and there is no need for them to be displayed/accessed at any other time. This requirement does not prevent the device from providing indication that these exist, are in use, or are still valid. It does, however, restrict the reading of the values outright.

Evaluation Activities

FPT_SKP_EXT.1

TSS
206. The evaluator shall examine the TSS to determine that it details how any preshared keys, symmetric keys, and private keys are stored and that they are unable to be viewed through any interface designed specifically for that purpose, by any enabled role, as outlined in the application note. If these values are not stored in plaintext, the TSS shall describe how they are protected/obscured.

Guidance
207. None

Tests
208. None

FPT_STM_EXT.1 Reliable Time Stamps

FPT_STM_EXT.1.1

The TSF shall be able to provide reliable time stamps for its own use.

FPT_STM_EXT.1.2

The TSF shall [**selection**: *allow the Security Administrator to set the time*, *synchronise time with an NTP server*, *obtain time from the underlying virtualization system*].

Application Note: Reliable time stamps are expected to be used with other TSF, e.g., for the generation of audit data that enables the Security Administrator to investigate incidents by checking the order of events and determining the actual local time when events occurred. The required level of accuracy is determined by the Administrator.

The TOE depends on time and date information that may be provided by a local real-time clock managed by the Security Administrator, obtained from one or more NTP servers, or received from the underlying virtualization system. The corresponding option(s) are selected in FPT_STM_EXT.1.2. Automatic synchronization with an NTP server is recommended but not required. When the TOE communicates with an NTP server, the inclusion of FCS_NTP_EXT.1 in the ST is expected. The ST author describes in the TSS how the TOE receives external time and date information and how this information is maintained. For a Case 1 vND, the virtualization system can act as an external time source. For a Case 2 vND, the virtualization system is part of the TOE, so the time is typically set by a Security Administrator or synchronized with an NTP server.

The term “reliable time stamps” refers to the strict use of the provided time and date information and to the logging of all discontinuous changes to the time settings, including information about the old and new time values. With this information, the real time for all audit data can be determined. All discontinuous time changes, whether initiated by an Administrator or an automated process, are expected to be audited. No audit is needed when time is changed through kernel or system facilities —such as daytime (3)—that do not introduce discontinuities.

For distributed TOEs, the Security Administrator is expected to maintain synchronization between the time settings of different TOE components. All components should either remain synchronized (for example, by internal synchronization or by using a common NTP source) or have a known and documented offset for each component pair, including those synchronized to different time zones.

Evaluation Activities

FPT_STM_EXT.1

TSS

209. The evaluator shall examine the TSS to ensure that it lists each security function that makes use of time, and that it provides a description of how the time is maintained and considered reliable in the context of each of the time related functions.

The following content should be included if:

- *obtain time from the underlying virtualization system is selected from FPT_STM_EXT.1.2*

210. If 'obtain time from the underlying virtualization system' is selected, the evaluator shall examine the TSS to ensure that it identifies the Virtualization System (VS) interface the TOE uses

to obtain time. If there is a delay between updates to the time on the VS and updating the time on the TOE, the TSS shall identify the maximum possible delay.

Guidance

211. The evaluator shall examine the guidance documentation to ensure it instructs the administrator how to set the time. If the TOE supports the use of an NTP server, the guidance documentation instructs how a communication path is established between the TOE and the NTP server, and any configuration of the NTP client on the TOE to support this communication.

The following content should be included if:

- *obtain time from the underlying virtualization system is selected from FPT_STM_EXT.1.2*

212. If the TOE supports obtaining time from the underlying VS, then the evaluator shall verify that the Guidance Documentation specifies any configuration steps necessary. If no configuration is necessary, then no statement is necessary in the Guidance Documentation. If there is a delay between updates to the time on the VS and updating the time on the TOE, then the evaluator shall ensure the Guidance Documentation informs the administrator of the maximum possible delay.

Tests

213. The evaluator shall perform the following tests:

The following content should be included if:

- *allow the Security Administrator to set the time is selected from FPT_STM_EXT.1.2*

Test 1: If the TOE supports direct setting of the time by the Security Administrator, then the evaluator shall use the guidance documentation to set the time. The evaluator shall then use an available interface to observe that the time was set correctly.

The following content should be included if:

- *synchronise time with an NTP server is selected from FPT_STM_EXT.1.2*

Test 2: If the TOE supports the use of an NTP server, then the evaluator shall use the guidance documentation to configure the NTP client on the TOE and set up a communication path with the NTP server. The evaluator shall observe that the NTP server has set the time to what is expected. If the TOE supports multiple protocols for establishing a connection with the NTP server, then the evaluator shall perform this test using each supported protocol claimed in the guidance documentation.

The following content should be included if:

- *obtain time from the underlying virtualization system is selected from FPT_STM_EXT.1.2*

Test 3 [conditional]: If the TOE obtains time from the underlying VS, then the evaluator shall record the time on the TOE, modify the time on the underlying VS, and verify the modified time is reflected by the TOE. If there is a delay between setting the time on the VS and when the time is reflected on the TOE, then the evaluator shall ensure this delay is consistent with the TSS and Guidance.

214. If the audit component of the TOE consists of several parts with independent time information, then the evaluator shall verify that the time information between the different parts are either

synchronized or that it is possible for all audit information to relate the time information of the different part to one base information unambiguously.

FPT_TST_EXT.1 TSF Testing

FPT_TST_EXT.1.1

The TSF shall run a suite of the following self-tests:

- During initial start-up (on power on) to verify the integrity of the TOE firmware and software;
- Prior to providing any cryptographic service and

[**selection**: *at no other time, on-demand, continuously, [**assignment**: conditions under which self-tests should occur]*] to verify correct operation of cryptographic implementation necessary to fulfil the TSF;

- [**selection**: *no other, start-up, on-demand, continuous, at the conditions [**assignment**: conditions under which self-tests should occur]*]

self-tests [**assignment**: *'list an identifier for each self-test that is additional to those identified in the first two bullet points'*].to demonstrate the correct operation of the TSF.

Application Note: Application Note 31

For the third bullet point, the following restriction applies: If, and only if 'no other' is selected in the selection, 'none' may be used in the second assignment.

Non-distributed TOEs may internally consist of several components that contribute to enforcing SFRs. Self-testing should cover all components that contribute to enforcing SFRs and verification of integrity should cover all software that contributes to enforcing SFRs on all components.

For distributed TOEs, all TOE components have to perform self-tests. This does not necessarily mean that each TOE component has to carry out the same self-tests.

FPT_TST_EXT.1.2

The TSF shall respond to [**selection**: *all failures, [**assignment**: list of failures detected by self-tests]*] by [**selection**: *entering a maintenance mode, rebooting, [**assignment**: other methods to enter a secure state]*].

Application Note: Application Note 32

For failed self-tests related to enforcing SFRs as defined in FPT_TST_EXT.1.1, the reaction of the TOE to each failure is described in the ST. FPT_TST_EXT.1.2 supports two modeling approaches. In the first, the TOE reacts in the same way to all self-test failures that enforce SFRs by selecting “all failures” in the first selection and identifying the corresponding reaction in the second selection. In the second approach, the TOE may define different reactions for specific self-test failures by listing the failures in the first selection and the associated reactions in the second. In this latter case, the ST should clearly identify which self-test failure corresponds to each defined TOE behaviour.

Evaluation Activities

FPT_TST_EXT.1

TSS

215. The evaluator shall examine the TSS to ensure that it details each of the selftests that are identified by the SFR; this description should include an outline of what the tests are actually doing (e.g., rather than saying "memory is tested", a description similar to "memory is tested by writing a value to each memory location and reading it back to ensure it is identical to what was written" shall be used). The evaluator shall ensure that the TSS makes an argument that the tests are sufficient to demonstrate that the TSF is operating correctly. If more than one failure response is listed in FPT_TST_EXT.1.2, then the evaluator shall examine the TSS to ensure it clarifies which response is associated with which type of failure.

216. For distributed TOEs, the evaluator shall examine the TSS to ensure that it details which TOE component performs which self-tests and when these selftests are run. The evaluator shall also examine the TSS to ensure it describes how the TOE reacts if one or more TOE components fail self-testing (e.g., halting and displaying an error message; failover behaviour).

Guidance

217. The evaluator shall also ensure that the guidance documentation describes the possible errors that may result from such tests, and actions the administrator should take in response; these possible errors shall correspond to those described in the TSS.

218. For distributed TOEs, the evaluator shall ensure that the guidance documentation describes how to determine from an error message returned which TOE component has failed the self-test.

Tests

219. It is expected that at least the following tests are performed:

- *Test FPT_TST_EXT.1:1:*
  Verification of the integrity of the firmware and executable software of the TOE.
- *Test FPT_TST_EXT.1:2:*
  Verification of the correct operation of the cryptographic functions necessary to fulfil any of the SFRs.
- *Test FPT_TST_EXT.1:3:*
  220. Although formal compliance is not mandated, the self-tests performed should aim for a level of confidence comparable to:

  a. [FIPS 140-2], Section 4.9.1, Software/firmware integrity test for the verification of the integrity of the firmware and executable software. Note that the testing is not restricted to the cryptographic functions of the TOE.

  b. [FIPS 140-2], Section 4.9.1, Cryptographic algorithm test for the verification of the correct operation of cryptographic functions. Alternatively, national requirements of any CCRA member state for the security evaluation of cryptographic functions should be considered as appropriate.

  c. [FIPS 140-3] ([ISO/IEC 19790:2015]), Section 7.10.2.2, Software/firmware integrity test for the verification of the integrity of the firmware and executable software. Note that the testing is not restricted to the cryptographic functions of the TOE.

d. [ISO/IEC 19790:2025], Section 7.10.3.2, Pre-operational software/firmware integrity test. Verification using an approved integrity technique. Note that the testing is not restricted to the cryptographic functions of the TOE.

221. The evaluator shall verify that the self-tests described above are carried out according to the SFR and in agreement with the descriptions in the TSS.

The following content should be included if:

- *the TOE implements "Distributed TOE"*

222. For distributed TOEs, the evaluator shall perform testing of self-tests on all TOE components according to the description in the TSS about which self-tests are performed by which component.

FPT_TUD_EXT.1 Trusted Update

FPT_TUD_EXT.1.1

The TSF shall provide *Security Administrators* the ability to query the currently executing version of the TOE firmware/software and

[**selection**: *the most recently installed version of the TOE firmware/software, no other TOE firmware/software version*].

Application Note: Application Note 33

If a trusted update can be installed on the TOE with a delayed activation the version of both the currently executing image and the installed but inactive image must be provided. In this case the option “the most recently installed version of the TOE firmware/software” must be chosen from the selection in FPT_TUD_EXT.1.1. If all trusted updates become active as part of the installation process, only the currently executing version needs to be provided. In this case the option “no other TOE firmware/software version” should be chosen from the selection in FPT_TUD_EXT.1.1.

For a distributed TOE, the method of determining the installed versions on each component of the TOE is described in the operational guidance.

FPT_TUD_EXT.1.2

The TSF shall provide *Security Administrators* the ability to manually initiate updates to TOE firmware/software and

[**selection**: *support automatic checking for updates, support automatic updates, no other update mechanism*].

Application Note: Application Note 34

The selection in FPT_TUD_EXT.1.2 distinguishes the support of automatic checking for updates and support of automatic updates. The first option refers to a TOE that checks whether a new update is available, communicates this to the Administrator (e.g., through a message during an administrative session, through log files) but requires some action by the Administrator to actually perform the update. The second option refers to a TOE that checks for updates and automatically installs them upon availability. If the TOE checks and automatically installs the update, then FMT_MOF.1/AutoUpdate should be included.

FPT_TUD_EXT.1.3

The TSF shall provide means to authenticate firmware/software updates to the TOE using a

[**selection**: *X.509 certificate*, *digital signature*] prior to installing those udpates.

Application Note: Application Note 35

The ST author selects “X.509 certificate” when the TOE uses X.509 certificates in a manner compliant with the certificate validation requirements in the Functional Package for X.509. The digital signature algorithm must be one of the algorithms specified in FCS_COP.1/SigVer.

The ST author selects ‘digital signature’ for all other digital mechanisms (e.g., X.509 certificates that do not meet the certificate validation requirements in the Functional Package for X.509, GPG, raw public key). The digital algorithm must be one of the algorithms specified in FCS_COP.1/SigVer.

The TOE itself must perform the verification of the update signature, regardless of whether the update is authenticated using an X.509 certificate or another digital signature mechanism.

For distributed TOEs, all TOE components must support Trusted Update. The verification of the signature on the update should be done by each TOE component itself (signature verification).

Updating a distributed TOE might lead to the situation where different TOE components are running different software versions. Depending on the differences between the different software versions the impact of a mixture of different software versions might be no problem at all or critical to the proper functioning of the TOE. The TSS must detail the mechanisms that support the continuous proper functioning of the TOE during trusted update of distributed TOEs.

If “X.509 certificate” is selected, certificates are validated in accordance with the Functional Package for X.509. Additionally, FPT_TUD_EXT.2 must be included in the ST.

‘Update’ in the context of this SFR refers to the process of replacing a non-volatile (NV), system resident software component with another. The former is referred to as the NV image, and the latter is the update image. While the update image is typically newer than the NV image, this is not a requirement. There are legitimate cases where the system owner may want to rollback a component to an older version (e.g., when the component manufacturer releases a faulty update, or when the system relies on an undocumented feature no longer present in the update). Likewise, the owner may want to update with the same version as the NV image to recover from faulty storage.

All discrete firmware and software elements (e.g., applications, drivers, and kernel) of the TSF need to be protected, (i.e., they should be digitally signed by the corresponding manufacturer and subsequently verified by the mechanism performing the update).

Evaluation Activities

FPT_TUD_EXT.1

TSS

223. The evaluator shall verify that the TSS describes how to query the currently active version. If a trusted update can be installed on the TOE with a delayed activation, the TSS shall describe how

and when the inactive version becomes active. The evaluator shall verify this description.

224. The evaluator shall verify that the TSS describes all TSF software update mechanisms for updating the system firmware and software (for simplicity the term 'software' will be used in the following although the requirements apply to firmware and software). The evaluator shall verify that the description includes the method used to authenticate the update, including either digital-signature verification or X.509 certificate-based verification when selected in FPT_TUD_EXT.1. The evaluator shall verify that the TSS describes how candidate updates are obtained, the processing performed to authenticate the update, and the actions taken for both successful and unsuccessful authentication.

The following content should be included if:

- *support automatic checking for updates is selected from FPT_TUD_EXT.1.2*

225. If the options 'support automatic checking for updates' or 'support automatic updates' are chosen from the selections in FPT_TUD_EXT.1.2, the evaluator shall verify that the TSS explains what actions are involved in automatic checking or automatic updating by the TOE, respectively.

226. For distributed TOEs, the evaluator shall examine the TSS to ensure that it describes how all TOE components are updated, that it describes all mechanisms that support continuous proper functioning of the TOE during update (when applying updates separately to individual TOE components) and how verification of the signature is performed for each TOE component.

Guidance

227. The evaluator shall verify that the guidance documentation describes how to query the currently active version. If a trusted update can be installed on the TOE with a delayed activation, the guidance documentation shall describe how to query the loaded but inactive version.

228. The evaluator shall verify that the guidance documentation describes how the verification of the authenticity of the update is performed (digital signature verification). The description shall include the procedures for successful and unsuccessful verification. The description shall correspond to the description in the TSS.

229. For distributed TOEs, the evaluator shall verify that the guidance documentation describes how the versions of individual TOE components are determined for FPT_TUD_EXT.1, how all TOE components are updated, and the error conditions that may arise from checking or applying the update (e.g., failure of signature verification, or exceeding available storage space) along with appropriate recovery actions. The guidance documentation only has to describe the procedures relevant for the Security Administrator; it does not need to give information about the internal communication (amongst TOE components) that takes place when applying updates.

230. For distributed TOEs, the evaluator shall examine the Guidance Documentation to ensure that it describes how all TOE components are updated, that it describes all mechanisms that support continuous proper functioning of the TOE during update (when applying updates separately to individual TOE components) and how verification of the signature is performed for each TOE component.

The following content should be included if:

- *X.509 certificate is selected from FPT_TUD_EXT.1.3*

231. If the ST author indicates that a certificate-based mechanism is used for software update digital signature verification, the evaluator shall verify that the Guidance Documentation contains a description of how the certificates are contained on the device. The evaluator shall

also ensure that the Guidance Documentation describes how the certificates are installed/updated/selected, if necessary.

Tests

232. The evaluator shall perform the following tests:

- *Test FPT_TUD_EXT.1:1:*

  Test 1: The evaluator shall perform the version verification activity to determine the current version of the product. If a trusted update can be installed on the TOE with a delayed activation, the evaluator shall also query the most recently installed version (for this test the TOE shall be in a state where these two versions match). The evaluator will obtain a legitimate update using procedures described in the guidance documentation and verify that it has been successfully installed on the TOE. For some TOEs loading the update onto the TOE and activation of the update are separate steps ('activation' could be performed e.g., by a distinct activation step or by rebooting the device). In that case, the evaluator shall verify after loading the update onto the TOE but before activation of the update that the current version of the product did not change but the most recently installed version has changed to the new product version. After the update, the evaluator shall perform the version verification activity again to verify the version correctly corresponds to that of the update and that current version of the product and most recently installed version match again.

- *Test FPT_TUD_EXT.1:2:*

  Test 2: If the TOE itself verifies a digital signature in order to authorise the installation of an image to update the TOE, the following test shall be performed (otherwise the test shall be omitted). The evaluator shall first confirm that no updates are pending and then perform the version verification activity to determine the current version of the product, verifying that it is different from the version claimed in the update(s) to be used in this test. The evaluator shall obtain or produce illegitimate updates as defined below and attempt to install them on the TOE. The evaluator shall verify that the TOE rejects all of the illegitimate updates. The evaluator shall perform this test using all of the following forms of illegitimate updates:

  i. A modified version (e.g., using a hex editor) of a legitimately signed update

  ii. An image that has not been signed

  iii. An otherwise valid image with a properly formed signature that was signed by an unknown key. The purpose of this test is to verify the TOE would only accept images signed by an explicitly trusted key (or a key associated with a trusted certificate).

  iv. The handling of version information of the most recently installed version might differ between different TOEs depending on the point in time when an attempted update is rejected. The evaluator shall verify that the TOE handles the most recently installed version information for that case as described in the guidance documentation. After the TOE has rejected the update, the evaluator shall verify that both the current version and most recently installed version reflect the same version information as prior to the update attempt.

- *Test FPT_TUD_EXT.1:3:*

  233. The evaluator shall perform Test 1 and Test 2 for all supported methods (manual updates, automatic checking for updates, automatic updates).

  The following content should be included if:
  - *the TOE implements "Distributed TOE"*

234. For distributed TOEs, the evaluator shall perform Test 1 and Test 2 for all TOE components.

FPT_TUD_EXT.2 Trusted Update Based on Certificates

This is a selection-based component. Its inclusion depends upon selection from FPT_TUD_EXT.1.3.

FPT_TUD_EXT.2.1

The TSF shall check the validity of the code signing certificate before installing each update.

FPT_TUD_EXT.2.2

If revocation information is not available for a certificate in the trust chain that is not a trusted certificate designated as a trust anchor, the TSF shall [**selection**: *not install the update, allow the Administrator to choose whether to accept the certificate in these cases*].

FPT_TUD_EXT.2.3

If the certificate is deemed invalid because the certificate has expired, the TSF shal [**selection**: *allow the Administrator to choose whether to install the update in these cases, not accept the certificate*].

FPT_TUD_EXT.2.4

If the certificate is deemed invalid for reasons other than expiration or revocation information being unavailable, the TSF shall not install the update.

Application Note: Application Note 79

This component must be included in the ST if “X.509 digital signature mechanism” is selected in FPT_TUD_EXT.1.3.

Validity is determined in accordance with FIA_X509_EXT.1. in the Functional Package for X.509

It is acceptable to provide a manual method for an administrator to provide revocation information (e.g., CRL upload) in addition to retrieving revocation information automatically in accordance with FIA_X509_EXT.1 and FIA_X509_EXT.2 in the Functional Package for X.509. It is expected that current updates are signed using current (not expired) certificates that will be valid at least until the next expected update. However, an administrator may desire to install previous updates that are signed by expired certificates. To indicate support for this practice, the author of the ST selects whether the certificate will be accepted, rejected, or the choice is left to the Administrator to accept or reject the certificate.

Evaluation Activities

FPT_TUD_EXT.2
Something
TSS

ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

5.1.8 TOE Access (FTA)

FTA_SSL.3 TSF-initiated Termination (Refinement)

FTA_SSL.3.1

The TSF shall terminate **a remote** interactive session after a *Security Administrator-configurable time interval of session inactivity*.

Application Note: Application Note 36
An interactive session governed by this SFR is a session in which an authenticated state is achieved and then preserved across multiple commands. By contrast, if authentication accompanies each individual command (without preservation of the same authenticated state) then this is not considered an interactive session.

Evaluation Activities

FTA_SSL.3

TSS
235. The evaluator shall examine the TSS to determine that it details the administrative remote session termination and the related inactivity time period.

Guidance
236. The evaluator shall confirm that the guidance documentation includes instructions for configuring the inactivity time period for remote administrative session termination.

Tests
237. For each method of remote administration, the evaluator shall perform the following test:

- *Test FTA_SSL.3:1:*
  Test 1: The evaluator shall follow the guidance documentation to configure several different values for the inactivity time period referenced in the component. For each period configured, the evaluator shall establish a remote interactive session with the TOE. The evaluator shall then observe that the session is terminated after the configured time period.

FTA_SSL.4 User-initiated Termination (Refinement)

FTA_SSL.4.1

The TSF shall allow Administrator-initiated termination of the Administrator's own interactive session.

Evaluation Activities

FTA_SSL.4

TSS

238. The evaluator shall examine the TSS to determine that it details how the remote administrative session (and if applicable the local administrative session) are terminated.

Guidance

239. The evaluator shall confirm that the guidance documentation states how to terminate a remote interactive session (and if applicable the local administrative session).

Tests

240. The evaluator shall perform the following tests:

The following content should be included if:

- *the TOE implements "Local security admin"*

Test 1 [conditional]: If the TOE supports local administration, the evaluator shall initiate an interactive local session with the TOE. The evaluator shall then follow the guidance documentation to exit or log off the session and observe that the session has been terminated.

The following content should be included if:

- *the TOE implements "Password-based remote administration"*

Test 2: For each method of remote administration, the evaluator shall initiate an interactive remote session with the TOE. The evaluator shall then follow the guidance documentation to exit or log off the session and observe that the session has been terminated.

FTA_SSL_EXT.1 TSF-initiated Session Locking

This is an implementation-based component. Its inclusion in depends on whether the TOE implements one or more of the following features:

- ***Local security admin***

as described in Appendix A.3: Implementation-based Requirements.

FTA_SSL_EXT.1.1

The TSF shall, for local interactive sessions, [**selection**: *lock the session - disable any activity of the Administrator's data access/display devices other than unlocking the session, and requiring that the Administrator reauthenticate to the TSF prior to unlocking the session*, *terminate the session*] after a Security Administrator-specified time period of inactivity.

Application Note: Application Note 84
An interactive session governed by this SFR is a session in which an authenticated state is achieved and then preserved across multiple commands. By contrast, if authentication accompanies each individual command (without preservation of the same authenticated state) then this is not considered an interactive session.

Evaluation Activities

FTA_SSL_EXT.1
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

FTA_TAB.1 Default TOE Access Banners (Refinement)

FTA_TAB.1.1

Before establishing an **Administrative** user session the *TSF* shall display **a Security Administrator-specified advisory notice and consent warning regarding use of the TOE** message.

Application Note: Application Note 37
This requirement is intended to apply to interactive sessions between a human administrator and a TOE. IT entities establishing connections or programmatic connections (e.g., remote procedure calls over a network) are not required to be covered by this requirement.

Evaluation Activities

FTA_TAB.1

TSS
241. The evaluator shall check the TSS to ensure that it details each administrative method of access (local and/or remote) available to the Security Administrator (e.g., serial port, SSH, HTTPS). The evaluator shall check the TSS to ensure that all administrative methods of access available to the Security Administrator are listed and that the TSS states that the TOE is displaying an advisory notice and a consent warning message for each administrative method of access. The advisory notice and the consent warning message may be different for different administrative methods of access and may be configured during initial configuration (e.g., via a configuration file).

Guidance
242. The evaluator shall check the guidance documentation to ensure that it describes how to configure the banner message.

Tests

243. The evaluator shall also perform the following test:

- *Test FTA_TAB.1:1:*
  Test 1: The evaluator shall follow the guidance documentation to configure a notice and consent warning message. The evaluator shall then, for each method of human interactive access specified in the TSS, establish a session with the TOE. The evaluator shall verify that the notice and consent warning message is displayed in each instance.

5.1.9 Trusted Channel (FTP_ITC)

FTP_ITC.1 Inter-TSF Trusted Channel (Refinement)

FTP_ITC.1.1

The TSF shall **be capable of using**

[**selection**: *IPsec, SSH as defined in the Functional Package for SSH, TLS as defined in the Functional Package for TLS, DTLS as defined in the Functional Package for TLS, HTTPS*]
to provide a **trusted** communication channel between itself and ~~another trusted IT product~~ **authorised IT entities supporting the following capabilities: audit server,**

[**selection**: *authentication server, [**assignment**: other capabilities], no other capabilities*]
that is logically distinct from other communication channels and provides assured identification of its end points and protection of the channel data from ~~modification or~~ disclosure **and detection of modification of the channel data**.

FTP_ITC.1.2

The TSF shall permit [**selection**: *the TSF, another trusted IT product, the authorised IT entities*] to initiate communication via the trusted channel.

FTP_ITC.1.3

The TSF shall initiate communication via the trusted channel for [**assignment**: *list of services for which the TSF is able to initiate communications*].

Application Note: Application Note 38
The intent of the above requirement is to provide a means by which a cryptographic protocol may be used to protect external communications with authorised IT entities that the TOE interacts with to perform its functions. The TOE uses at least one of the listed protocols for communications with the server that collects the audit information. If it communicates with an authentication server (e.g., RADIUS), then the ST author chooses “authentication server” in FTP_ITC.1.1 and this connection must be capable of being protected by one of the listed protocols. If other authorised IT entities are protected, the ST author makes the appropriate assignments (for those entities) and selections (for the protocols that are used to protect those connections). The ST author selects the mechanism or mechanisms supported by the TOE, and then ensures that the detailed protocol requirements in Annex B corresponding to their selection are included in the ST.

While there are no requirements on the party initiating the communication, the ST author lists in the assignment for FTP_ITC.1.3 the services for which the TOE can initiate the communication with the authorised IT entity.

The requirement implies that not only are communications protected when they are initially established, but also on resumption after an outage. It may be the case that some part of the TOE setup involves manually setting up tunnels to protect other communication, and if after an outage the TOE attempts to re-establish the communication automatically with (the necessary) manual intervention, there may be a window created where an attacker might be able to gain critical information or compromise a connection.

Where X.509 certificates are used to authenticate remote endpoints in support of an FTP_ITC.1 channel, relevant SFR claims from the Functional Package for X.509 must be used. This requires support for attributes such as certificate revocation status and intermediate CAs.

If the TOE claims FCS_TLSS_EXT.2 (TLS Server Support for Mutual Authentication) from the Functional Package for TLS and the TOE passes presented identifiers of clients used for client authentication to a directory server for comparison, then the connection to the directory server used to verify presented identifiers of TLS clients needs to be protected by a trusted channel (i.e., FTP_ITC.1). If a trusted channel is used for the integrity protection for communication between the TOE and a directory server, then the directory server must be added to the assignment for other capabilities in FTP_ITC.1. Note: The directory server is only expected to handle the comparison of the presented identifier but not to perform full X.509 certificate validation on behalf of the TOE.

See Section B.4.1 for additional requirements.

If "TLS" or "DTLS" is selected, then the TSF is validated against the applicable requirements of the Functional Package for TLS.

If "SSH" is selected, then the TSF is validated against the applicable requirements of the Functional Package for SSH.

Evaluation Activities

FTP_ITC.1

TSS

244. The evaluator shall examine the TSS to determine that, for all communications with authorised IT entities identified in the requirement, each secure communication mechanism is identified in terms of the allowed protocols for that IT entity, whether the TOE acts as a server or a client, and the method of assured identification of the non-TSF endpoint. The evaluator shall also confirm that all secure communication mechanisms are described in sufficient detail to allow the evaluator to match them to the cryptographic protocol SFRs listed in the ST.

Guidance

245. The evaluator shall confirm that the guidance documentation contains instructions for establishing the allowed protocols with each authorised IT entity, and that it contains recovery instructions should a connection be unintentionally broken.

Tests

246. The developer shall provide to the evaluator application layer configuration settings for all secure communication mechanisms specified by the FTP_ITC.1 requirement. This information should be sufficiently detailed to allow the evaluator to determine the application layer timeout

settings for each cryptographic protocol. There is no expectation that this information must be recorded in any public-facing document or report.

247. The evaluator shall perform the following tests:

- *Test FTP_ITC.1:1:*
  Test 1: The evaluator shall ensure that communications using each protocol with each authorised IT entity is tested during the course of the evaluation, setting up each connection as described in the guidance documentation and ensuring that communication is successful.
- *Test FTP_ITC.1:2:*
  Test 2: For each protocol that the TOE can initiate as defined in the requirement, the evaluator shall follow the guidance documentation to ensure that the communication channel can in fact be initiated from the TOE.
- *Test FTP_ITC.1:3:*
  Test 3: The evaluator shall verify that, for each communication channel with an authorised IT entity, the channel data is not sent in plaintext.
- *Test FTP_ITC.1:4:*
  Test 4: Objective: The objective of this test is to ensure that the TOE reacts appropriately to any connection outage or interruption of the route to the external IT entities.

  The evaluator shall, for each instance where the TOE acts as a client using a secure communication mechanism with a distinct IT entity, interrupt the connection of that IT entity for: i) a duration that exceeds the TOE's application layer timeout setting, ii) a duration shorter than the application layer timeout but of sufficient length to interrupt the network link layer.

  The evaluator shall ensure that, when the connectivity is restored, communications are appropriately protected and no TSF data is sent in plaintext.

  In the case where the TOE is able to detect TOE external interruption (such as a cable being physically removed or a virtual connection being disabled), another network device shall be used to interrupt the connection between the TOE and the distinct IT entity. The interruption shall be external to the TOE (i.e., by manipulating the test environment and not by TOE configuration change).

248. Further assurance activities are associated with the specific protocols.

249. For distributed TOEs, the evaluator shall perform tests on all TOE components according to the mapping of external secure channels to TOE components in the Security Target.

250. The developer shall provide to the evaluator application layer configuration settings for all secure communication mechanisms specified by the FTP_ITC.1 requirement. This information should be sufficiently detailed to allow the evaluator to determine the application layer timeout settings for each cryptographic protocol. There is no expectation that this information must be recorded in any public- facing document or report.

FTP_TRP.1/Admin Trusted Path (Refinement)

FTP_TRP.1.1/Admin

The TSF shall **be capable of using**

[**selection**: *IPsec, SSH as defined in the Functional Package for SSH, TLS as defined in the Functional Package for TLS, DTLS as defined in the Functional Package for TSL, HTTPS*]

to provide a communication path between itself and **authorised** remote **Administrators** ~~users~~ that is logically distinct from other communication paths and provides assured identification of its endpoints and protection of the communicated data from disclosure **and provides detection of modification of the channel data.**

FTP_TRP.1.2/Admin

The TSF shall permit remote **Administrators** ~~users~~ to initiate communication via the trusted path.

FTP_TRP.1.3/Admin

The TSF shall require the use of the trusted path for initial **Administrator** authentication and all remote administration actions.

Application Note: Application Note 39

This requirement ensures that authorised remote Administrators initiate all communication with the TOE via a human-interactive trusted path, and that all communication with the TOE by remote Administrators is performed over this path. The data passed in this trusted communication channel is encrypted as defined by the protocol chosen in the first selection. The ST author selects the mechanism or mechanisms supported by the TOE, and then ensures that the detailed protocol requirements in Annex B corresponding to their selection, or the protocol requirements of the packages specified in Section 2.1 are included in the ST. Where X.509 certificates are used to authenticate authorised Administrators, FIA_X509_EXT.1 in the Functional Package for X.509 is to be used (which requires checking certificate revocation, implementing a trust store, and supporting a certificate chain).

See Section B.4.1 for additional requirements.

If "TLS" or "DTLS" is selected, then the TSF is validated against the applicable requirements of the Functional Package for TLS.

If "SSH" is selected, then the TSF is evaluated against the applicable requirements of the Functional Package for SSH.

Evaluation Activities

FTP_TRP.1/Admin

TSS

251. The evaluator shall examine the TSS to determine that the remote TOE administration methods are indicated, along with how those communications are protected. The evaluator shall also confirm that all protocols listed in the TSS in support of TOE administration are consistent with those specified in the requirement, and are included in the requirements in the ST.

Guidance

252. The evaluator shall confirm that the guidance documentation contains instructions for establishing the remote administrative sessions for each supported method.

Tests

253. The evaluator shall perform the following tests:

- *Test FTP_TRP.1/Admin:1:*
  Test 1: The evaluators shall ensure that communications using each specified (in the guidance documentation) remote administration method are tested during the course of the evaluation, setting up the connections as described in the guidance documentation and verifying that communication is successful.
- *Test FTP_TRP.1/Admin:2:*
  Test 2: The evaluator shall verify, for each communication channel, the channel data is not sent in plaintext.

254. Further assurance activities are associated with the specific protocols.

255. For distributed TOEs, the evaluator shall perform tests on all TOE components according to the mapping of trusted paths to TOE components in the Security Target.

FTP_TRP.1/Join Trusted Path (Refinement)

This is an optional component. However, applied modules or packages might redefine it as mandatory.

FTP_TRP.1.1/Join

The TSF shall provide a communication path between itself and **a joining component** ~~[selection: remote, local] users~~ that is logically distinct from other communication paths and provides assured identification of

[**selection**: *the TSF endpoint, both joining component and TSF endpoint*]
~~its endpoints~~ and protection of the communicated data from modification **and**

[**selection**: *disclosure, no other mechanisms*].

FTP_TRP.1.2/Join

The TSF shall permit [**selection**: *the TSF, the joining component*] to initiate communication via the trusted path.

FTP_TRP.1.3/Join

The TSF shall require the use of the trusted path for joining components to the TSF under environmental constraints identified in

[**assignment**: *reference to operational guidance*].

Application Note: Application Note 45
This SFR implements one of the types of channel identified in the main selection for FCO_CPC_EXT.1.2. The “joining component” in FTP_TRP.1/Join is the IT entity that is attempting to join the distributed TOE by using the registration process.

The effect of this SFR is to require the ability for components to communicate in a secure manner while the distributed TSF is being created (or when adding components to an existing distributed TSF). When creating the TSF from the initial pair of components, either of these components may be identified as the TSF for the purposes of satisfying the meaning of ‘TSF’ in this SFR.

The selection at the end of FTP_TRP.1.1/Join recognises that in some cases confidentiality (i.e., protection of the data from disclosure) may not be provided by the channel. The ST author distinguishes in the TSS whether in this case the TOE relies on the environment to provide confidentiality (as part of the constraints referenced in FTP_TRP.1.3/Join) or whether the registration data exchanged does not require confidentiality (in which case this assertion must be justified). If 'no other mechanisms' is selected, the ST author may omit this phrase in the completed SFR text to improve readability.

The assignment in FTP_TRP.1.3/Join ensures that the ST highlights any specific details needed to protect the registration environment.

Note: When the ST uses FTP_TRP.1/Join for the registration channel then this channel cannot be reused as the normal inter-component communication channel (the latter channel must meet FTP_ITC.1 or FPT_ITT.1).

The trusted path used for joining might utilise X.509 certificates; however, there are no required X.509 SFRs associated with this trusted path as there are many ways the security of the joining path could be provided. It is up to the ST author to describe how the security of this trusted path is implemented; whether the security relies on X.509 SFRs, environmental constraints from FTP_TRP.1.3/Join, and/or some other method.

See Section B.4.1 for additional requirements.

Specific requirements for Preparative Procedures relating to FTP_TRP.1/Join are defined in the Evaluation Activities in [SD].

Evaluation Activities

FTP_TRP.1/Join
Something
TSS
ABC

Guidance
There are no additional Guidance evaluation activities for this component.

Tests
There are no test activities for this component.

5.1.10 TOE Security Functional Requirements Rationale

The following rationale provides justification for each SFR for the TOE, showing that the SFRs are suitable to address the specified threats:

Table 14: SFR Rationale

Threat	Addressed by	Rationale
T.SECURITY_ FUNCTIONALITY_ COMPROMISE	FCS_CKM.6	
	FIA_PMG_EXT.1 (Implementation-based)	

	FIA_UAU.7 (Implementation-based)	
	FMT_MTD.1/CryptoKeys (Implementation-based)	
	FMT_SMF.1	
	FPT_APW_EXT.1 (Implementation-based)	
	FPT_SKP_EXT.1	
T.SECURITY_FUNCTIONALITY_FAILURE	FPT_TST_EXT.1	
T.UNAUTHORISED_ADMINISTRATOR_ACCESS	FIA_AFL.1 (Implementation-based)	If the TOE provides remote administration using a password-based authentication mechanism, FIA_AFL.1 provides actions on reaching a threshold number of consecutive password failures
	FIA_UIA_EXT.1	
	FMT_MOF.1/Functions (Selection-based)	optional additional administration capabilities are offered in FMT_MOF.1/Functions
	FMT_MOF.1/Services (Selection-based)	optional additional administration capabilities are offered in FMT_MOF.1/Servicesoptional additional administration capabilities in FMT_MOF.1/Services and FMT_MOF.1/Functions
	FMT_MTD.1/CoreData	The relevant administration capabilities are defined in FMT_MTD.1/CoreData
	FMT_SMF.1	The relevant administration capabilities are defined in FMT_SMF.1 The relevant administration capabilities are defined in FMT_SMF.1 The relevant administration capabilities are defined in FMT_SMF.1
	FMT_SMR.2	The Administrator role is defined in FMT_SMR.2
	FTA_SSL_EXT.1 (Implementation-based)	
	FTA_SSL.3	
	FTA_SSL.4	
	FTA_TAB.1	

	FTP_TRP.1/Admin	The secure channel used for remote Administrator connections is specified in FTP_TRP.1/AdminThe secure channel used for remote Administrator connections is specified in FTP_TRP.1/Admin
T.UNDETECTED_ ACTIVITY	FAU_GEN.1	
	FAU_GEN.2	
	FAU_SAR.1 (Selection-based)	
	FAU_STG_EXT.1	
	FAU_STG_EXT.4 (Selection-based)	
	FAU_STG.2 (Optional)	
	FCS_NTP_EXT.1 (Selection-based)	
	FMT_MOF.1/Functions (Selection-based)	
	FMT_SMF.1	
	FPT_STM_EXT.1	
T.UNTRUSTED_ COMMUNICATIONS_ CHANNELS	FCS_IPSEC_EXT.1 (Selection-based)	
	FPT_ITT.1 (Optional)	
	FTP_ITC.1	
	FTP_TRP.1/Admin	
T.UPDATE_ COMPROMISE	FMT_MOF.1/AutoUpdate (Selection-based)	
	FMT_MOF.1/ManualUpdate	
	FMT_SMF.1	
	FPT_TUD_EXT.1	
	FPT_TUD_EXT.2 (Selection-based)	
T.WEAK_ AUTHENTICATION_ ENDPOINTS	FCO_CPC_EXT.1 (Optional)	
	FPT_ITT.1 (Optional)	
	FTP_ITC.1	

	FTP_TRP.1/Admin
	FTP_TRP.1/Join (Optional)
T.WEAK_ CRYPTOGRAPHY	FCS_CKM_EXT.7
	FCS_CKM.1/AKG
	FCS_COP.1/CMAC (Selection-based)
	FCS_COP.1/DataEncryption
	FCS_COP.1/Hash
	FCS_COP.1/KeyedHash
	FCS_COP.1/SigGen
	FCS_COP.1/SigVer
	FCS_RBG.1
	FMT_SMF.1

5.2 Security Assurance Requirements

5.2.1 Class ADV: Development

The information about the TOE is contained in the guidance documentation available to the end user as well as the TSS portion of the ST. The TOE developer must concur with the description of the product that is contained in the TSS as it relates to the functional requirements. The evaluation activities contained in Section 5.1 Security Functional Requirements should provide the ST authors with sufficient information to determine the appropriate content for the TSS section.

ADV_FSP.1 Basic Functional Specification (ADV_FSP.1)

The functional specification describes the TSFIs. It is not necessary to have a formal or complete specification of these interfaces. Additionally, because TOEs conforming to this PP will necessarily have interfaces to the Operational Environment that are not directly invocable by TOE users, there is little point specifying that such interfaces be described in and of themselves since only indirect testing of such interfaces may be possible. For this PP, the activities for this family should focus on understanding the interfaces presented in the TSS in response to the functional requirements and the interfaces presented in the AGD documentation. No additional “functional specification” documentation is necessary to satisfy the evaluation activities specified. The interfaces that need to be evaluated are characterized through the information needed to perform the assurance activities listed, rather than as an independent, abstract list.

Developer action elements:

ADV_FSP.1.1D

The developer shall provide a functional specification.

ADV_FSP.1.2D

The developer shall provide a tracing from the functional specification to the SFRs.

Content and presentation elements:

ADV_FSP.1.1C

The functional specification shall describe the purpose and method of use for each SFR-enforcing and SFR-supporting TSFI.

ADV_FSP.1.2C

The functional specification shall identify all parameters associated with each SFR-enforcing and SFR-supporting TSFI.

ADV_FSP.1.3C

The functional specification shall provide rationale for the implicit categorization of interfaces as SFR-non-interfering.

ADV_FSP.1.4C

The tracing shall demonstrate that the SFRs trace to TSFIs in the functional specification.

Evaluator action elements:

ADV_FSP.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ADV_FSP.1.2E

The evaluator shall determine that the functional specification is an accurate and complete instantiation of the SFRs.

Evaluation Activities

ADV_FSP.1
There are no specific assurance activities associated with these SARs, except ensuring the information is provided. The functional specification documentation is provided to support the evaluation activities described in Section 5.1 Security Functional Requirements, and other activities described for AGD, ATE, and AVA SARs. The requirements on the content of the functional specification information is implicitly assessed by virtue of the other assurance activities being performed; if the evaluator is unable to perform an activity because there is insufficient interface information, then an adequate functional specification has not been provided.

5.2.2 Class AGD: Guidance Documents

The guidance documents will be provided with the ST. Guidance must include a description of how the IT personnel verifies that the Operational Environment can fulfill its role for the security functionality. The documentation should be in an informal style and readable by the IT personnel. Guidance must be provided for every operational environment that the product supports as claimed in the ST. This guidance includes instructions to successfully install the TSF in that environment; and Instructions to manage the security of the TSF as a product and as a component of the larger operational environment. Guidance pertaining to particular security functionality is also provided; requirements on such guidance are contained in the evaluation activities specified with each requirement.

AGD_OPE.1 Operational User Guidance (AGD_OPE.1)

Developer action elements:

AGD_OPE.1.1D

The developer shall provide operational user guidance.

Application Note: The operational user guidance does not have to be contained in a single document. Guidance to users, administrators and application developers can be spread among documents or web pages. Rather than repeat information here, the developer should review the assurance activities for this component to ascertain the specifics of the guidance that the evaluator will be checking for. This will provide the necessary information for the preparation of acceptable guidance.

Content and presentation elements:

AGD_OPE.1.1C

The operational user guidance shall describe, for each user role, the user-accessible functions and privileges that should be controlled in a secure processing environment, including appropriate warnings.

Application Note: User and administrator are to be considered in the definition of user role.

AGD_OPE.1.2C

The operational user guidance shall describe, for each user role, how to use the available interfaces provided by the TOE in a secure manner.

AGD_OPE.1.3C

The operational user guidance shall describe, for each user role, the available functions and interfaces, in particular all security parameters under the control of the user, indicating secure values as appropriate.

Application Note: This portion of the operational user guidance should be presented in the form of a checklist that can be quickly executed by IT personnel (or end-users, when necessary) and suitable for use in compliance activities. When possible, this guidance is to be expressed in the eXtensible Configuration Checklist Description Format (XCCDF) to support security automation. Minimally, it should be presented in a structured format which includes a title for each configuration item, instructions for achieving the secure configuration, and any relevant rationale.

AGD_OPE.1.4C

The operational user guidance shall, for each user role, clearly present each type of security-relevant event relative to the user-accessible functions that need to be performed, including changing the security characteristics of entities under the control of the TSF.

AGD_OPE.1.5C

The operational user guidance shall identify all possible modes of operation of the TOE (including operation following failure or operational error), their consequences, and implications for maintaining secure operation.

AGD_OPE.1.6C

The operational user guidance shall, for each user role, describe the security measures to be followed in order to fulfill the security objectives for the operational environment as described in the ST.

AGD_OPE.1.7C

The operational user guidance shall be clear and reasonable.

Evaluator action elements:

AGD_OPE.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

Evaluation Activities

AGD_OPE.1
Some of the contents of the operational guidance are verified by the assurance activities in Section 5.1 Security Functional Requirements and evaluation of the OS according to the [CEM]. The following additional information is also required. If cryptographic functions are provided by the OS, the operational guidance shall contain instructions for configuring the cryptographic engine associated with the evaluated configuration of the OS. It shall provide a warning to the administrator that use of other cryptographic engines was not evaluated nor tested during the CC evaluation of the OS. The documentation must describe the process for verifying updates to the OS by verifying a digital signature – this may be done by the OS or the underlying platform. The evaluator will verify that this process includes the following steps: Instructions for obtaining the update itself. This should include instructions for making the update accessible to the OS (e.g., placement in a specific directory). Instructions for initiating the update process, as well as discerning whether the process was successful or unsuccessful. This includes generation of the hash/digital signature. The OS will likely contain security functionality that does not fall in the scope of evaluation under this PP. The operational guidance shall make it clear to an administrator which security functionality is covered by the evaluation activities.

AGD_PRE.1 Preparative Procedures (AGD_PRE.1)

Developer action elements:

AGD_PRE.1.1D

The developer shall provide the TOE, including its preparative procedures.

Content and presentation elements:

AGD_PRE.1.1C

The preparative procedures shall describe all the steps necessary for secure acceptance of the delivered TOE in accordance with the developer's delivery procedures.

AGD_PRE.1.2C

The preparative procedures shall describe all the steps necessary for secure installation of the TOE and for the secure preparation of the operational environment in accordance with the security objectives for the operational environment as described in the ST.

Evaluator action elements:

AGD_PRE.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

AGD_PRE.1.2E

The evaluator shall apply the preparative procedures to confirm that the TOE can be prepared securely for operation.

5.2.3 Class ALC: Life-cycle Support

At the assurance level provided for TOEs conformant to this PP, life-cycle support is limited to end-user-visible aspects of the life-cycle, rather than an examination of the TOE vendor's development and configuration management process. This is not meant to diminish the critical role that a developer's practices play in contributing to the overall trustworthiness of a product; rather, it is a reflection on the information to be made available for evaluation at this assurance level.

ALC_CMC.1 Labeling of the TOE (ALC_CMC.1)

This component is targeted at identifying the TOE such that it can be distinguished from other products or versions from the same vendor and can be easily specified when being procured by an end user.

Developer action elements:

ALC_CMC.1.1D

The developer shall provide the TOE and a reference for the TOE.

Content and presentation elements:

ALC_CMC.1.1C

The application shall be labeled with a unique reference.

Evaluator action elements:

ALC_CMC.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

Evaluation Activities

ALC_CMC.1
The evaluator will check the ST to ensure that it contains an identifier (such as a product name/version number) that specifically identifies the version that meets the requirements of the ST. Further, the evaluator will check the AGD guidance and OS samples received for testing to ensure that the version number is consistent with that in the ST. If the vendor maintains a web site advertising the OS, the evaluator will examine the information on the web site to ensure that the information in the ST is sufficient to distinguish the product.

ALC_CMS.1 TOE CM Coverage (ALC_CMS.1)

Developer action elements:

ALC_CMS.1.1D

The developer shall provide a configuration list for the TOE.

Content and presentation elements:

ALC_CMS.1.1C

The configuration list shall include the following: the TOE itself; and the evaluation evidence required by the SARs.

ALC_CMS.1.2C

The configuration list shall uniquely identify the configuration items.

Evaluator action elements:

ALC_CMS.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

Evaluation Activities

ALC_CMS.1
The "evaluation evidence required by the SARs" in this PP is limited to the information in the ST coupled with the guidance provided to administrators and users under the AGD requirements. By ensuring that the OS is specifically identified and that this identification is consistent in the ST and in the AGD guidance (as done in the assurance activity for ALC_CMC.1), the evaluator implicitly confirms the information required by this component. Life-cycle support is targeted aspects of the developer's life-cycle and instructions to providers of applications for the developer's devices, rather than an in-depth examination of the TSF manufacturer's development and configuration management process. This is not meant to diminish the critical role that a developer's practices play in contributing to the overall trustworthiness of a product; rather, it's a reflection on the information to be made available for evaluation.
The evaluator will ensure that the developer has identified (in guidance documentation for application developers concerning the targeted platform) one or more development environments appropriate for use in developing applications for the developer's platform. For each of these development environments, the developer shall provide information on how to configure the environment to ensure that buffer overflow protection mechanisms in the environment(s) are invoked (e.g., compiler and linker flags). The evaluator will ensure that this documentation also includes an indication of whether such protections are on by default, or have to be specifically enabled. The evaluator will ensure that the TSF is uniquely identified (with respect to other products from the TSF vendor), and that documentation provided by the developer in association with the requirements in the ST is associated with the TSF using this unique identification.

5.2.4 Class ASE: ST Evaluation

As per ASE activities defined in [CEM].

ASE_CCL.1 Conformance Claims

Developer action elements:

ASE_CCL.1.1D

The developer shall provide a conformance claim.

ASE_CCL.1.2D

The developer shall provide a conformance claim rationale.

Content and presentation elements:

ASE_CCL.1.1C

The conformance claim shall identify the edition of the CC to which the ST and the TOE claim conformance.

ASE_CCL.1.2C

The conformance claim shall describe the conformance of the ST to CC Part 2 as either CC Part 2 conformant or CC Part 2 extended.

ASE_CCL.1.3C

The conformance claim shall describe the conformance of the ST as either "CC Part 3 conformant" or "CC Part 3 extended".

ASE_CCL.1.4C

The conformance claim shall be consistent with the extended components definition.

ASE_CCL.1.5C

The conformance claim shall identify a PP-Configuration, or all PPs and security requirement packages to which the ST claims conformance.

ASE_CCL.1.6C

The conformance claim shall describe any conformance of the ST to a package as either package-conformant or package-augmented.

ASE_CCL.1.7C

The conformance claim shall describe any conformance of the ST to a PP as PP-Conformant.

ASE_CCL.1.8C

The conformance claim rationale shall demonstrate that the TOE type is consistent with the TOE type in the PP-Configuration or PPs for which conformance is being claimed.

ASE_CCL.1.9C

The conformance claim rationale shall demonstrate that the statement of the security problem definition is consistent with the statement of the security problem definition in the PP-Configuration, PPs and any functional packages for which conformance is being claimed.

ASE_CCL.1.10C

The conformance claim rationale shall demonstrate that the statement of security objectives is consistent with the statement of security objectives in the PP-Configuration, PPs, and any functional package for which conformance is being claimed.

ASE_CCL.1.11C

The conformance claim rationale shall demonstrate that the statement of security requirements is consistent with the statement of security requirements in the PP-Configuration, PPs, and any functional packages for which conformance is being claimed.

ASE_CCL.1.12C

The conformance claim for PP(s) or a PP-Configuration shall be exact, strict, or demonstrable or a list of conformance types.

ASE_CCL.1.13C

If the conformance claim identifies a set of Evaluation methods and Evaluation activities derived from CEM work units that shall be used to evaluate the TOE then

this set shall include all those that are included in any package, PP, or PP-Module in a PP-Configuration to which the ST claims conformance, and no others.

Evaluator action elements:

ASE_CCL.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ASE_ECD.1 Extended Components Definition

Developer action elements:

ASE_ECD.1.1D

The developer shall provide a statement of security requirements.

ASE_ECD.1.2D

The developer shall provide an extended components definition.

Content and presentation elements:

ASE_ECD.1.1C

The statement of security requirements shall identify all extended security requirements.

ASE_ECD.1.2C

The extended components definition shall define an extended component for each extended security requirement.

ASE_ECD.1.3C

The extended components definition shall describe how each extended component is related to the existing CC components, families, and classes.

ASE_ECD.1.4C

The extended components definition shall use the existing CC components, families, classes, and methodology as a model for presentation.

ASE_ECD.1.5C

The extended components shall consist of measurable and objective elements such that conformance or nonconformance to these elements may be demonstrated.

Evaluator action elements:

ASE_ECD.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ASE_ECD.1.2E

The evaluator shall confirm that no extended component may be clearly expressed using existing components.

ASE_INT.1 ST Introduction

Developer action elements:

ASE_INT.1.1D

The developer shall provide an ST introduction.

Content and presentation elements:

ASE_INT.1.1C

The ST introduction shall contain an ST reference, a TOE reference, a TOE overview and a TOE description.

ASE_INT.1.2C

The ST reference shall uniquely identify the ST.

ASE_INT.1.3C

The TOE reference shall uniquely identify the TOE.

ASE_INT.1.4C

The TOE overview shall summarize the usage and major security features of the TOE.

ASE_INT.1.5C

The TOE overview shall identify the TOE type.

ASE_INT.1.6C

The TOE overview shall identify any non-TOE hardware/software/firmware required by the TOE.

ASE_INT.1.7C

For a multi-assurance ST, the TOE overview shall describe the TSF organization in terms of the sub-TSFs defined in the PP-Configuration the ST claims conformance to.

ASE_INT.1.8C

The TOE description shall describe the physical scope of the TOE.

ASE_INT.1.9C

The TOE description shall describe the logical scope of the TOE.

Evaluator action elements:

ASE_INT.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ASE_INT.1.2E

The evaluator shall confirm that the TOE reference, the TOE overview, and the TOE description are consistent with each other.

ASE_OBJ.1 ST Objectives for the Operational Environment

Developer action elements:

ASE_OBJ.1.1D

The developer shall provide a statement of security objectives for the operational environment.

ASE_OBJ.1.2D

The developer shall provide a security objectives rationale for the operational environment.

Content and presentation elements:

ASE_OBJ.1.1C

The statement of security objectives shall describe the security objectives for the operational environment.

ASE_OBJ.1.2C

The security objectives rationale shall trace each security objective for the operational environment back to threats countered by that security objective, OSPs enforced by that security objective, and assumptions upheld by that security objective.

ASE_OBJ.1.3C

The security objectives rationale shall demonstrate that the security objectives for the operational environment uphold all assumptions.

Evaluator action elements:

ASE_OBJ.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ASE_REQ.1 Stated Security Requirements

Developer action elements:

ASE_REQ.1.1D

The developer shall provide a statement of security requirements.

ASE_REQ.1.2D

The developer shall provide a security requirements rationale.

Content and presentation elements:

ASE_REQ.1.1C

The statement of security requirements shall describe the SFRs and the SARs.

ASE_REQ.1.2C

For a single-assurance ST, the statement of security requirements shall define the global set of SARs that apply to the entire TOE. The sets of SARs shall be consistent with the PPs or PP-Configuration to which the ST claims conformance.

ASE_REQ.1.3C

For a multi-assurance ST, the statement of security requirements shall define the global set of SARs that apply to the entire TOE and the sets of SARs that apply to each sub-TSF. The sets of SARs shall be consistent with the multi-assurance PP-Configuration to which the ST claims conformance.

ASE_REQ.1.4C

All subjects, objects, operations, security attributes, external entities and other terms that are used in the SFRs and the SARs shall be defined.

ASE_REQ.1.5C

The statement of security requirements shall identify all operations on the security requirements.

ASE_REQ.1.6C

All operations shall be performed correctly.

ASE_REQ.1.7C

Each dependency of the security requirements shall either be satisfied, or the security requirements rationale shall justify the dependency not being satisfied.

ASE_REQ.1.8C

The security requirements rationale shall demonstrate that the SFRs (in conjunction with the security objectives for the environment) counter all threats for the TOE.

ASE_REQ.1.9C

The security requirements rationale shall demonstrate that the SFRs (in conjunction with the security objectives for the environment) enforce all OSPs.

ASE_REQ.1.10C

The security requirements rationale shall explain why the SARs were chosen.

ASE_REQ.1.11C

The statement of security requirements shall be internally consistent.

ASE_REQ.1.12C

If the ST defines sets of SARs that expand the sets of SARs of the PPs or PP-Configuration it claims conformance to, the security requirements rationale shall include an assurance rationale that justifies the consistency of the extension and provides a rationale for the disposition of any Evaluation methods and Evaluation activities identified in the conformance statement that are affected by the extension of the sets of SARs

Evaluator action elements:

ASE_REQ.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ASE_TSS.1 TOE Summary Specification

Developer action elements:

ASE_TSS.1.1D

The developer shall provide a TOE summary specification.

Content and presentation elements:

ASE_TSS.1.1C

The TOE summary specification shall describe how the TOE meets each SFR.

Evaluator action elements:

ASE_TSS.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ASE_TSS.1.2E

The evaluator shall confirm that the TOE summary specification is consistent with the TOE overview and the TOE description.

5.2.5 Class ATE: Tests

Testing is specified for functional aspects of the system as well as aspects that take advantage of design or implementation weaknesses. The former is done through the ATE_IND family, while the latter is through the AVA_VAN family. At the assurance level specified in this PP, testing is based on advertised functionality and interfaces with dependency on the availability of design information. One of the primary outputs of the evaluation process is the test report as specified in the following requirements.

ATE_IND.1 Independent Testing – Conformance (ATE_IND.1)

Testing is performed to confirm the functionality described in the TSS as well as the administrative (including configuration and operational) documentation provided. The focus of the testing is to confirm that the requirements specified in Section 5.1 Security Functional Requirements being met, although some additional testing is specified for SARs in Section 5.2 Security Assurance Requirements. The evaluation activities identify the additional testing activities associated with these components. The evaluator produces a test report documenting the plan for and results of testing, as well as coverage arguments focused on the platform/TOE combinations that are claiming conformance to this PP. Given the scope of the TOE and its associated evaluation evidence requirements, this component's evaluation activities are covered by the evaluation activities listed for ALC_CMC.1.

Developer action elements:

ATE_IND.1.1D

The developer shall provide the TOE for testing.

Content and presentation elements:

ATE_IND.1.1C

The TOE shall be suitable for testing.

Evaluator action elements:

ATE_IND.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ATE_IND.1.2E

The evaluator shall test a subset of the TSF to confirm that the TSF operates as specified.

Application Note: The evaluator will test the OS on the most current fully patched version of the platform.

Evaluation Activities

ATE_IND.1
The evaluator will prepare a test plan and report documenting the testing aspects of the system, including any application crashes during testing. The evaluator shall determine the root cause of any application crashes and include that information in the report. The test plan covers all of the testing actions contained in the [CEM] and the body of this PP's Assurance Activities.
While it is not necessary to have one test case per test listed in an Assurance Activity, the evaluator must document in the test plan that each applicable testing requirement in the ST is covered. The test plan identifies the platforms to be tested, and for those platforms not included in the test plan but included in the ST, the test plan provides a justification for not testing the platforms. This

justification must address the differences between the tested platforms and the untested platforms, and make an argument that the differences do not affect the testing to be performed. It is not sufficient to merely assert that the differences have no affect; rationale must be provided. If all platforms claimed in the ST are tested, then no rationale is necessary. The test plan describes the composition of each platform to be tested, and any setup that is necessary beyond what is contained in the AGD documentation. It should be noted that the evaluator is expected to follow the AGD documentation for installation and setup of each platform either as part of a test or as a standard pre-test condition. This may include special test drivers or tools. For each driver or tool, an argument (not just an assertion) should be provided that the driver or tool will not adversely affect the performance of the functionality by the OS and its platform.
This also includes the configuration of the cryptographic engine to be used. The cryptographic algorithms implemented by this engine are those specified by this PP and used by the cryptographic protocols being evaluated (IPsec, TLS). The test plan identifies high-level test objectives as well as the test procedures to be followed to achieve those objectives. These procedures include expected results.
The test report (which could just be an annotated version of the test plan) details the activities that took place when the test procedures were executed, and includes the actual results of the tests. This shall be a cumulative account, so if there was a test run that resulted in a failure; a fix installed; and then a successful re-run of the test, the report would show a "fail" and "pass" result (and the supporting details), and not just the "pass" result.

5.2.6 Class AVA: Vulnerability Assessment

For the current generation of this protection profile, the evaluation lab is expected to survey open sources to discover what vulnerabilities have been discovered in these types of products. In most cases, these vulnerabilities will require sophistication beyond that of a basic attacker. Until penetration tools are created and uniformly distributed to the evaluation labs, the evaluator will not be expected to test for these vulnerabilities in the TOE. The labs will be expected to comment on the likelihood of these vulnerabilities given the documentation provided by the vendor. This information will be used in the development of penetration testing tools and for the development of future protection profiles.

AVA_VAN.1 Vulnerability Survey (AVA_VAN.1)

Developer action elements:

AVA_VAN.1.1D

The developer shall provide the TOE for testing.

Content and presentation elements:

AVA_VAN.1.1C

The TOE shall be suitable for testing.

Evaluator action elements:

AVA_VAN.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

AVA_VAN.1.2E

The evaluator shall perform a search of public domain sources to identify potential vulnerabilities in the TOE.

AVA_VAN.1.3E

The evaluator shall conduct penetration testing, based on the identified potential vulnerabilities, to determine that the TOE is resistant to attacks performed by an attacker possessing Basic attack potential.

Appendix A - Implementation-dependent Requirements

Implementation-dependent Requirements Appendix defines requirements that must be claimed in the ST if the TOE implements particular product features. For this technology type, the following product features require the claiming of additional SFRs:

A.0.1 Password-based remote administration

If the TOE provides remote administration using a password-based authentication mechanism, certain SFRs in this PP must be included.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FIA_AFL.1**

A.0.2 Password-based local authentication

If the TOE provides a password-based local authentication mechanism, certain SFRs in this PP must be included.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FIA_UAU.7**

A.0.3 Password-based authentication

If the TOE provides a password-based authentication mechanism, certain SFRs in this PP must be included.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FIA_PMG_EXT.1**
- **FPT_APW_EXT.1**

A.0.4 Pre-shared keys for IPsec

The TOE may support pre-shared keys for use in the IPsec protocol that conform to RFC 8784. If so, certain SFRs in this PP must be included.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FIA_PSK_EXT.1**

A.0.5 Admin management of crypto keys

True if cryptographic keys can be managed (e.g., modified, deleted or generated/imported) by the Security Administrator.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FMT_MTD.1/CryptoKeys**

A.0.6 Local security admin

True if the TOE provides the Security Administrator the ability to administer the TOE locally.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FTA_SSL_EXT.1**

A.0.7 Distributed TOE

True if the TOE is implemented as a Distributed TOE as described in the Distributed TOE section of this cPP.
If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

Appendix B - Entropy Documentation and Assessment

blah

Appendix C - Glossary

C.1 Terms

The following sections list Common Criteria and technology terms used in this document.

C.1.1 Common Criteria Terms

Assurance	Grounds for confidence that a TOE meets the SFRs [CC].
Base Protection Profile (Base-PP)	Protection Profile used as a basis to build a PP-Configuration.
Collaborative Protection Profile (cPP)	A Protection Profile developed by international technical communities and approved by multiple schemes.
Common Criteria (CC)	Common Criteria for Information Technology Security Evaluation (International Standard ISO/IEC 15408).
Common Criteria Testing Laboratory	Within the context of the Common Criteria Evaluation and Validation Scheme (CCEVS), an IT security evaluation facility accredited by the National Voluntary Laboratory Accreditation Program (NVLAP) and approved by the NIAP Validation Body to conduct Common Criteria-based evaluations.
Common Evaluation Methodology (CEM)	Common Evaluation Methodology for Information Technology Security Evaluation.
Direct Rationale	A type of Protection Profile, PP-Module, or Security Target in which the security problem definition (SPD) elements are mapped directly to the SFRs and possibly to the security objectives for the operational environment. There are no security objectives for the TOE.
Distributed TOE	A TOE composed of multiple components operating as a logical whole.
Extended Package (EP)	A deprecated document form for collecting SFRs that implement a particular protocol, technology, or functionality. See Functional Packages.
Functional Package (FP)	A document that collects SFRs for a particular protocol, technology, or functionality.
Operational Environment (OE)	Hardware and software that are outside the TOE boundary that support the TOE functionality and security policy.
Protection Profile (PP)	An implementation-independent set of security requirements for a category of products.

Protection Profile Configuration (PP-Configuration)	A comprehensive set of security requirements for a product type that consists of at least one Base-PP and at least one PP-Module.
Protection Profile Module (PP-Module)	An implementation-independent statement of security needs for a TOE type complementary to one or more Base-PPs.
Security Assurance Requirement (SAR)	A requirement to assure the security of the TOE.
Security Functional Requirement (SFR)	A requirement for security enforcement by the TOE.
Security Target (ST)	A set of implementation-dependent security requirements for a specific product.
Target of Evaluation (TOE)	The product under evaluation.
TOE Security Functionality (TSF)	The security functionality of the product under evaluation.
TOE Summary Specification (TSS)	A description of how a TOE satisfies the SFRs in an ST.

C.1.2 Technical Terms

Address Space Layout Randomization (ASLR)	An anti-exploitation feature which loads memory mappings into unpredictable locations. ASLR makes it more difficult for an attacker to redirect control to code that they have introduced into the address space of a process.
Administrator	An administrator is responsible for management activities, including setting policies that are applied by the enterprise on the operating system. This administrator could be acting remotely through a management server, from which the system receives configuration policies. An administrator can enforce settings on the system which cannot be overridden by non-administrator users.
Application (app)	Software that runs on a platform and performs tasks on behalf of the user or owner of the platform, as well as its supporting documentation.
Application Programming Interface (API)	A specification of routines, data structures, object classes, and variables that allows an application to make use of services provided by another software component, such as a library. APIs are often provided for a set of libraries included with the platform.
Credential	Data that establishes the identity of a user, e.g. a cryptographic key or password.
Critical Security Parameters (CSP)	Information that is either user or system defined and is used to operate a cryptographic module in processing encryption functions including cryptographic keys and authentication data, such as passwords, the disclosure or modification of which can compromise the security of a cryptographic module or the security of the information protected by the module.
DAR Protection	Countermeasures that prevent attackers, even those with physical access, from extracting data from non-volatile storage. Common techniques include data encryption and wiping.

Data Execution Prevention (DEP)	An anti-exploitation feature of modern operating systems executing on modern computer hardware, which enforces a non-execute permission on pages of memory. DEP prevents pages of memory from containing both data and instructions, which makes it more difficult for an attacker to introduce and execute code.
Developer	An entity that writes OS software. For the purposes of this document, vendors and developers are the same.
General Purpose Operating System	A class of OSes designed to support a wide-variety of workloads consisting of many concurrent applications or services. Typical characteristics for OSes in this class include support for third-party applications, support for multiple users, and security separation between users and their respective resources. General Purpose Operating Systems also lack the real-time constraint that defines Real Time Operating Systems (RTOS). RTOSes typically power routers, switches, and embedded devices.
Host-based Firewall	A software-based firewall implementation running on the OS for filtering inbound and outbound network traffic to and from processes running on the OS.
Operating System (OS)	Software that manages physical and logical resources and provides services for applications. The terms *TOE* and *OS* are interchangeable in this document.
Personally Identifiable Information (PII)	Any information about an individual maintained by an agency, including, but not limited to, education, financial transactions, medical history, and criminal or employment history and information which can be used to distinguish or trace an individual's identity, such as their name, social security number, date and place of birth, mother's maiden name, biometric records, etc., including any other personal information which is linked or linkable to an individual. [OMB]
Sensitive Data	Sensitive data may include all user or enterprise data or may be specific application data such as PII, emails, messaging, documents, calendar items, and contacts. Sensitive data must minimally include credentials and keys. Sensitive data shall be identified in the OS's TSS by the ST author.
User	A user is subject to configuration policies applied to the operating system by administrators. On some systems under certain configurations, a normal user can temporarily elevate privileges to that of an administrator. At that time, such a user should be considered an administrator.
Virtual Machine (VM)	Blah Blah Blah

Appendix D - Acronyms

Table 15: Acronyms

Acronym	Meaning
AES	Advanced Encryption Standard
API	Application Programming Interface
API	Application Programming Interface
app	Application
ASLR	Address Space Layout Randomization
Base-PP	Base Protection Profile
CC	Common Criteria
CEM	Common Evaluation Methodology
CESG	Communications-Electronics Security Group
CMC	Certificate Management over CMS
CMS	Cryptographic Message Syntax
CN	Common Names
cPP	Collaborative Protection Profile
CRL	Certificate Revocation List
CSA	Computer Security Act
CSP	Critical Security Parameters
DAR	Data At Rest
DEP	Data Execution Prevention
DES	Data Encryption Standard
DHE	Diffie-Hellman Ephemeral
DNS	Domain Name System
DRBG	Deterministic Random Bit Generator
DSS	Digital Signature Standard

DSS	Digital Signature Standard
DT	Date/Time Vector
DTLS	Datagram Transport Layer Security
EAP	Extensible Authentication Protocol
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
ECDSA	Elliptic Curve Digital Signature Algorithm
EP	Extended Package
EST	Enrollment over Secure Transport
FIPS	Federal Information Processing Standards
FP	Functional Package
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IETF	Internet Engineering Task Force
IP	Internet Protocol
ISO	International Organization for Standardization
IT	Information Technology
ITSEF	Information Technology Security Evaluation Facility
NIAP	National Information Assurance Partnership
NIST	National Institute of Standards and Technology
OCSP	Online Certificate Status Protocol
OE	Operational Environment
OID	Object Identifier
OMB	Office of Management and Budget
OS	Operating System
PII	Personally Identifiable Information
PKI	Public Key Infrastructure
PP	Protection Profile

PP	Protection Profile
PP-Configuration	Protection Profile Configuration
PP-Module	Protection Profile Module
RBG	Random Bit Generator
RFC	Request for Comment
RNG	Random Number Generator
RNGVS	Random Number Generator Validation System
S/MIME	Secure/Multi-purpose Internet Mail Extensions
SAN	Subject Alternative Name
SAR	Security Assurance Requirement
SFR	Security Functional Requirement
SHA	Secure Hash Algorithm
SIP	Session Initiation Protocol
ST	Security Target
SWID	Software Identification
TLS	Transport Layer Security
TOE	Target of Evaluation
TSF	TOE Security Functionality
TSFI	TSF Interface
TSS	TOE Summary Specification
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
USB	Universal Serial Bus
VM	Virtual Machine
XCCDF	eXtensible Configuration Checklist Description Format
XOR	Exclusive Or

Appendix E - Bibliography

Table 16: Bibliography

Identifier	Title
[CC]	Common Criteria for Information Technology Security Evaluation - • Part 1: Introduction and general model, CCMB-2022-11-001, CC:2022, Revision 1, November 2022. • Part 2: Security functional requirements, CCMB-2022-11-002, CC:2022, Revision 1, November 2022. • Part 3: Security assurance requirements, CCMB-2022-11-003, CC:2022, Revision 1, November 2022. • Part 4: Framework for the specification of evaluation methods and activities, CCMB-2022-11-004, CC:2022, Revision 1, November 2022. • Part 5: Pre-defined packages of security requirements, CCMB-2022-11-005, CC:2022, Revision 1, November 2022.
[CEM]	Common Methodology for Information Technology Security Evaluation - • Evaluation methodology, CCMB-2022-11-006, CC:2022, Revision 1, November 2022.
[CEM]	Common Evaluation Methodology for Information Technology Security - Evaluation Methodology, CCMB-2012-09-004, Version 3.1, Revision 4, September 2012.
[CESG]	CESG - End User Devices Security and Configuration Guidance
[CSA]	Computer Security Act of 1987, H.R. 145, June 11, 1987.
[OMB]	Reporting Incidents Involving Personally Identifiable Information and Incorporating the Cost for Security in Agency Information Technology Investments, OMB M-06-19, July 12, 2006.